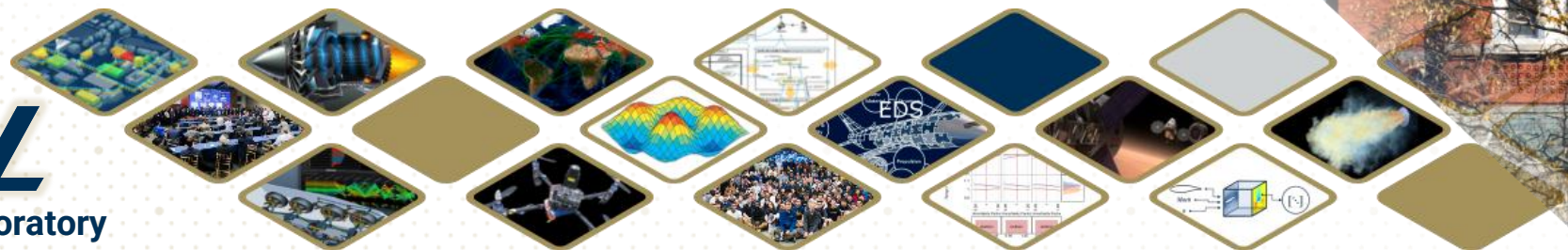
Unsupervised Online Subspace Adaptation for Dynamic Data-Driven Digital Twins

Paul Wang

PhD Thesis Defense

March 26th, 2026



Aerospace Systems Design Laboratory

Thesis Committee

Prof. Dimitri Mavris,
Advisor
School of Aerospace Engineering
Georgia Institute of Technology

Prof. Graeme Kennedy
School of Aerospace Engineering
Georgia Institute of Technology

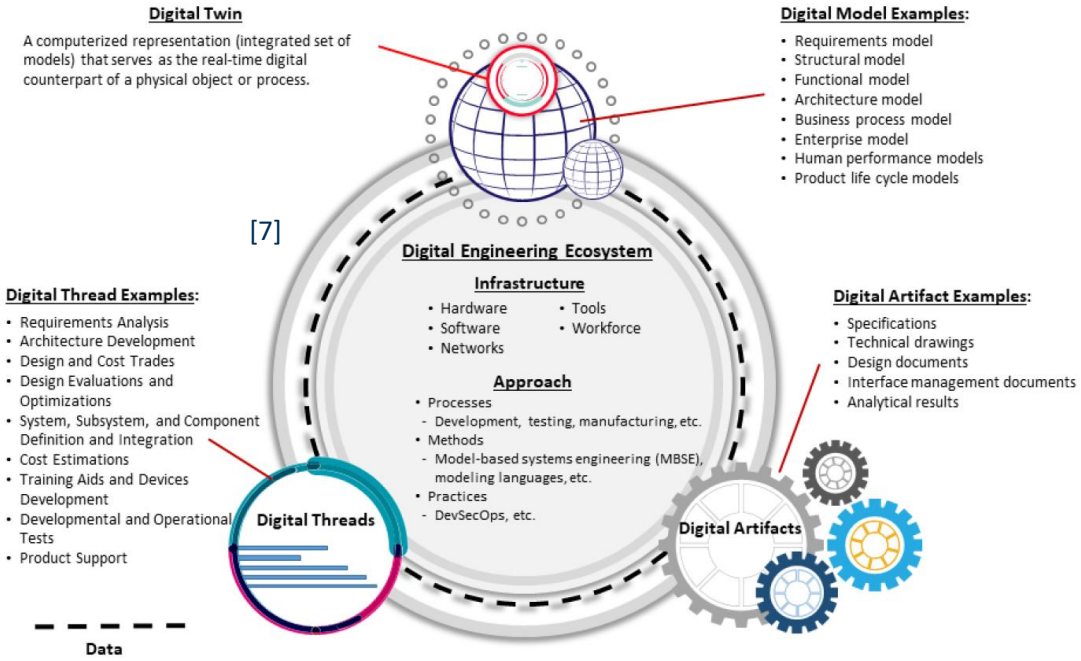
Prof. Kyriakos Vamvoudakis
School of Aerospace Engineering
Georgia Institute of Technology

Dr. Olivia Fischer,
School of Aerospace Engineering
Georgia Institute of Technology

Dr. Elisabeth Nguyen,
Electronic Programs Division/ Directorate L
The Aerospace Corporation

Digital Transformations

- **Engineered systems are subject to ever-changing operating conditions, contexts, and environments**
  - Changes in terrain, atmosphere, forces, loads
  - Variation in system configuration (e.g., sensors, equipment, avionics)
- The growing complexity of systems makes it costly and difficult to physically test for all possible conditions and configurations [1]
- In response, engineering organizations are rushing to build M&S capabilities to reduce costs and testing, such as **digital twins** [2-7]



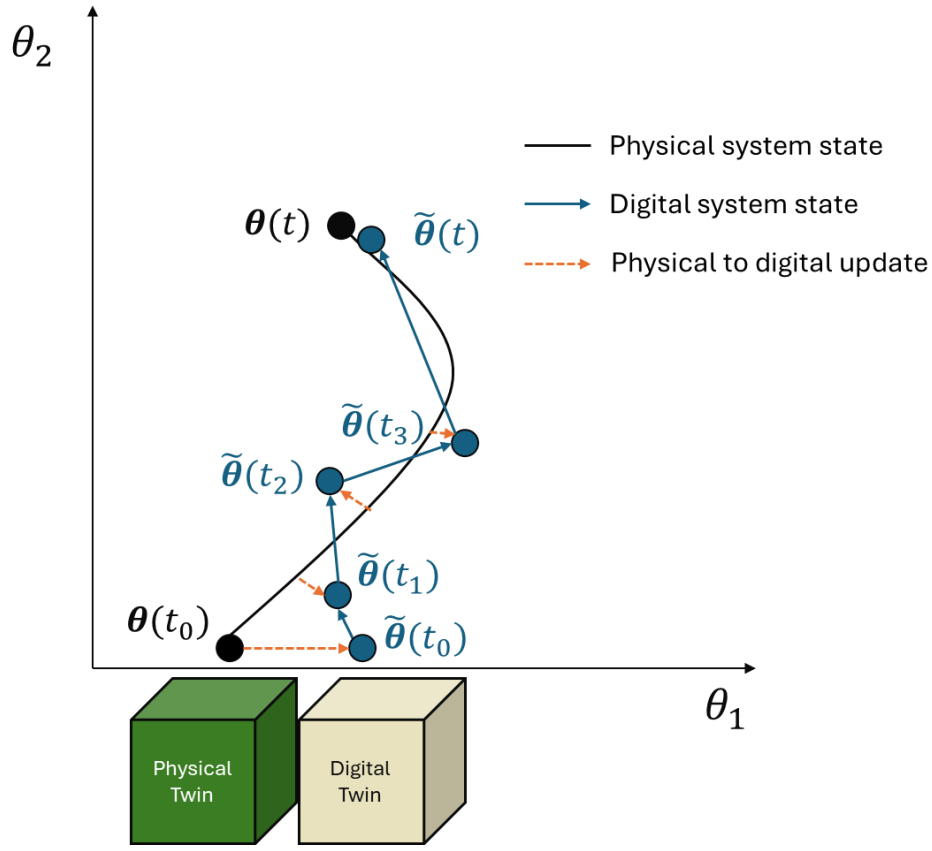
Digital Twins

Digital twins are virtual constructs that mimic a system and predict quantities of interest through computational models [1]

- Unlike a static simulation model, a digital twin continuously shares data with its physical twin
- Data feedback between twins keeps models up-to-date and accurate
- Predictions are used to inform activities such as prognostics, autonomous control, health monitoring, optimization, etc. [2-5]

A digital twin and its physical twin can be represented as dynamical systems

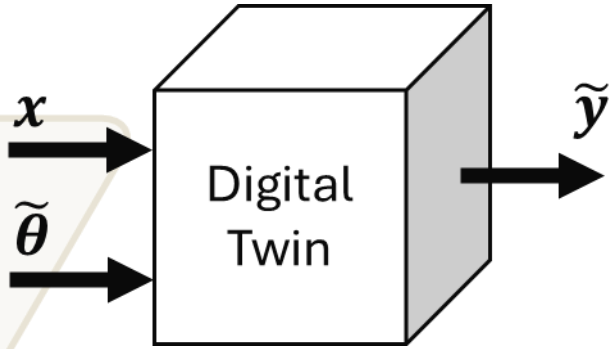
- Evolving, time-dependent model parameters
- Continuous updates *calibrate* the digital model
- Different modeling approaches are used depending on the twin's purpose



A physical system and its digital twin can be treated as dynamic systems with evolving parameters. Data from the physical twin is used to calibrate the digital twin parameters.

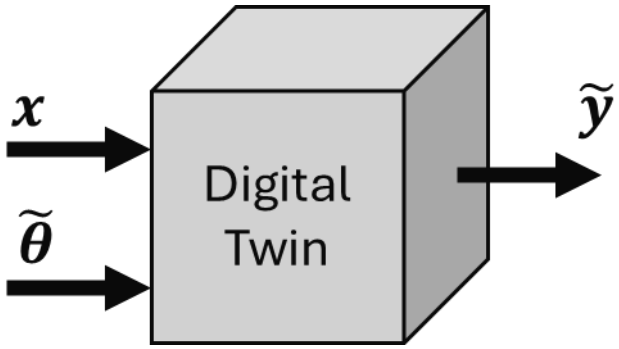
Modeling Digital Twins

$\boldsymbol{x}$: model input
$\tilde{\boldsymbol{\theta}}$: model parameters
$\tilde{\boldsymbol{y}}$: output response of interest



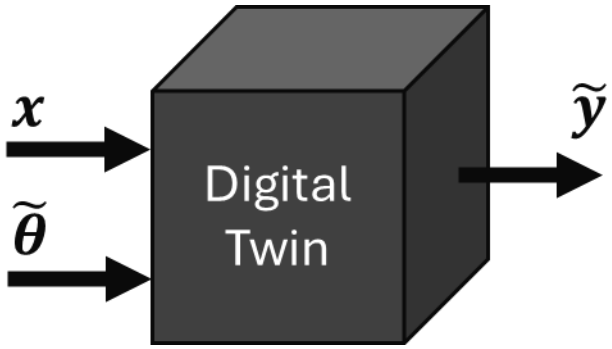
Purely Physics-Driven

- Models are created using only the physics of the problem
- Interpretable and generalizable, equations are independent of data
- Computationally intractable in large-scale, probabilistic systems (e.g., turbulence in multiscale resolution)
- **"White-box" approach**



Hybrid Twins

- Models are created by fusing physics and machine learning techniques (e.g., PINNs [1])
- Benefits/tradeoffs in evaluation speed, accuracy, interpretability over purely physics or data-driven approaches
- Complex to create
- **"Gray-box" approach**



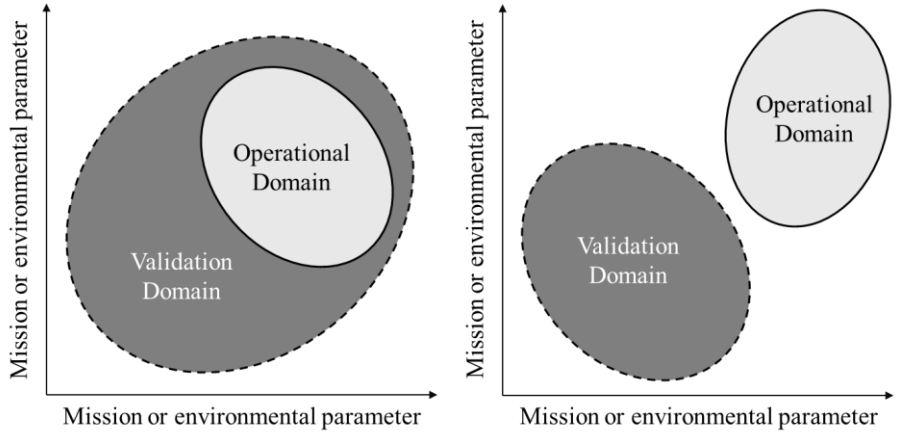
Purely Data Driven

- Models are created using only machine learning techniques on data
- Useful in situations where the exact governing physics are unknown (e.g., complex, large-scale, probabilistic systems)
- Requires sufficient data to accurately model the physical phenomena
- **"Black-box" approach**

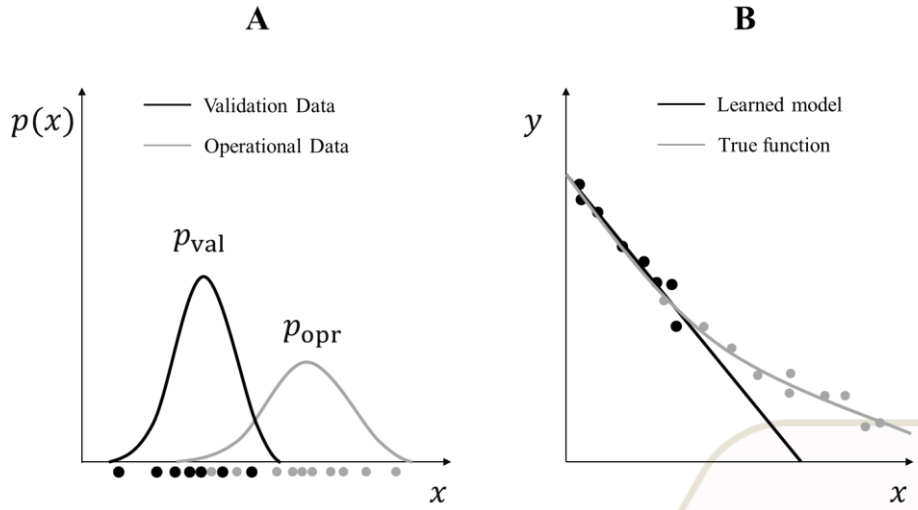
Researchers are increasingly leveraging data-driven approaches to model complex systems [1-3], so these models will be this presentation's focus

Mismatched Validation and Operational Domains

- Data-driven/hybrid models become inaccurate outside their **training domain** [1]
  - Models are trained on one set of data and are expected to generalize to other sets of data
  - If the training data is not representative of the true data encountered during operation, the model will be inaccurate
- It can be hard to know if the training data is truly representative of the operating domain
  - Uncertain deployment conditions or operating envelope
  - Undersampling
  - Barriers to data-gathering (costs, accessibility, resources)
- **Digital twins can calibrate on new data from the physical system during operation**
  - Calibration allows models to adapt to mismatches in domains



Mismatched validation (training) and operational (test) domains [2-4].



Domain shifts correspond to a difference in the *data distributions,* and the digital twin will predict poorly on *out-of-distribution data.*

Supervised Model Training/Calibration

Models take in an input and predict an output

$$y = f(x)$$

Offline Training

1) Gather training data $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{n}$
2) Minimize the model's error w.r.t. to its parameters

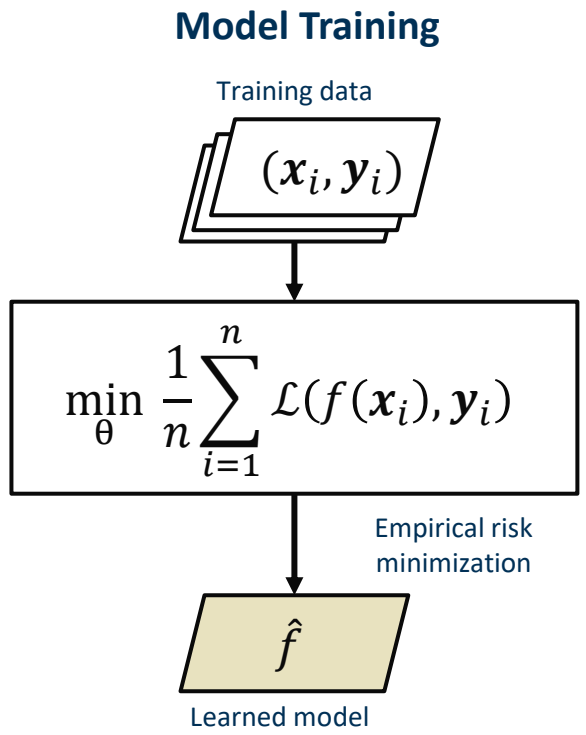
$$\min_{\theta} \hat{R}(f, p_{\text{val}}) = \min_{\theta} \frac{1}{n} \sum_{i=1}^{n} \mathcal{L}(f(\boldsymbol{x}_i), \boldsymbol{y}_i)$$

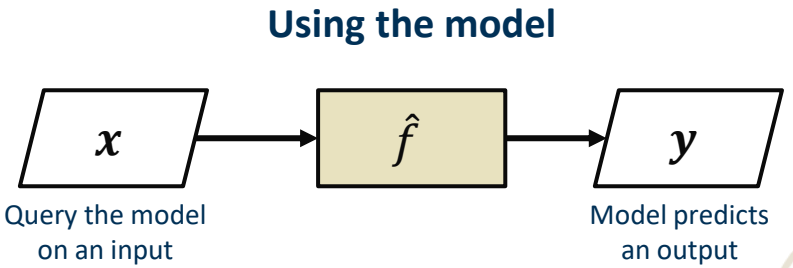
3) Use the model $\hat{f}$ for predictions

Online Training

1) Receive a new set of data $\{(\boldsymbol{x}_i, \boldsymbol{y}_i)\}_{i=1}^{m}$
2) Minimize model's error on combined set of data

This is called supervised learning, where the inputs and outputs are available for training/calibration



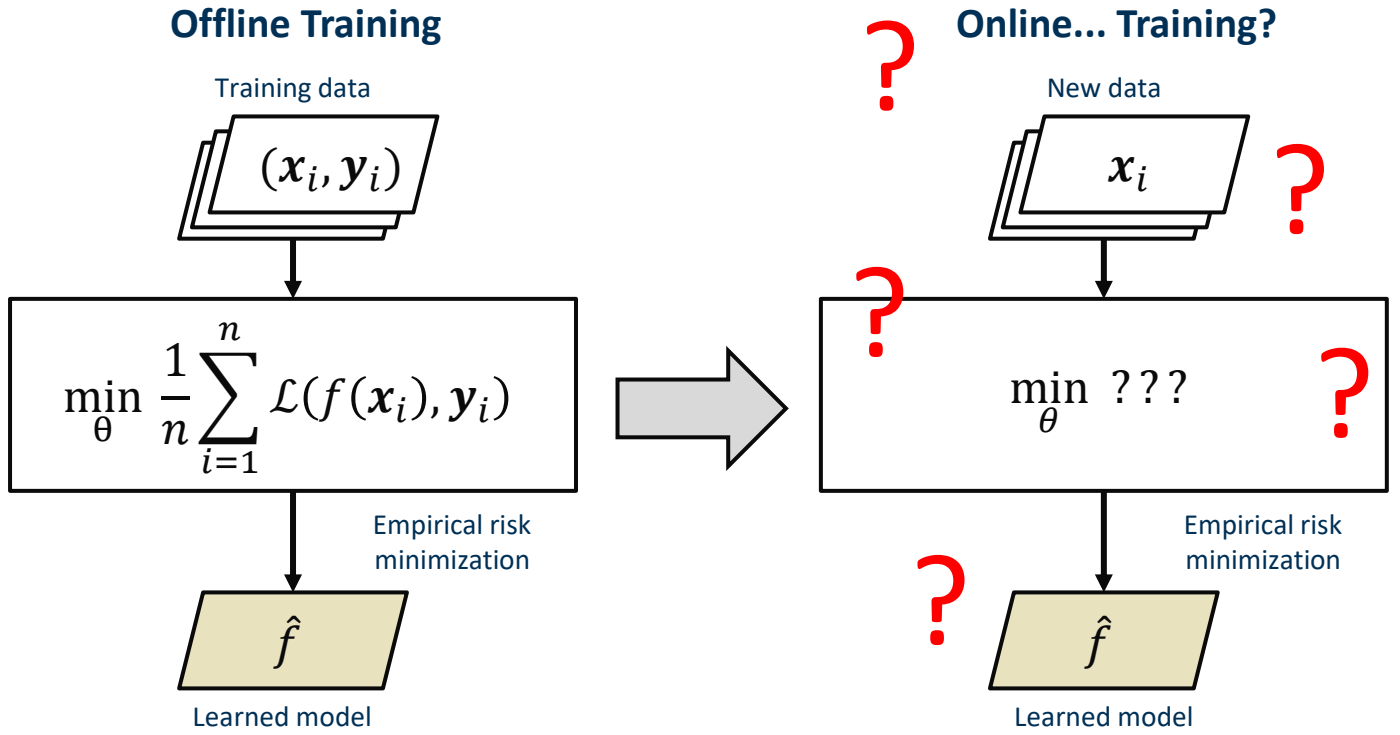


Limitations in Supervised Model Training/Calibration

Often, it is not possible to observe the *outputs during system deployment*

- Intentional design decision (e.g., lack of space for onboard compute, cost)
- Latency in communication between physical system and digital twin (e.g., uplink/downlink times and bandwidth)
- Output is a future state of the system (e.g., when does the system fail?)

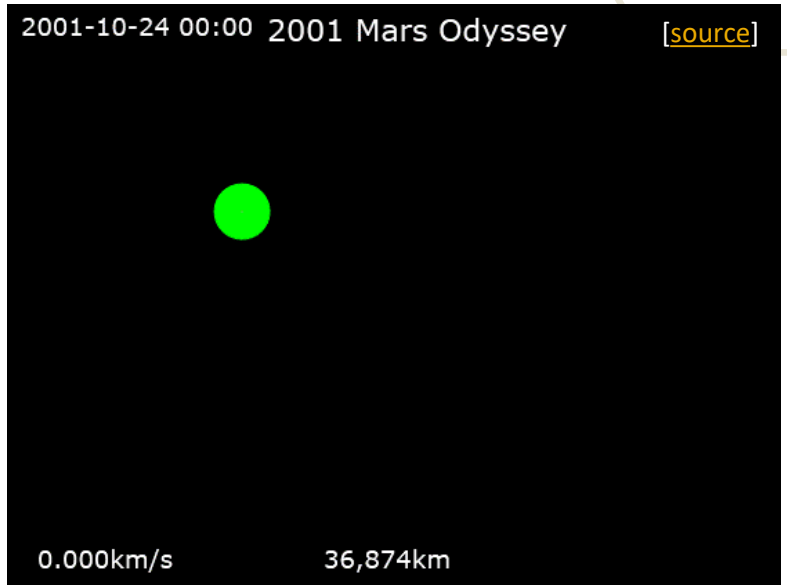
If the digital twin still receives input data, then **this is an unsupervised learning scenario, where only the inputs are available for training/calibration**

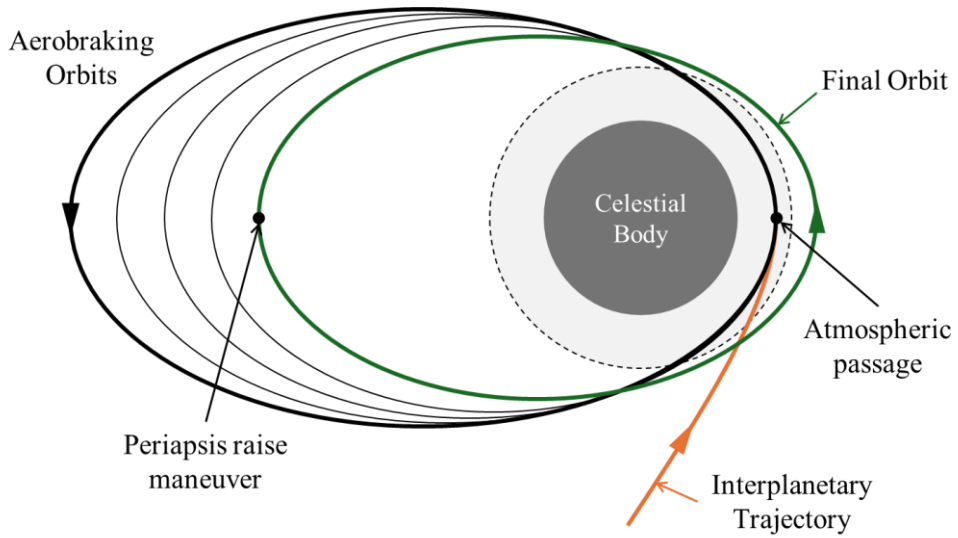


Let's go through an example!

A Motivating Example: Aerobraking

- Aerobraking is a spaceflight maneuver to insert a spacecraft into planetary orbit
  - Utilizes atmospheric drag to slow the spacecraft
  - Reduces propellant usage by leveraging drag instead of thrusters
  - Key technology for future spaceflight missions [1-2]
- Highly dynamic atmospheric environment:
  - Pressure, temperature, velocity, density in constant flux
  - Exact operating envelope is uncertain before AND during deployment [3-5]
- Huge operational costs:
  - Ground team, constant Deep Space Network coverage, around the clock monitoring for months
  - Vast distances in space make it difficult to send large amounts of sensor data back to Earth
- **Constant monitoring and orbit adjustment ensures that the spacecraft stays within an acceptable flight corridor (i.e., periapsis) to prevent the spacecraft from burning during reentry**

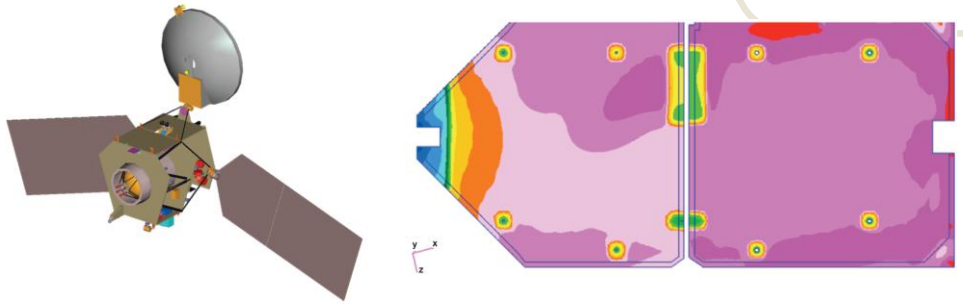




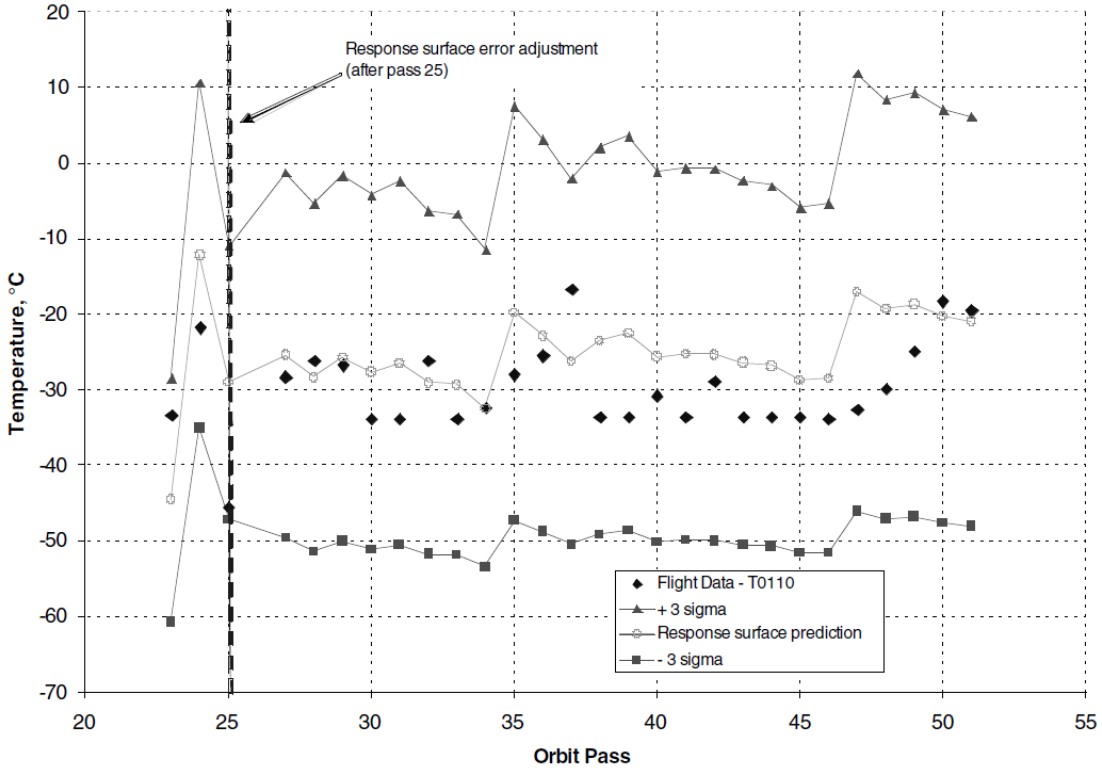
There exists a need for onboard models to enable *autonomous aerobraking*, reducing operational costs

Thermal Modeling

- Thermal models are used to predict aerodynamic heating, and the ground team conducts maneuvers to ensure temperature remains below a limit [1]
- Past and present missions use response surface equations to calculate the temperature on the spacecraft solar panels [2-4]
  - Data-driven models necessary for near real-time prediction [2,6]
- Existing models require multiple thermocouple measurements at several locations to adapt to in-flight conditions [2-5]
- **Having an unsupervised method of adapting the model would reduce the number of sensors required for calibration, saving costs on design and weight**



MRO in aerobraking configuration and solar array temperature in FEM, from [2].



Response surface temperature predictions, from [2].

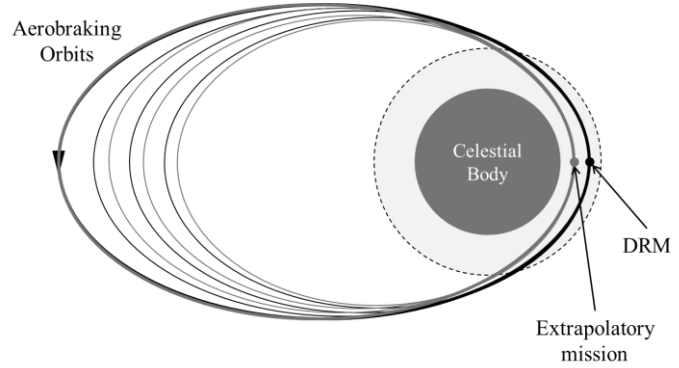
Problem Setup: Missions and Data

- Construct a digital twin to predict the heat rate applied on a reentry vehicle in atmospheric flight at Mars*
- Model f takes in atmospheric density, velocity**, and free-stream temperature as an inputs and outputs the heat rate, such that:

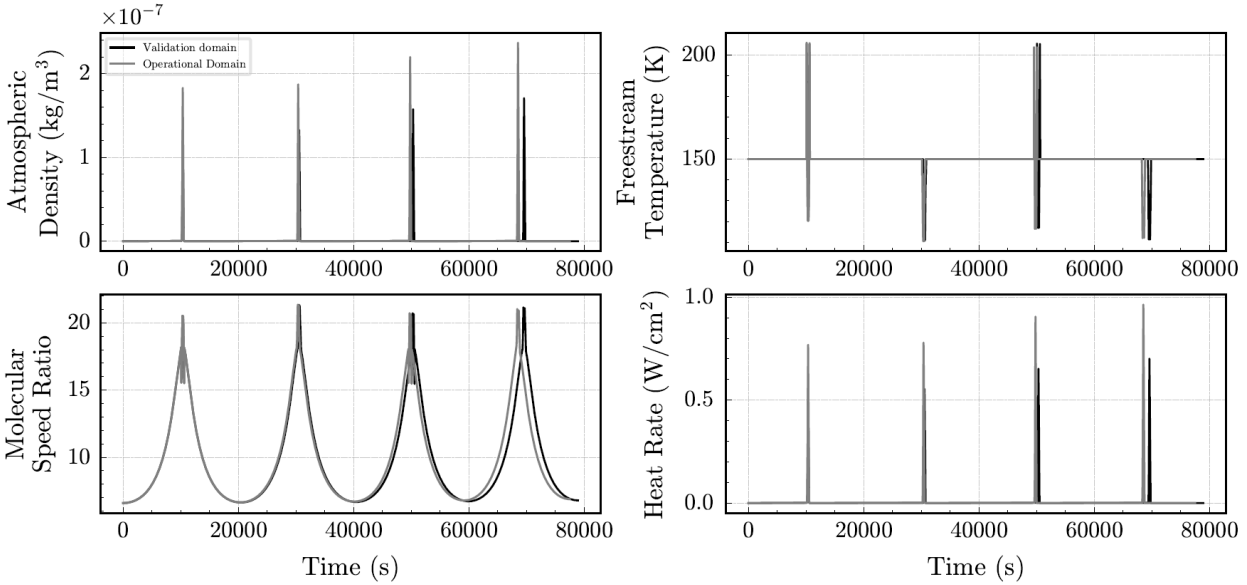
$$\dot{Q} = f\big(\rho(t), V(t), T(t)\big)$$

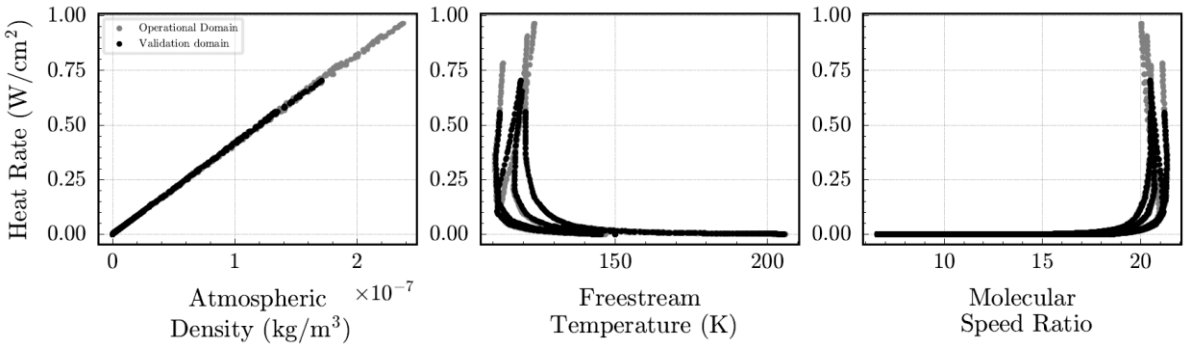
- Learn model from experiments/past mission
  - Inputs $\rho(t), V(t), T(t)$ are observable during deployment
  - Heat rate $\dot{Q}$ is unobservable during deployment
- **Model will be deployed on a new mission with lower periapsis to predict heat rate**

If we train the model on the DRM and deploy it on the new mission, how does it perform?



In this example, the DRM has an apoapsis of 12000 km and a periapsis of 97 km. The extrapolatory mission has the same apoapsis and a periapsis of 95 km. Data was generated using [1] and [2].



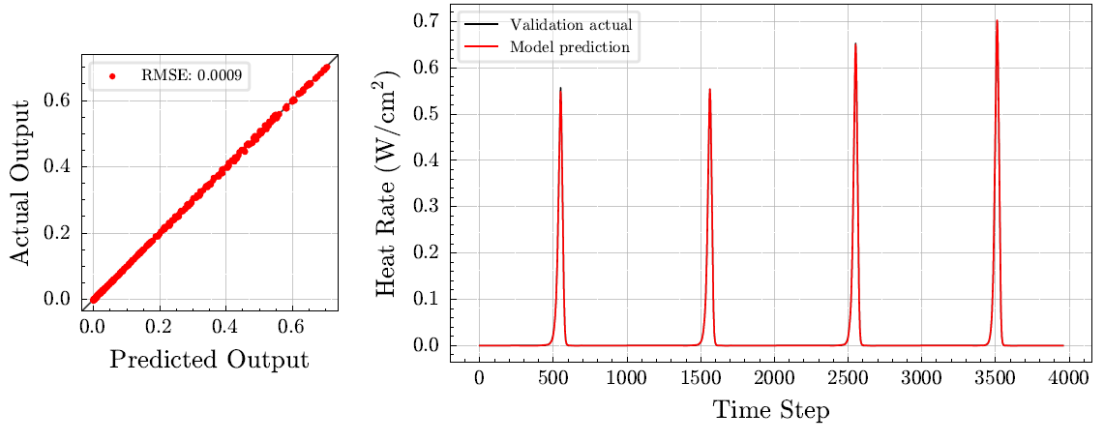


Georgia Tech Aerospace Systems Design Laboratory

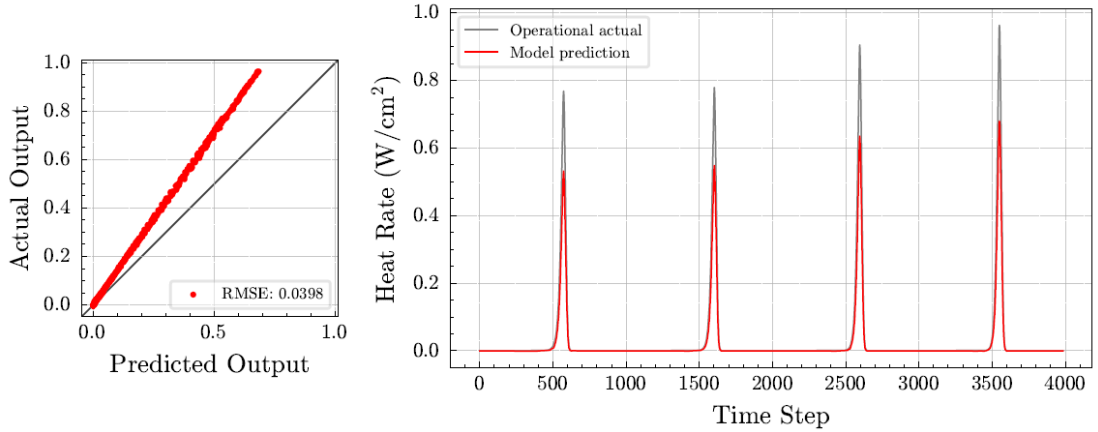
* Heat rate can be directly derived from atmospheric density and past missions used max heat rate as a constraint in place of temperature.
**The actual input to the model is molecular speed ratio S, which is calculated directly from velocity V ($S = V/\sqrt{2RT}$).

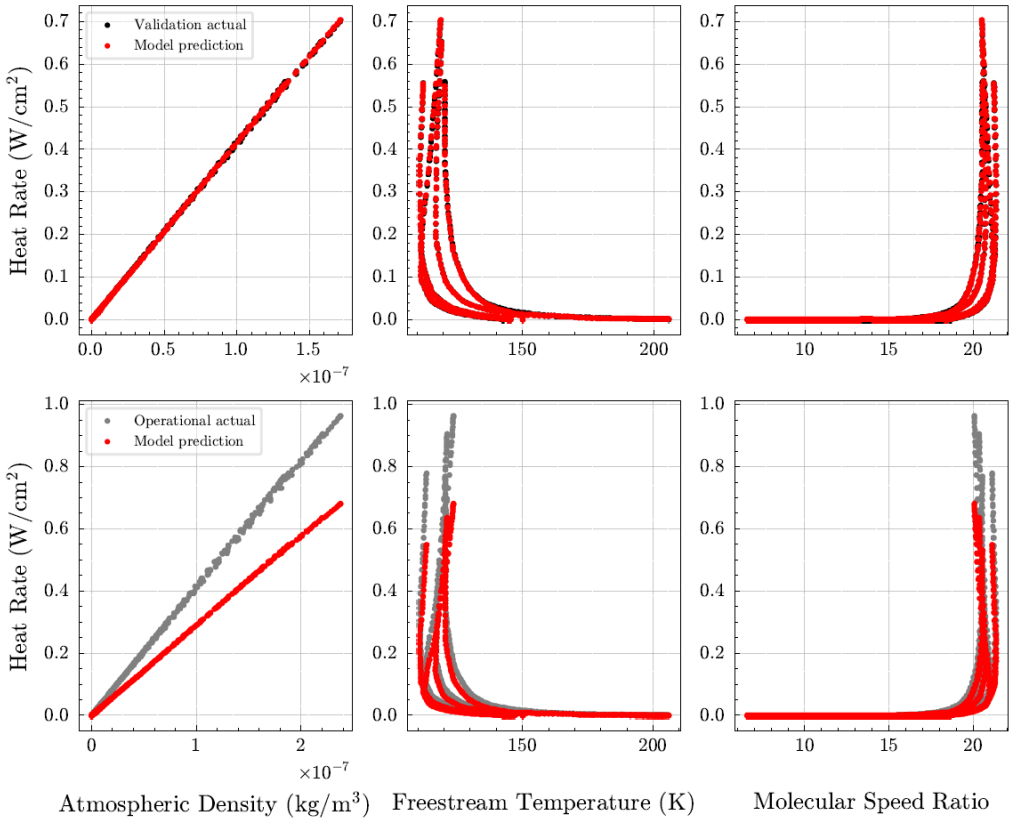
Problem Setup: Baseline Model

Design reference mission



Extrapolatory mission





- Training a model on just the DRM and deploying it on the extrapolatory mission yields inaccurate predictions
  - Model underpredicts the true heat rate
  - Recall that we cannot retrain the model without knowing the heat rate values during operation

How do we get our model to adapt to the new mission?

How do we get our model to adapt to the new mission?

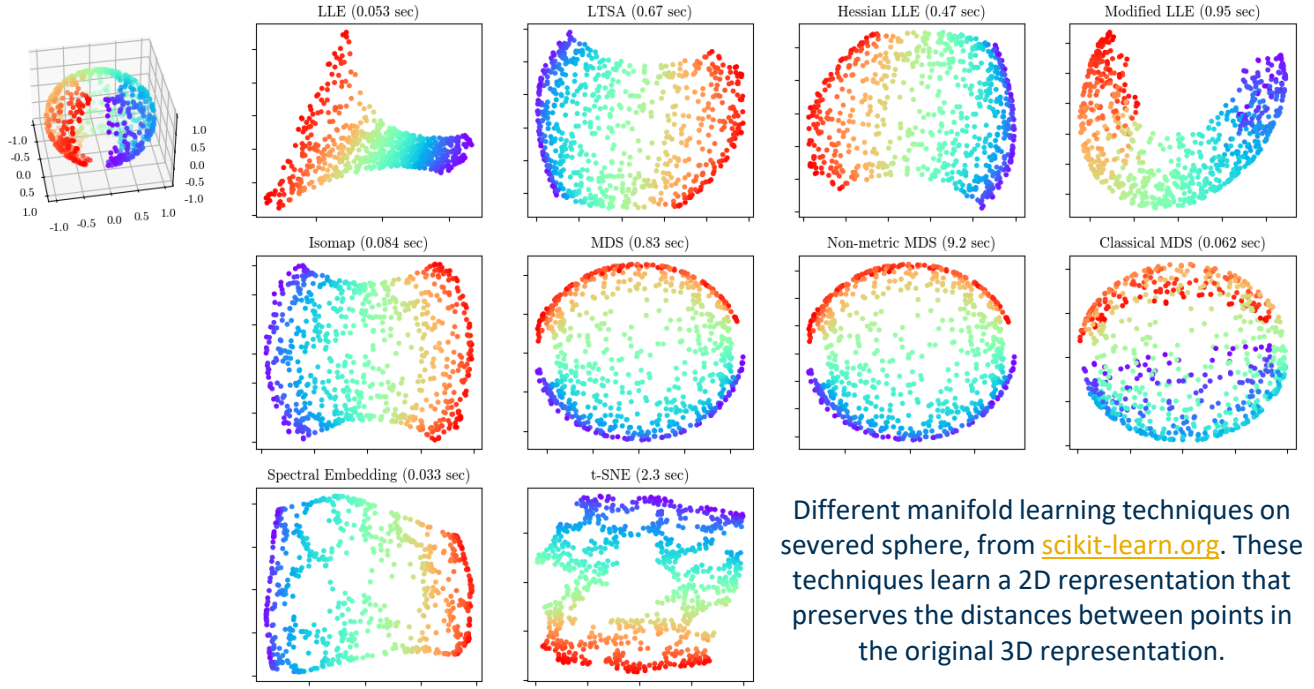
↓

Research Objective

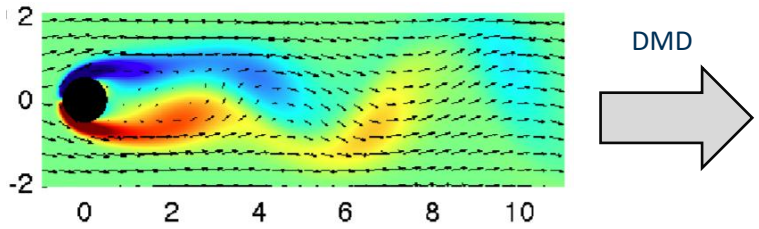
Develop an unsupervised method to allow a system digital twin to retain predictive performance on out-of-distribution data in evolving, unknown domains, such as changing environmental conditions, operating conditions, or configurations.

Learning Patterns in Input Data

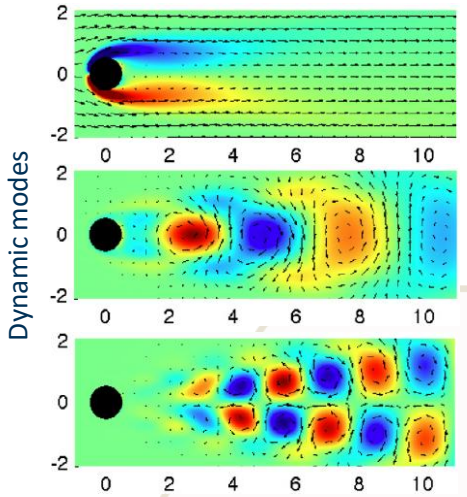
- Without access to the outputs, we can instead *learn underlying features within the inputs to adapt the model*
  - Features include dominant modes, coherent patterns in time, low-dimensional manifolds/subspaces
  - Ex: underlying flow structures in flow field
- If there exists common low-dimensional features within each mission's inputs, we can *align* these features from both missions to mitigate the extrapolation error
- This is called **unsupervised domain adaptation** [1-7], a type of transfer learning method



Different manifold learning techniques on severed sphere, from scikit-learn.org. These techniques learn a 2D representation that preserves the distances between points in the original 3D representation.



DMD



Vorticity and velocity field in cylinder flow, from [8].

Unsupervised Domain Adaptation

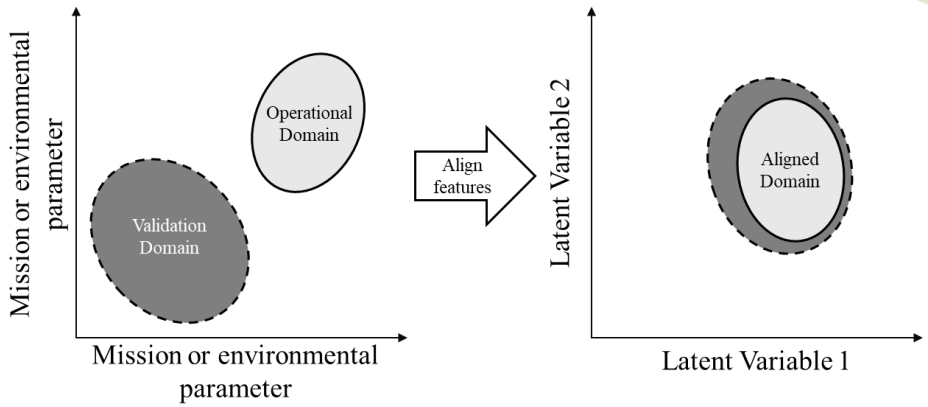
- A domain is a set of data points, their labels, and their probability distribution
  - DRM is the *validation* domain*, with sampled inputs $\boldsymbol{X}$ and corresponding outputs $\boldsymbol{Y}_S$

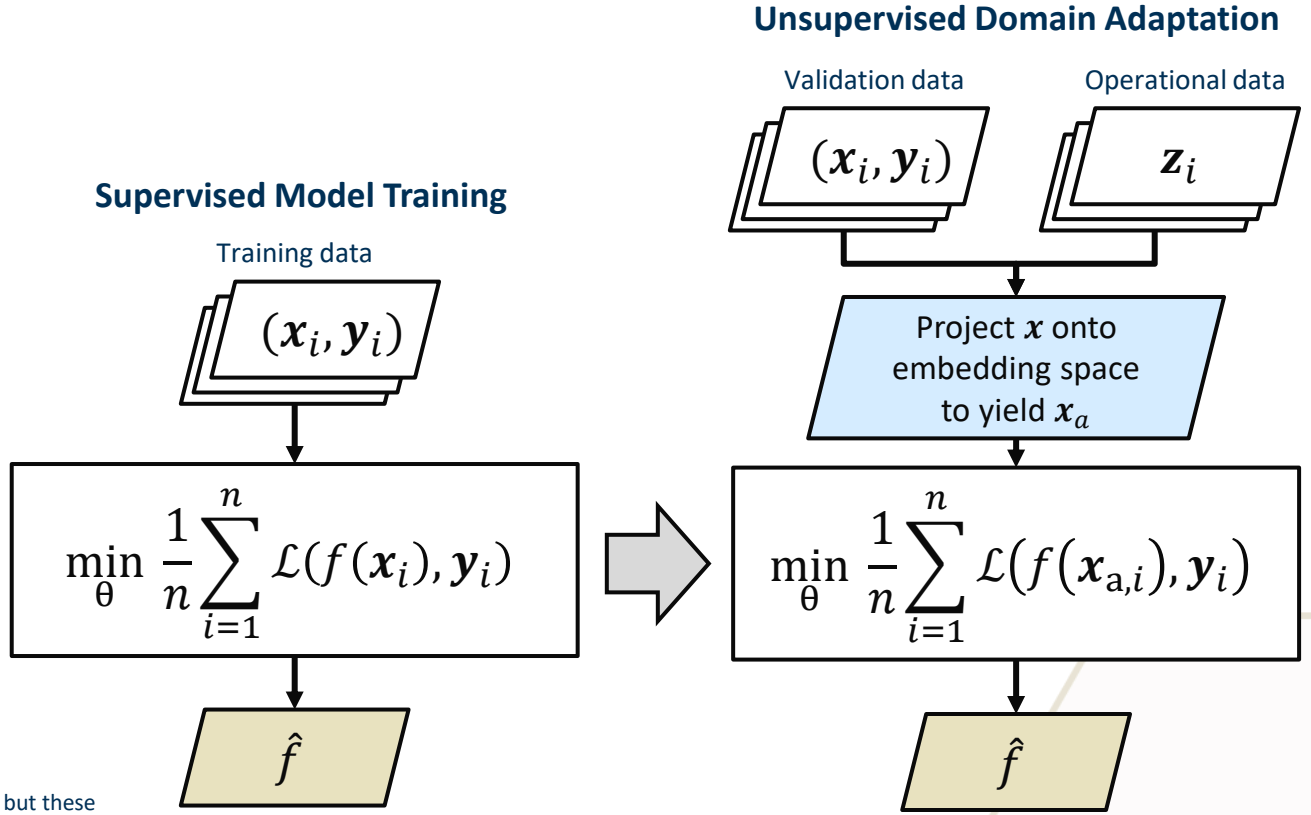
$$D_{\text{val}} = \{X, Y_S, p_{\text{val}}(X)\}$$

  - Extrapolatory mission is the *operational* domain, with sampled inputs $\boldsymbol{Z}$ and corresponding outputs $\boldsymbol{Y}_T$, where $p_{\text{opr}}(Z) \neq p_{\text{val}}(X)$

$$D_{\text{opr}} = \{Z, Y_T, p_{\text{opr}}(Z)\}$$

- The goal of domain adaptation is to enable a model to generalize to the operational domain [1]
  - **Done by finding a common embedding space that makes the distributions of X and Z more similar by aligning low-dimensional features**
  - Can train a model using the aligned embedding of X and the original validation outputs Y_S
  - Resultant model can accurately predict the unseen operational outputs Y_T by predicting on the embeddings of Z





*I use Oberkampf's notation [2] for *validation* and *operational* domains, but these are known as *source* and *target* domains, respectively, in ML literature.

Domain Adaptation Approaches

Importance Weighting [1-3]

- Re-weight points in validation domain based on their distance from the operational domain
- Weights are applied to data points during model training

$$\hat{R}(f, p_{\text{opr}}) = \frac{1}{n}\sum_{i=1}^{n} w_i \mathcal{L}(f(\boldsymbol{x}_i), \boldsymbol{y}_i)$$

Manifold/Subspace Alignment [4-6]

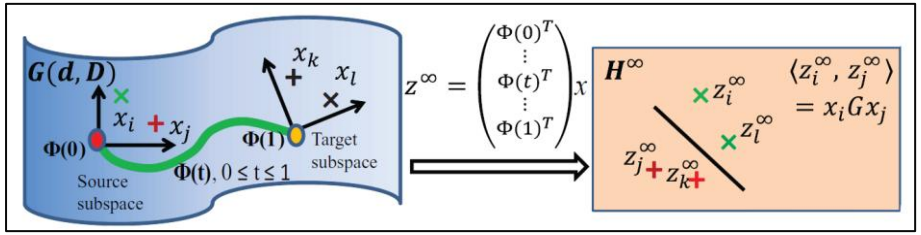
- Find subspaces within the input data in each domain, align the basis vectors by finding a mapping between their representations on a Grassmannian, and project the validation inputs onto the aligned subspace
- Model training is performed using the projected validation inputs and their labels

$$\hat{R}(f, p_{\text{opr}}) = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}(f(\boldsymbol{x}_{\text{proj}}^{(i)})), \boldsymbol{y}_i)$$

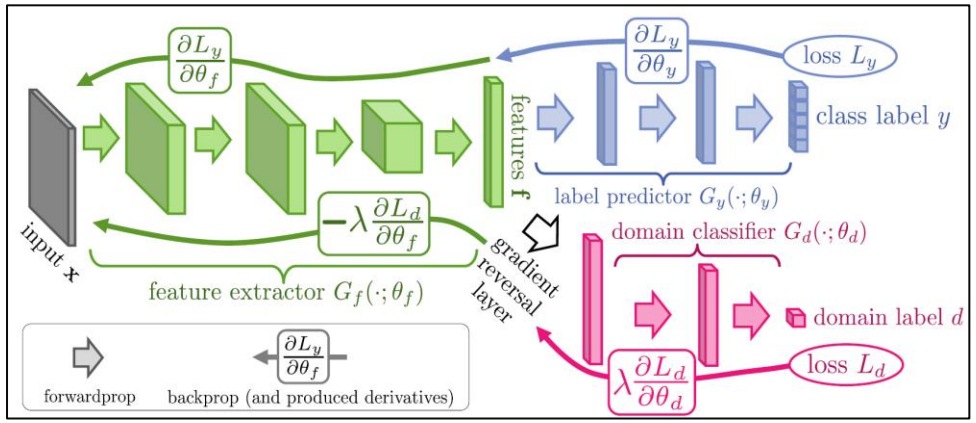
Adversarial Networks [7-11]

- Two-headed neural network with label predictor and domain classifier heads
- The label predictor and a domain classifier play an adversarial game, where the label predictor must learn domain-invariant features that confuse the classifier*

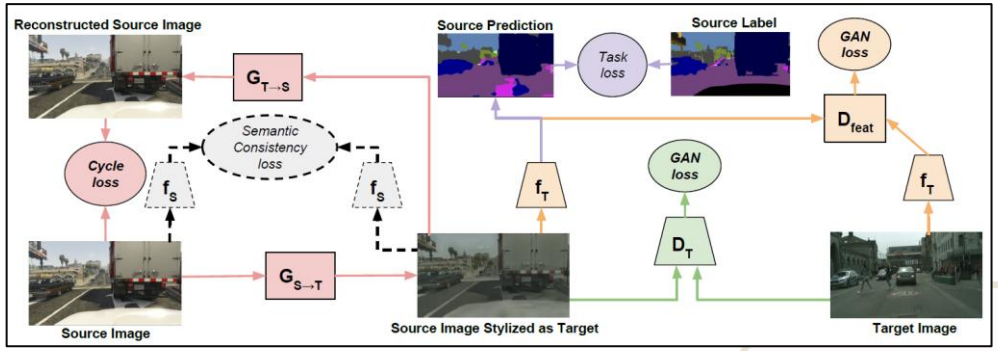
$$\hat{R}(f, p_{\text{opr}}) = \frac{1}{n}\sum_{i=1}^{n} \mathcal{L}_y^i(\theta_F, \theta_y) - \lambda\left(\frac{1}{n}\sum_{i=1}^{n} \mathcal{L}_d^i(\theta_F, \theta_d) + \frac{1}{m}\sum_{i=1}^{m} \mathcal{L}_d^i(\theta_F, \theta_d)\right)$$



Geodesic flow kernel on a Grassmann manifold, from Gong et al. [4].



Domain adversarial neural network architecture (DANN), from Ganin et al. [7].



Cycle-Consistent Adversarial Domain Adaptation, from Hoffman et al. [10].

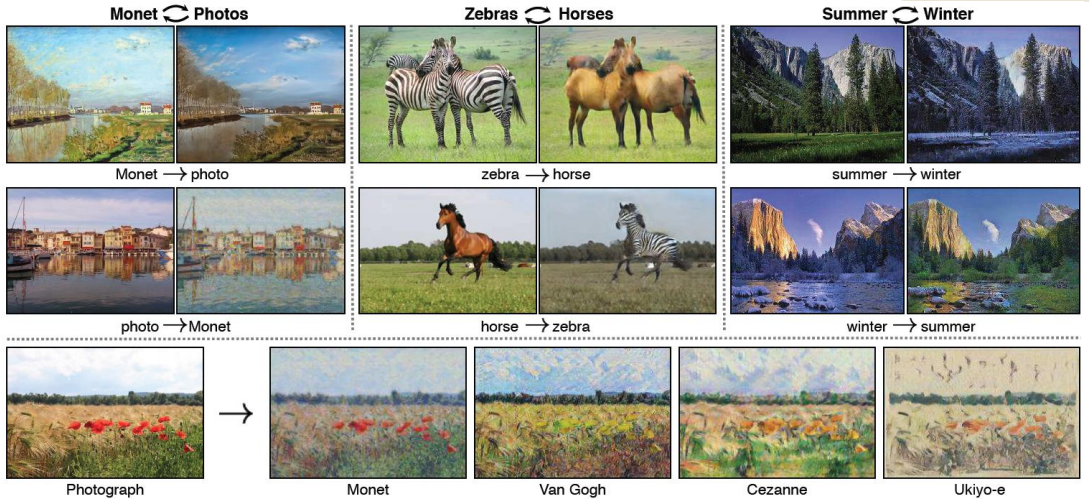
*The expression below for the empirical risk is based on Ganin et al.'s implementation [7]. Note that $\mathcal{L}_y^i(\theta_F, \theta_y) = \mathcal{L}_y(G_y(G_F(x_i; \theta_F); \theta_y), y_i)$ and $\mathcal{L}_d^i(\theta_F, \theta_y) = \mathcal{L}_d(G_d(G_F(x_i; \theta_F); \theta_y), d_i)$.

Limitations in Existing Methods

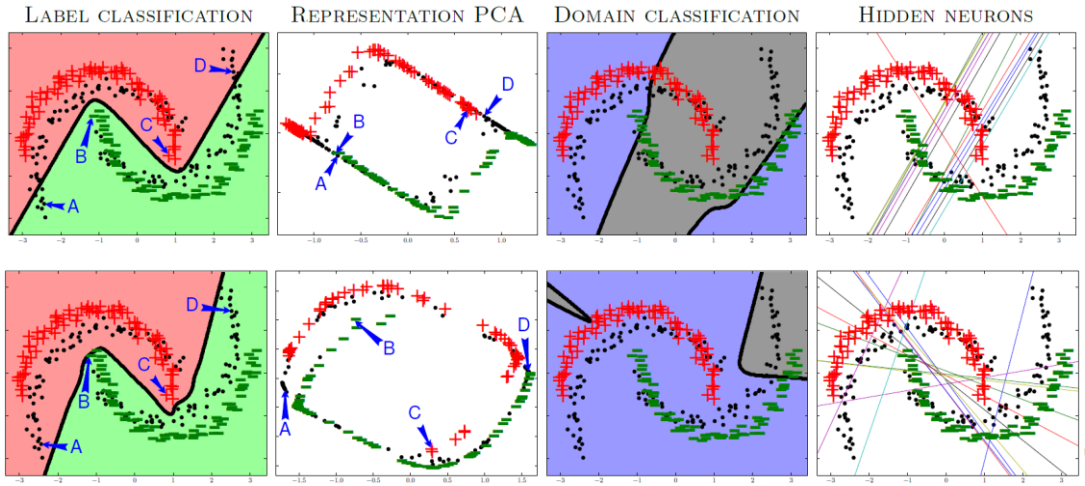
Vast majority of existing methods are intended for image classification and do not easily translate to modeling physical systems

- Individual inputs must be high-dimensional with inherent low-dimensional structure
  - Images are matrices that are flattened into high-dimensional vectors
  - Feature alignment is performed between low-dimensional features within individual images
- Outputs must be discrete and finite
  - Classification problems have bounded outputs [1]*
  - Existing literature on regression is sparse [2-5]

Physical systems often deal with continuous, scalar variables, making many existing methods intractable



CycleGAN image-to-image translation, which transforms a source image into a target image using an adversarial loss [6].

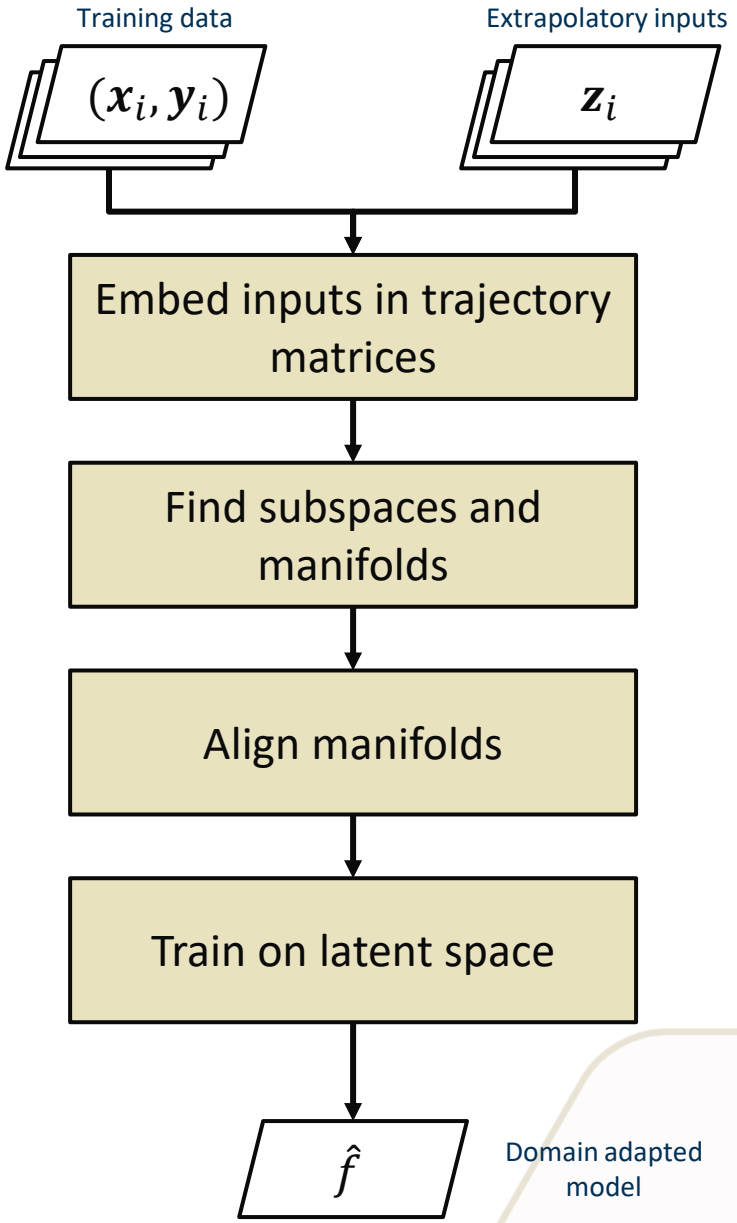


DANN twin moons classification results by Ganin et al. [1].

*Ganin et al.'s DANN architecture [1] relies on minimizing the empirical H-divergence between domains, inspired heavily by domain learning theory from Ben-David et al. [2-4]. Attempting to find the H-divergence for an MSE loss causes the divergence to (ironically) diverge to infinity (see Appendix C for an explanation or Sec. 2.3 in manuscript).

Proposed Approach

- To address these limitations, I can rely on the fact that the data in the problem is determined by *missions*
  - Inputs/outputs are time-series data
  - DRM and extrapolatory mission will exhibit similar dynamic behavior (e.g., entry/exit in atmosphere leads to repeating spikes in the inputs)
- **In many physical systems, the system state evolves on a low-dimensional manifold**
  - Using time-delayed windows can encode the system dynamics in a low-dimensional representation of the data [1]*
  - Manifold alignment can compute a latent space that mitigates model extrapolation [4]



*This is Takens' embedding theorem [1] and is used extensively in system identification for algorithms like eigensystem realization [2] and singular spectrum analysis [3].

Method of Delays

- A foundational concept in data-driven modeling of dynamical systems is the *method of delays*
  - Used in system identification, attractor reconstruction, and spectral analysis of time-series data [1-3]
- **Key idea is that the state space of a dynamical system can be reconstructed from time-lagged sequences of a time-series measurement [4]**
- SVD of the time-lagged vectors yields a subspace that encodes low-dimensional temporal features*

Time-series

$$\boldsymbol{X} = [x_1, \ldots, x_n]^T \in \mathbb{R}^n$$

Trajectory matrix

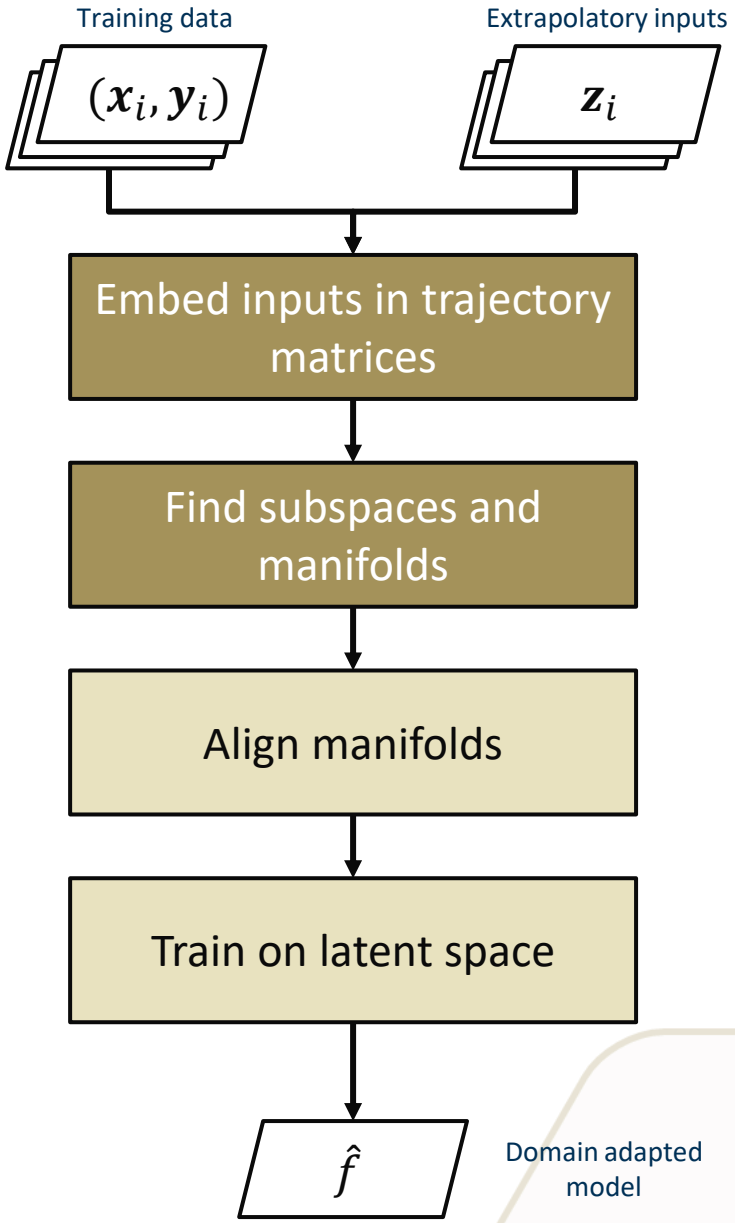
$$\boldsymbol{H}_x = \begin{bmatrix} x_1 & x_2 & \cdots & x_{n-L+1} \\ x_2 & x_3 & \cdots & x_{n-L+2} \\ \vdots & \vdots & \ddots & \vdots \\ x_L & x_{L+1} & \cdots & x_n \end{bmatrix} \in \mathbb{R}^{L\times(n-L+1)}$$

SVD

$$\boldsymbol{H}_x = \boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T$$

Manifold

$$\boldsymbol{H}_{x,\text{proj}} = \boldsymbol{U}_k^T \boldsymbol{H}_x$$

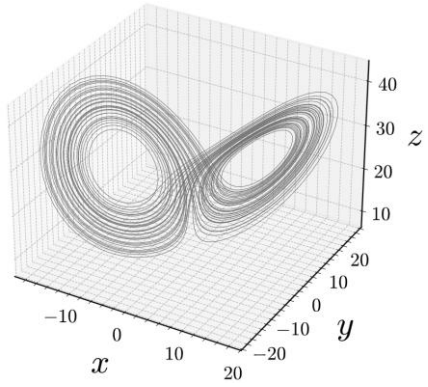


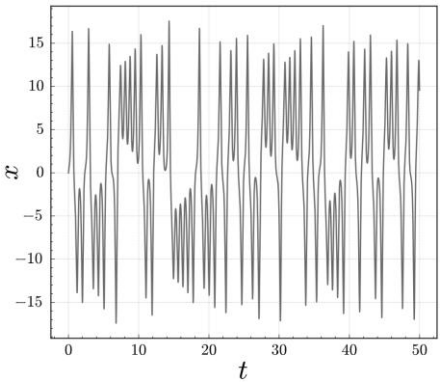
*This technique is known as singular spectrum analysis (SSA) [1].

Example: Lorenz System

- Consider the Lorenz system:

$$\dot{x} = \sigma(y - x)$$
$$\dot{y} = x(\rho - z) - y$$
$$\dot{z} = xy - \beta z$$

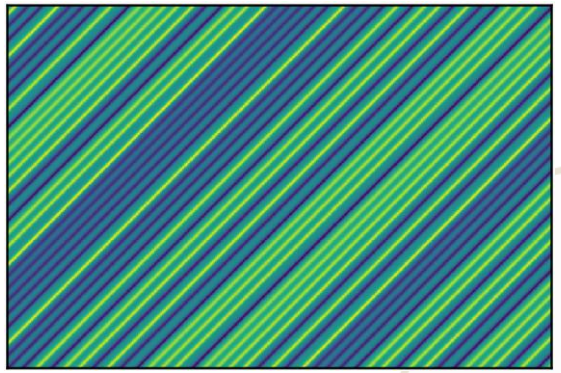




- For parameters $\beta = 8/3, \sigma = 10, \rho = 28$, the system trajectory is governed by a *strange attractor**
- Can reconstruct the attractor by taking time-delayed windows of $x(t)$
- Given a window length of L, take the first L number of measurements $x_1, \dots, x_L$ from $x(t)$ and store them as the first column of a *trajectory matrix*
  - Then take the next L number of measurements $x_2, \dots, x_{L+1}$ and store them as the second column of the trajectory matrix
    - And so on…

Embed signal in trajectory matrix

$$\boldsymbol{H}_x = \begin{bmatrix} x_1 & x_2 & \cdots & x_{n-L+1} \\ x_2 & x_3 & \cdots & x_{n-L+2} \\ \vdots & \vdots & \ddots & \vdots \\ x_L & x_{L+1} & \cdots & x_n \end{bmatrix} \in \mathbb{R}^{L \times (n-L+1)}$$

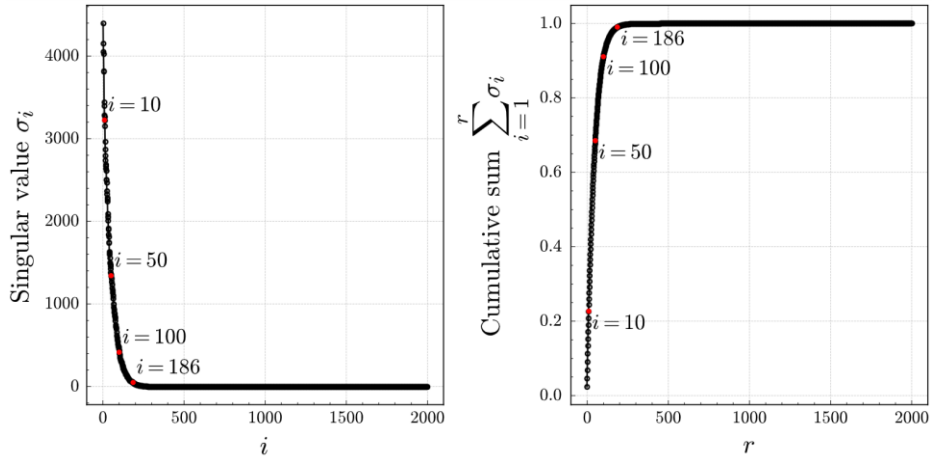


Colorized trajectory matrix

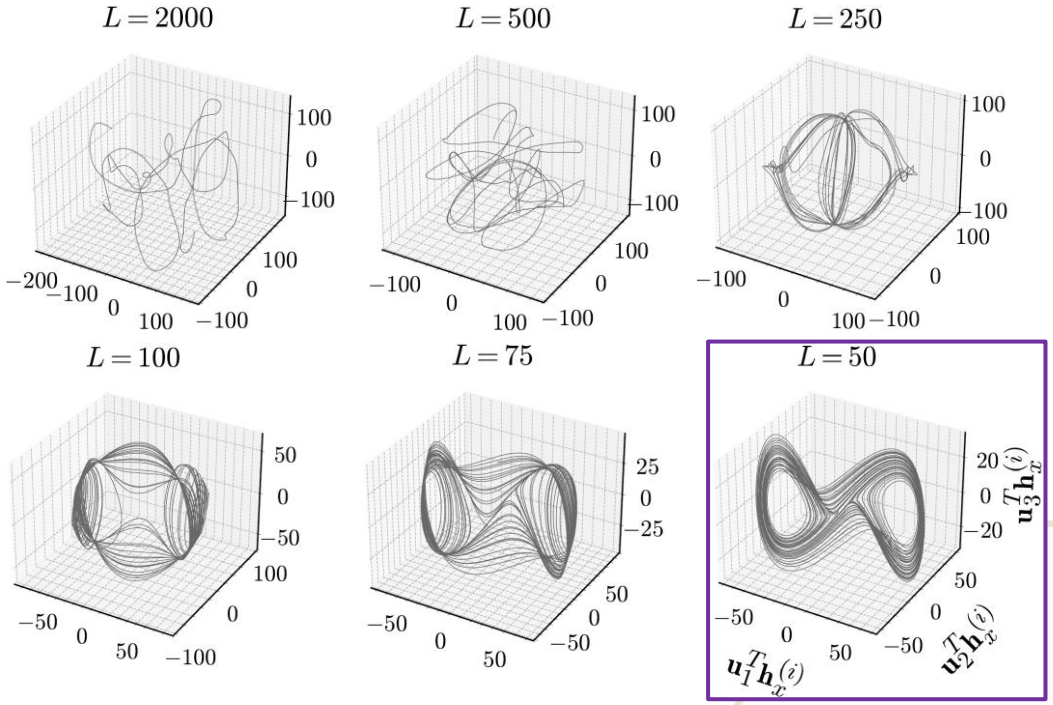
*An attractor is a set of states that a system will evolve towards from a set of initial conditions. More precise examples of attractors include stable fixed points and limit cycles. A strange attractor is neither of these.

Subspaces and Manifolds

- The resultant trajectory matrix has windows of $x(t)$ as its columns, offset by one timestep
  - There are $n - L + 1$ number of windows in the matrix
  - The matrix is Hankel (each skew-diagonal is constant)
- The dominant singular vectors of the trajectory matrix can be used as a subspace
- Projection of trajectory matrix onto this subspace returns a manifold that encodes the system state evolution
  - If window size is decreased, top singular vectors capture more information in the data
  - **Manifold reconstructs the Lorenz attractor at smaller window sizes**



Trajectory matrix ($L = 2000$) singular values and normalized cumulative sum.



Projection of trajectory matrix onto top 3 left singular vectors for different window sizes.

Manifold Alignment

- Disparate signals governed by the same physics still share similar dynamics
  - Can align manifolds of different time-series data to perform domain adaptation
  - The resultant aligned manifold is a latent space that reduces discrepancy between input data
- For example, Procrustes analysis solves for rotation and scaling terms between the manifolds of two time-series [1]:

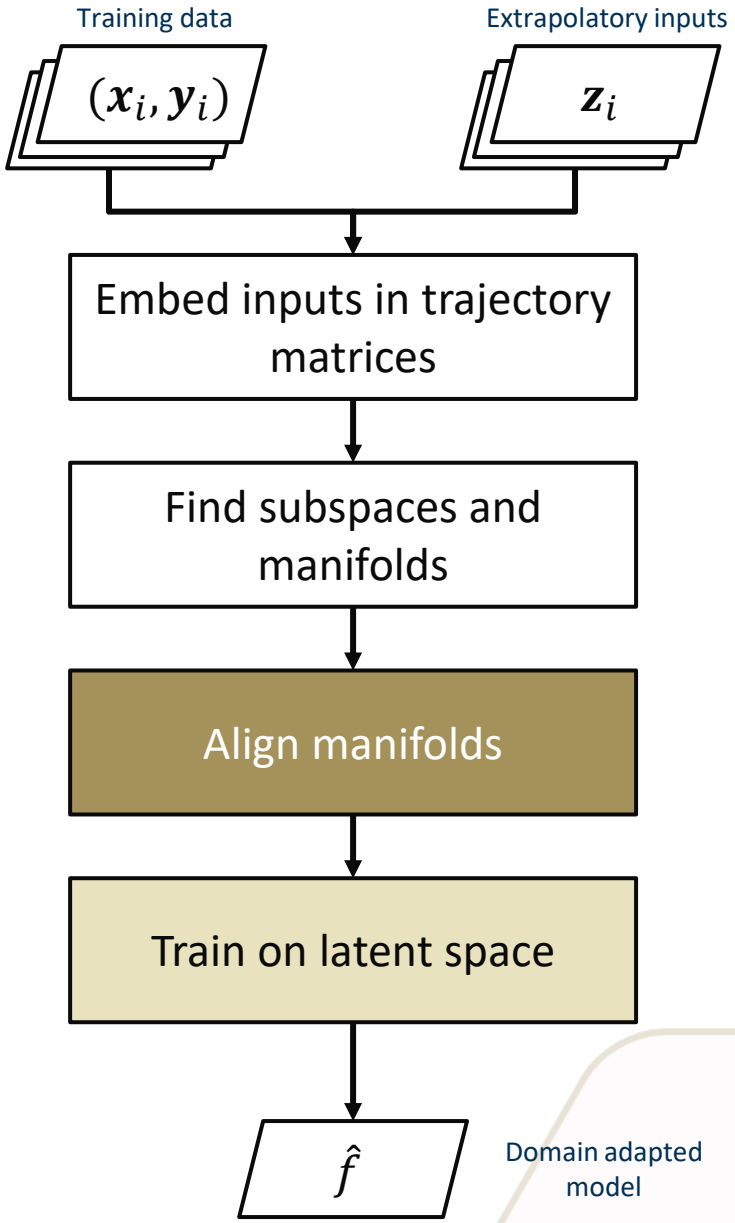
Procrustes manifold alignment

$$\min_{\boldsymbol{Q},s,\boldsymbol{t}} \|\boldsymbol{H}_{Z,\text{proj}} - s\boldsymbol{Q}(\boldsymbol{H}_{X,\text{proj}} - \boldsymbol{t})\|_F \quad \text{s.t.} \quad \boldsymbol{Q}^T\boldsymbol{Q} = \boldsymbol{I}$$

$$\boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T = \boldsymbol{H}_{X,\text{proj}}\boldsymbol{H}_{Z,\text{proj}}^T,$$

$$\boldsymbol{Q} = \boldsymbol{V}\boldsymbol{U}^T,$$

$$s = \frac{\text{tr}(\boldsymbol{S})}{\text{tr}(\boldsymbol{H}_{X,\text{proj}}\boldsymbol{H}_{X,\text{proj}}^T)}$$

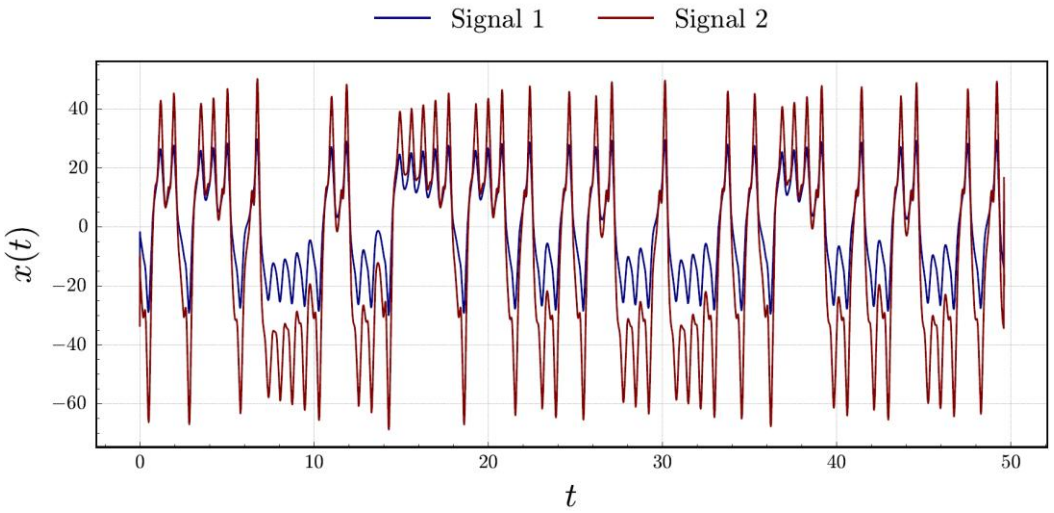


Example: Lorenz System (again)

- Consider two different signals from a Lorenz system
- These two signals have different manifolds but still share similar low-dimensional features (e.g., two foci, fractal patterns)
- To perform Procrustes manifold alignment, solve for rotation matrix $\boldsymbol{Q}$ and scaling factor s
- Rotate and scale the manifold of signal 1 so that it aligns with the manifold of signal 2

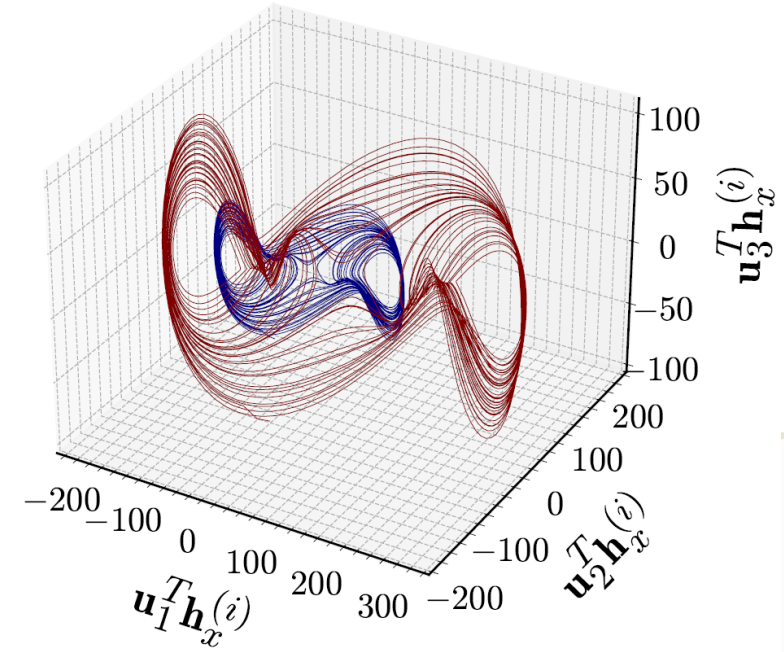
$$\boldsymbol{X}_a = s\boldsymbol{Q}\boldsymbol{H}_{X,\text{proj}} \in \mathbb{R}^{k\times(n-L+1)},$$

$$\boldsymbol{Z}_a = \boldsymbol{H}_{Z,\text{proj}} \in \mathbb{R}^{k\times(n-L+1)}.$$



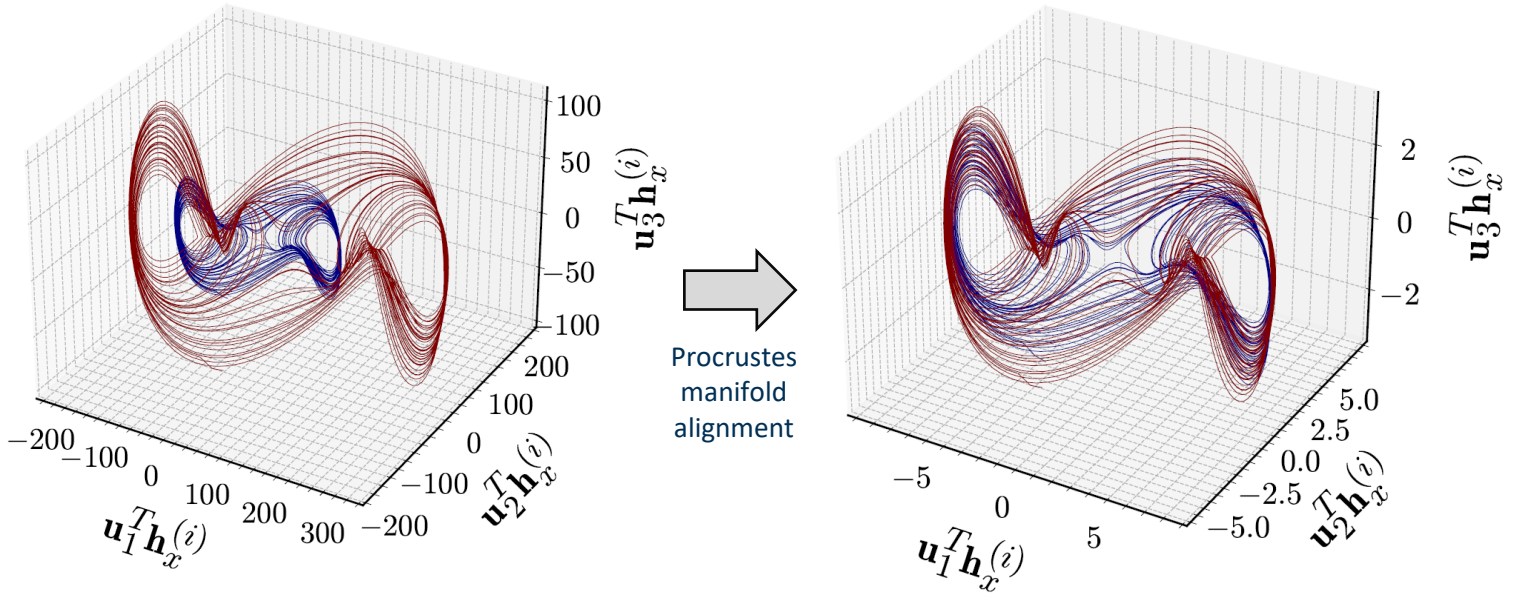
Signal 1 and signal 2, drawn from a Lorenz attractor.

Manifolds of signal 1 and signal 2 found using SSA



Manifold Alignment Results

- Procrustes manifold alignment reduces the discrepancy between the manifolds of signal 1 and signal 2
- Training a model in this aligned representation should mitigate model extrapolation
- **For streaming data, the subspaces/manifolds can be computed online by subspace tracking [1]**



Training and Inference

Training

- Training in the latent space allows the model to learn temporal features common to both domains
- The domain adapted loss consists of:
  - Projection of trajectory matrix of training inputs onto the latent space
  - The original training outputs

Prediction/Inference

- To predict on the operating inputs:
  - Embed the data in a trajectory matrix
  - Project the trajectory matrix onto the latent space
- Model predicts on the projection of the trajectory matrix

Algorithm 1 Procrustes Subspace Adaptation (PSA)

Input: $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]^T \in \mathbb{R}^{n \times D}$, $\boldsymbol{Y}_S = [y_S^{(1)}, \ldots, y_S^{(n)}] \in \mathbb{R}^n$, timestamps $\boldsymbol{t}_S \in \mathbb{R}^n$, subspace dimension k, window length L, learning rate η

Output: Model $\tilde{f} : \boldsymbol{x}_a \mapsto \tilde{\boldsymbol{h}}_Y$, embedding space $\boldsymbol{H}_{Z_t,\text{sub}}$

Find $\boldsymbol{H}_X = \mathcal{H}(\boldsymbol{X}) \in \mathbb{R}^{DL \times (n-L+1)}$ and $\boldsymbol{H}_{Y_S} = \mathcal{H}(\boldsymbol{Y}_S) \in \mathbb{R}^{DL \times (n-L+1)}$

$\boldsymbol{H}_{X,\text{sub}} \in \mathbb{R}^{DL \times k} \leftarrow \text{pca}(\boldsymbol{H}_X)$

Set initial guess for $\boldsymbol{H}_{Z_t,\text{sub}}(t=0) = \boldsymbol{H}_{X,\text{sub}}$

repeat

  Receive observations of z and append to $\boldsymbol{Z}_t$

  Record corresponding timestamps of $\boldsymbol{Z}_t$ as $\boldsymbol{t}_T$

  Truncate $\boldsymbol{X}$ to $\boldsymbol{X}_t$, containing x seen up to time t

  $\boldsymbol{X}_t = (\boldsymbol{X}_t - \bar{\boldsymbol{X}}_t)/\sigma_{X_t}$ ▷ Mean $\bar{\boldsymbol{X}}_t$, standard deviation σ_{X_t}

  $\boldsymbol{Z}_t = (\boldsymbol{Z}_t - \bar{\boldsymbol{Z}}_t)/\sigma_{Z_t}$ ▷ Mean $\bar{\boldsymbol{Z}}_t$, standard deviation σ_{Z_t}

  $\boldsymbol{H}_{X_t} = \mathcal{H}(\boldsymbol{X}_t)$

  $\boldsymbol{H}_{Z_t} = \mathcal{H}(\boldsymbol{Z}_t)$

  Define common timesteps for interpolation $\mathbf{t} = \text{sort}\left([\boldsymbol{t}_S | \boldsymbol{t}_T]^T\right)$

  $\tilde{\boldsymbol{X}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{X}_t)$

  $\tilde{\boldsymbol{Z}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{Z}_t)$

  $\tilde{\boldsymbol{H}}_{X_t} = \mathcal{H}(\tilde{\boldsymbol{X}}_t)$

  $\tilde{\boldsymbol{H}}_{Z_t} = \mathcal{H}(\tilde{\boldsymbol{Z}}_t)$

  $\boldsymbol{H}_{X_t,\text{sub}} \leftarrow \text{pca}(\boldsymbol{H}_{X_t})$

  $\boldsymbol{H}_{Z_t,\text{sub}}(t) = \boldsymbol{H}_{Z_t,\text{sub}}(t-1) + \eta \tilde{\boldsymbol{h}}_{Z_t}(t) \left(\tilde{\boldsymbol{h}}_{Z_t}(t)\right)^T \boldsymbol{H}_{Z_t,\text{sub}}(t-1)$

  $\tilde{\boldsymbol{H}}_{X_t,\text{proj}} = \boldsymbol{H}_{X_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{X_t}$

  $\tilde{\boldsymbol{H}}_{Z_t,\text{proj}} = \boldsymbol{H}_{Z_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{Z_t}$

  $\boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T = \tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{Z_t,\text{proj}}^T$

  $\boldsymbol{Q} = \boldsymbol{V}\boldsymbol{U}^T$

  $s = \dfrac{\text{tr}(\boldsymbol{S})}{\text{tr}(\tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{X_t,\text{proj}}^T)}$

  $\boldsymbol{X}_a = s\boldsymbol{Q}(\boldsymbol{H}_{X_t,\text{sub}}^T \boldsymbol{H}_{X_t})$

  $\boldsymbol{Z}_a = \boldsymbol{H}_{Z_t,\text{sub}}^T \boldsymbol{H}_{Z_t}$

  $\tilde{f} = \arg\min_{\tilde{\theta}} \sum_{i=1}^{n} \mathcal{L}\left(\tilde{f}(\boldsymbol{x}_a^{(i)} | \tilde{\boldsymbol{\theta}}), \boldsymbol{h}_{Y_S}^{(i)}\right)$

  $t = t + 1$

until termination or all z seen

return $\tilde{f}$, $\boldsymbol{H}_{Z_t,\text{sub}}$

Research Objective

Develop an unsupervised method to allow a system digital twin to retain predictive performance on out-of-distribution data in evolving, unknown domains, such as changing environmental conditions, operating conditions, or configurations.

Observation 1: In many physical systems, the system state evolves on a low-dimensional manifold

Observation 2: The state space of a dynamical system can be reconstructed from a time-delay embedding of a scalar time-series

Observation 3: Manifold alignment techniques can find a latent space that reduces domain discrepancy

Observation 4: Subspace tracking can find a subspace of streaming data in a computational tractable manner

Overarching Hypothesis

If subspace tracking and manifold alignment are used to learn a common low-dimensional latent space of time-delay embedded data, then a data-driven digital twin trained in this latent space can retain predictive performance in evolving, unknown domains.

Procrustes Subspace Adaptation

Let's apply this to the aerobraking example

Recap

Build model to predict $\dot{Q}$ from ρ, V, T:

$$\dot{Q} = f\big(\rho(t), V(t), T(t)\big)$$

- Inputs $\rho(t), V(t), T(t)$ are observable during deployment
- Heat rate $\dot{Q}$ is unobservable during deployment
- New mission has out-of-distribution, extrapolatory data

Objective is to predict $\dot{Q}$ accurately on new mission

Algorithm 1 Procrustes Subspace Adaptation (PSA)

Input: $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]^T \in \mathbb{R}^{n \times D}$, $\boldsymbol{Y}_S = [y_S^{(1)}, \ldots, y_S^{(n)}] \in \mathbb{R}^n$, timestamps $\boldsymbol{t}_S \in \mathbb{R}^n$, subspace dimension k, window length L, learning rate η

Output: Model $\tilde{f} : \boldsymbol{x}_a \mapsto \tilde{h}_Y$, embedding space $\boldsymbol{H}_{Z_t,\text{sub}}$

Find $\boldsymbol{H}_X = \mathcal{H}(\boldsymbol{X}) \in \mathbb{R}^{DL \times (n-L+1)}$ and $\boldsymbol{H}_{Y_S} = \mathcal{H}(\boldsymbol{Y}_S) \in \mathbb{R}^{DL \times (n-L+1)}$

$\boldsymbol{H}_{X,\text{sub}} \in \mathbb{R}^{DL \times k} \leftarrow \text{pca}(\boldsymbol{H}_X)$

Set initial guess for $\boldsymbol{H}_{Z_t,\text{sub}}(t=0) = \boldsymbol{H}_{X,\text{sub}}$

repeat

  Receive observations of $\boldsymbol{z}$ and append to $\boldsymbol{Z}_t$

  Record corresponding timestamps of $\boldsymbol{Z}_t$ as $\boldsymbol{t}_T$

  Truncate $\boldsymbol{X}$ to $\boldsymbol{X}_t$, containing x seen up to time t

  $\boldsymbol{X}_t = (\boldsymbol{X}_t - \bar{\boldsymbol{X}}_t)/\sigma_{X_t}$ ▷ Mean $\bar{\boldsymbol{X}}_t$, standard deviation σ_{X_t}

  $\boldsymbol{Z}_t = (\boldsymbol{Z}_t - \bar{\boldsymbol{Z}}_t)/\sigma_{Z_t}$ ▷ Mean $\bar{\boldsymbol{Z}}_t$, standard deviation σ_{Z_t}

  $\boldsymbol{H}_{X_t} = \mathcal{H}(\boldsymbol{X}_t)$

  $\boldsymbol{H}_{Z_t} = \mathcal{H}(\boldsymbol{Z}_t)$

  Define common timesteps for interpolation $\mathbf{t} = \text{sort}\left([\boldsymbol{t}_S | \boldsymbol{t}_T]^T\right)$

  $\tilde{\boldsymbol{X}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{X}_t)$

  $\tilde{\boldsymbol{Z}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{Z}_t)$

  $\tilde{\boldsymbol{H}}_{X_t} = \mathcal{H}(\tilde{\boldsymbol{X}}_t)$

  $\tilde{\boldsymbol{H}}_{Z_t} = \mathcal{H}(\tilde{\boldsymbol{Z}}_t)$

  $\boldsymbol{H}_{X_t,\text{sub}} \leftarrow \text{pca}(\boldsymbol{H}_{X_t})$

  $\boldsymbol{H}_{Z_t,\text{sub}}(t) = \boldsymbol{H}_{Z_t,\text{sub}}(t-1) + \eta \tilde{\boldsymbol{h}}_{Z_t}(t) \left(\tilde{\boldsymbol{h}}_{Z_t}(t)\right)^T \boldsymbol{H}_{Z_t,\text{sub}}(t-1)$

  $\tilde{\boldsymbol{H}}_{X_t,\text{proj}} = \boldsymbol{H}_{X_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{X_t}$

  $\tilde{\boldsymbol{H}}_{Z_t,\text{proj}} = \boldsymbol{H}_{Z_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{Z_t}$

  $\boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T = \tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{Z_t,\text{proj}}^T$

  $\boldsymbol{Q} = \boldsymbol{V}\boldsymbol{U}^T$

  $s = \dfrac{\text{tr}(\boldsymbol{S})}{\text{tr}(\tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{X_t,\text{proj}}^T)}$

  $\boldsymbol{X}_a = s\boldsymbol{Q}(\boldsymbol{H}_{X_t,\text{sub}}^T \boldsymbol{H}_{X_t})$

  $\boldsymbol{Z}_a = \boldsymbol{H}_{Z_t,\text{sub}}^T \boldsymbol{H}_{Z_t}$

  $\tilde{f} = \arg\min_{\tilde{\theta}} \sum_{i=1}^{n} \mathcal{L}\left(\tilde{f}(\boldsymbol{x}_a^{(i)} | \tilde{\boldsymbol{\theta}}), \boldsymbol{h}_{Y_S}^{(i)}\right)$

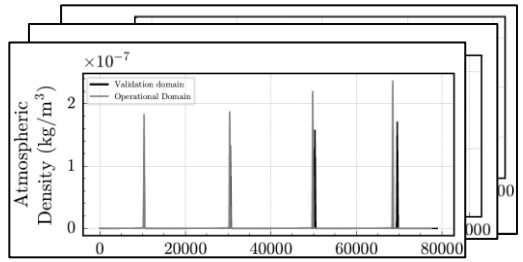
  $t = t + 1$

until termination or all $\boldsymbol{z}$ seen

return $\tilde{f}, \boldsymbol{H}_{Z_t,\text{sub}}$

Finding Low-Dimensional Embeddings

1) Let $\boldsymbol{X}$ and $\boldsymbol{Z}$ be the inputs (density, velocity, temperature time series) from the DRM and extrapolatory mission, respectively



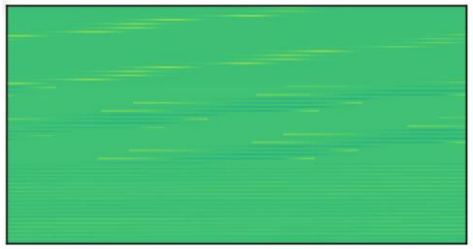
$$\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]^T \in \mathbb{R}^{n \times D}$$

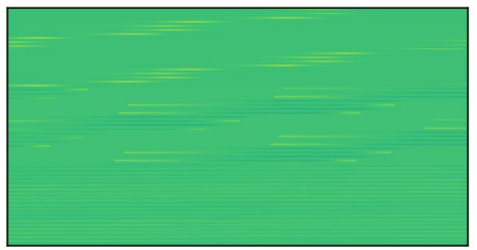
$$\boldsymbol{Z} = [\boldsymbol{z}_1, \ldots, \boldsymbol{z}_m]^T \in \mathbb{R}^{m \times D}$$

2) Specify a window length of points L, and stack $\boldsymbol{X}$ and $\boldsymbol{Z}$ into trajectory matrices* $\boldsymbol{H}_X$ and $\boldsymbol{H}_Z$

$\boldsymbol{X}$ → Trajectory matrix embedding → $\mathbf{H}_X$

$\boldsymbol{Z}$ → Trajectory matrix embedding → $\mathbf{H}_Z$





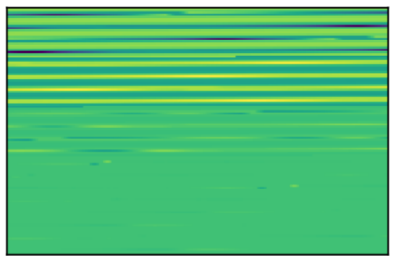
$$\boldsymbol{H}_{X,1} = \begin{bmatrix} x_{1,1} & x_{2,1} & \cdots & x_{n-L+1,1} \\ x_{2,1} & x_{3,1} & \cdots & x_{n-L+2,1} \\ \vdots & \vdots & \ddots & \vdots \\ x_{L,1} & x_{L+1,1} & \cdots & x_{n,1} \end{bmatrix}, \boldsymbol{H}_{X,2} = \begin{bmatrix} x_{1,2} & x_{2,2} & \cdots & x_{n-L+1,2} \\ x_{2,2} & x_{3,2} & \cdots & x_{n-L+2,2} \\ \vdots & \vdots & \ddots & \vdots \\ x_{L,2} & x_{L+1,2} & \cdots & x_{n,2} \end{bmatrix}, \ldots, \boldsymbol{H}_{X,D}$$

$$\boldsymbol{H}_{Z,1} = \begin{bmatrix} z_{1,1} & z_{2,1} & \cdots & z_{n-L+1,1} \\ z_{2,1} & z_{3,1} & \cdots & z_{n-L+2,1} \\ \vdots & \vdots & \ddots & \vdots \\ z_{L,1} & z_{L+1,1} & \cdots & z_{n,1} \end{bmatrix}, \boldsymbol{H}_{Z,2} = \begin{bmatrix} z_{1,2} & z_{2,2} & \cdots & z_{n-L+1,2} \\ z_{2,2} & z_{3,2} & \cdots & z_{n-L+2,2} \\ \vdots & \vdots & \ddots & \vdots \\ z_{L,2} & z_{L+1,2} & \cdots & z_{n,2} \end{bmatrix}, \ldots, \boldsymbol{H}_{Z,D}$$

$$\boldsymbol{H}_X := \begin{bmatrix} \boldsymbol{H}_{X,1} \\ \boldsymbol{H}_{X,2} \\ \vdots \\ \boldsymbol{H}_{X,D} \end{bmatrix} \in \mathbb{R}^{DL \times (n-L+1)} \quad \boldsymbol{H}_Z := \begin{bmatrix} \boldsymbol{H}_{Z,1} \\ \boldsymbol{H}_{Z,2} \\ \vdots \\ \boldsymbol{H}_{Z,D} \end{bmatrix} \in \mathbb{R}^{DL \times (m-L+1)}$$

3) Project $\boldsymbol{H}_X$ and $\boldsymbol{H}_Z$ onto their respective subspaces

Project H_X and H_Z onto their respective top k left singular vectors



$$\boldsymbol{U}_X, \boldsymbol{S}_X, \boldsymbol{V}_X^T = \text{svd}(\boldsymbol{H}_X)$$

$$\boldsymbol{H}_{X,\text{sub}} := \boldsymbol{U}_X^{(k)}$$

$$\boldsymbol{H}_{X,\text{proj}} = \boldsymbol{H}_{X,\text{sub}}^T \boldsymbol{H}_X$$

$$\boldsymbol{H}_{Z_t,\text{sub}}(t) = \boldsymbol{H}_{Z_t,\text{sub}}(t-1) + \eta \boldsymbol{h}_{Z_t}(t) \left(\boldsymbol{h}_{Z_t}(t)\right)^T \boldsymbol{H}_{Z_t,\text{sub}}(t-1)$$

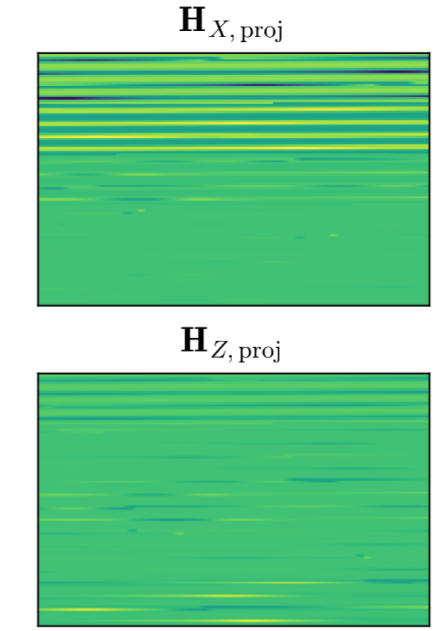
$$\boldsymbol{H}_{Z,\text{proj}} = \boldsymbol{H}_{Z,\text{sub}}^T \boldsymbol{H}_Z$$

Georgia Tech
Aerospace Systems Design Laboratory

*These matrix visuals have been reshaped to be approximately square for better viewing, but all operations are performed on the original Hankel matrices.

Aligning the Subspaces

4) Find transformation to align the subspace projections of the data by Procrustes analysis*



Procrustes analysis

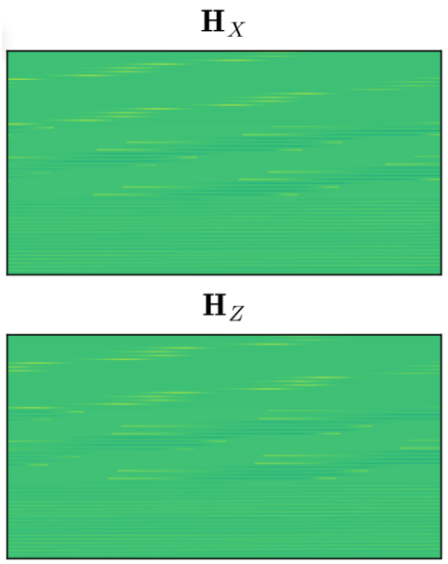
$$\min_{\boldsymbol{Q},s,\boldsymbol{t}} \|\boldsymbol{H}_{Z,\text{proj}} - s\boldsymbol{Q}(\boldsymbol{H}_{X,\text{proj}} - \boldsymbol{t})\|_F \quad \text{s.t.} \quad \boldsymbol{Q}^T\boldsymbol{Q} = \boldsymbol{I}$$

$$\boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T = \text{svd}(\boldsymbol{H}_{X,\text{proj}}\boldsymbol{H}_{Z,\text{proj}}^T)$$

$$\boldsymbol{Q} = \boldsymbol{V}\boldsymbol{U}^T$$

$$s = \frac{\text{tr}(\boldsymbol{S})}{\text{tr}(\boldsymbol{H}_{X,\text{proj}}\boldsymbol{H}_{X,\text{proj}}^T)}$$

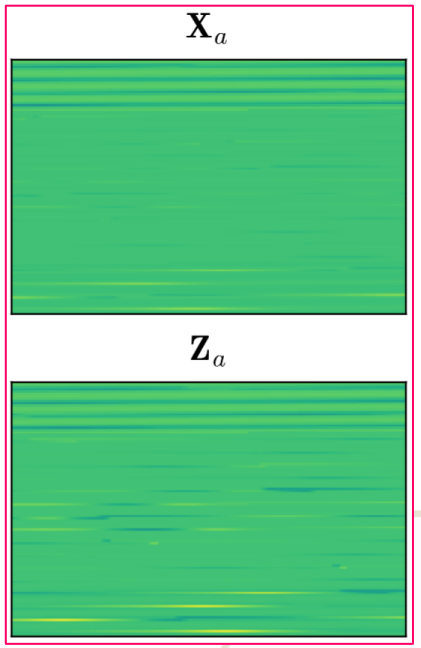
5) Apply rotation and scaling to $\boldsymbol{H}_{X,\text{proj}}$ and $\boldsymbol{H}_{Z,\text{proj}}$ to yield aligned manifolds $\boldsymbol{X}_a$ and $\boldsymbol{Z}_a$



Manifold alignment

$$\boldsymbol{X}_a = s\boldsymbol{Q}(\boldsymbol{H}_{X,\text{sub}}^T\boldsymbol{H}_X)$$

$$\boldsymbol{Z}_a = \boldsymbol{H}_{Z,\text{sub}}^T\boldsymbol{H}_Z$$



*In this example, both $\boldsymbol{X}$ and $\boldsymbol{Z}$ have an identical number of points ($n = m$) and are mean-centered and scaled to unit variance as a pre-processing step. Mean-centering the data removes the need to compute the translation term t. If $n \neq m$, the data is interpolated to a common set of timesteps.

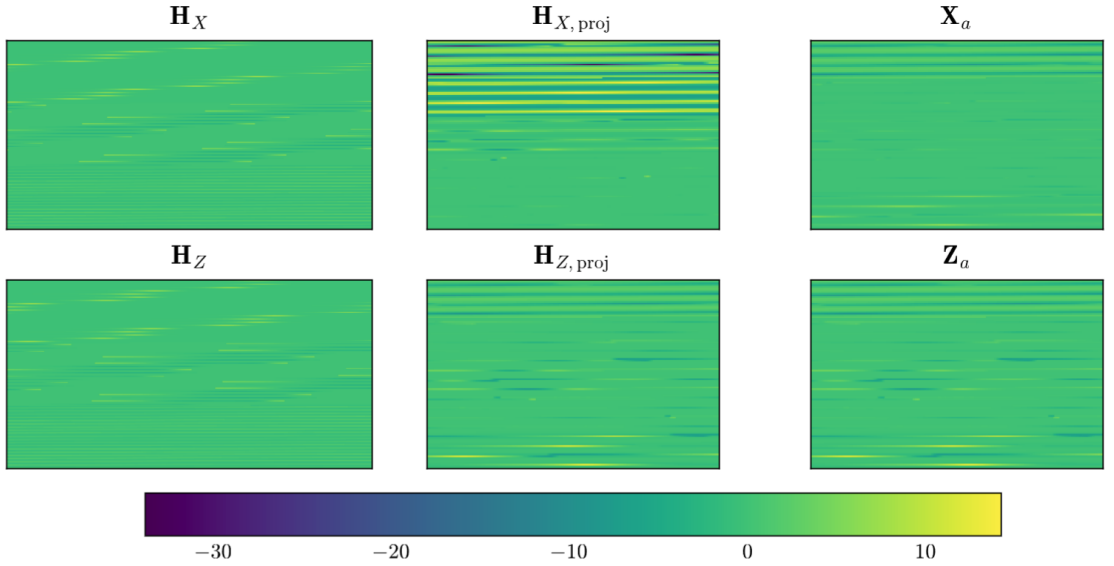
Visualizing the Embedding Space

6) The manifold alignment makes the distribution of $\boldsymbol{H}_{X,\text{proj}}$ more similar to that of $\boldsymbol{H}_{Z,\text{proj}}$, which is reflected in the aligned embeddings $\boldsymbol{X}_a$ and $\boldsymbol{Z}_a$.

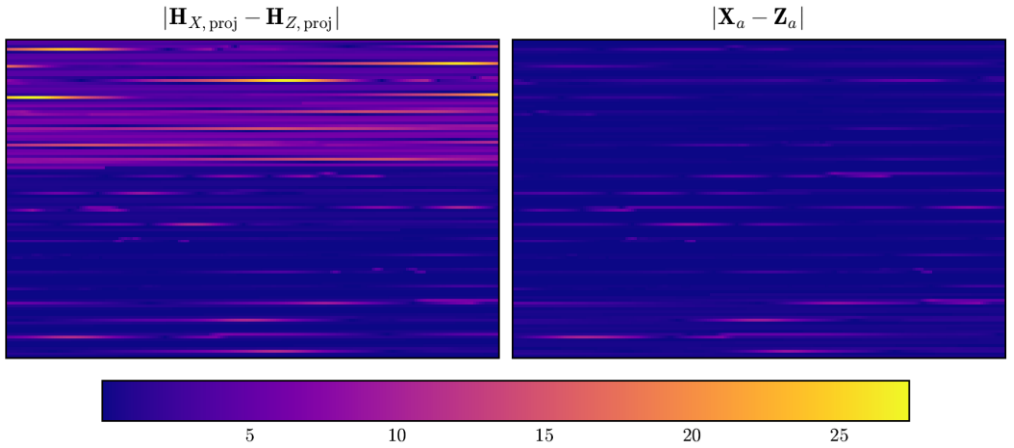
7) To ensure consistent dimensions, we embed the outputs $\boldsymbol{Y}_S$ of the DRM into a trajectory matrix $\boldsymbol{H}_{Y_S}$

A domain adapted model can be trained using the columns of $\boldsymbol{X}_a$ as inputs and $\boldsymbol{H}_Y$ as outputs

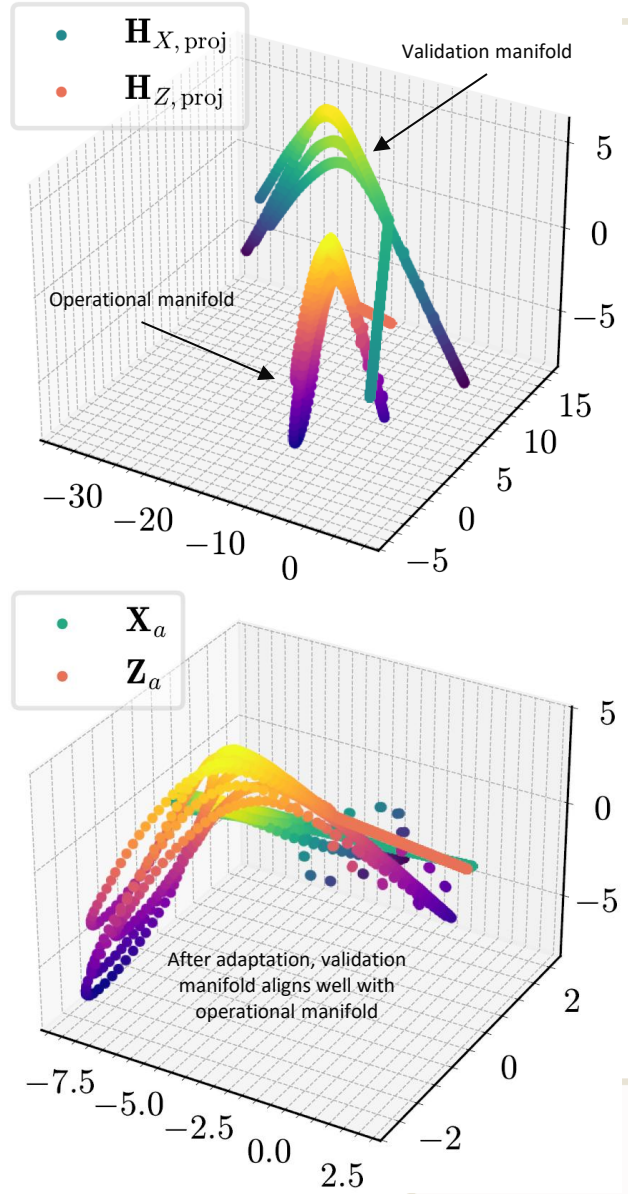
$$\tilde{f} = \arg\min_{\tilde{\theta}} \sum_{i=1}^{n} \mathcal{L}\left(\tilde{f}(\boldsymbol{x}_a^{(i)}|\tilde{\boldsymbol{\theta}}), \boldsymbol{h}_{Y_S}^{(i)}\right)$$



Trajectory matrices (left column), trajectory matrix projections onto subspaces (middle column), and aligned subspace projections (right column).



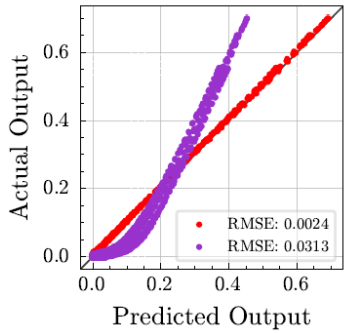
Absolute difference between trajectory matrix projections (left) and aligned projections (right).

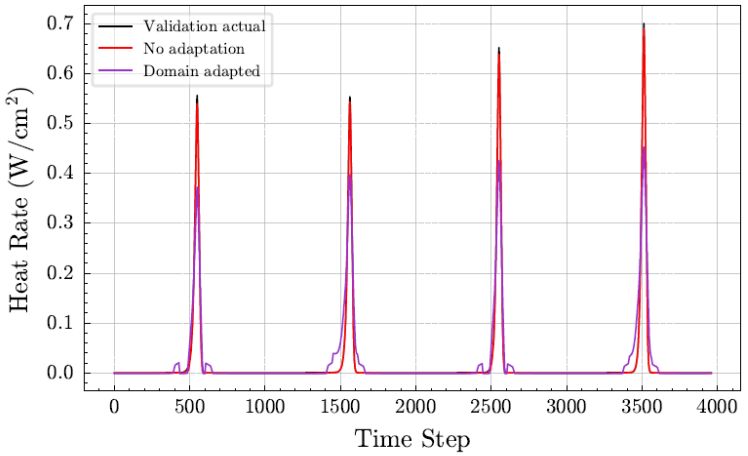


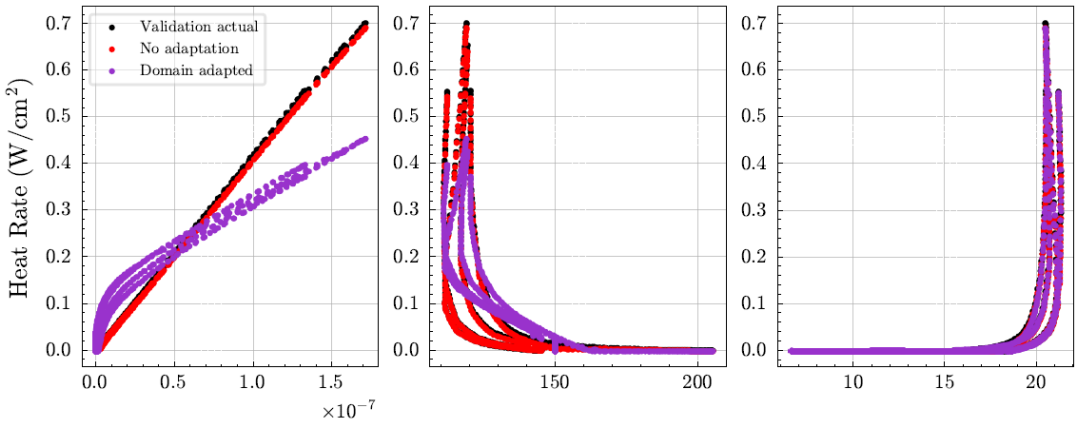
(top) Unaligned and (bottom) aligned manifolds.

Demonstration Results

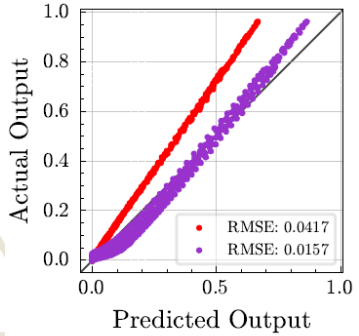
Design reference mission (validation domain)

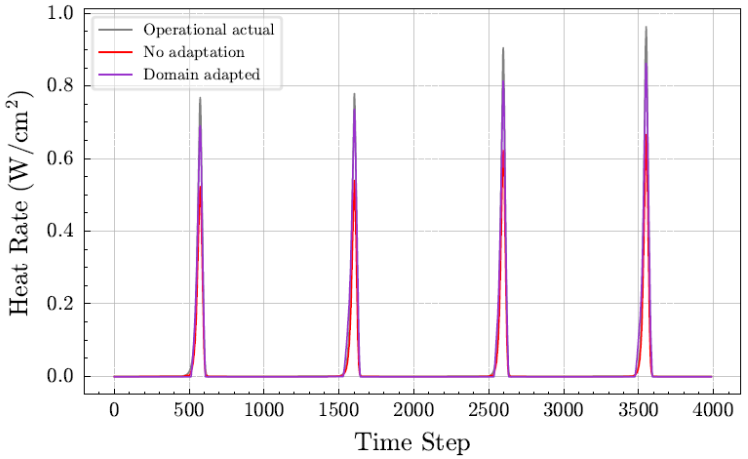


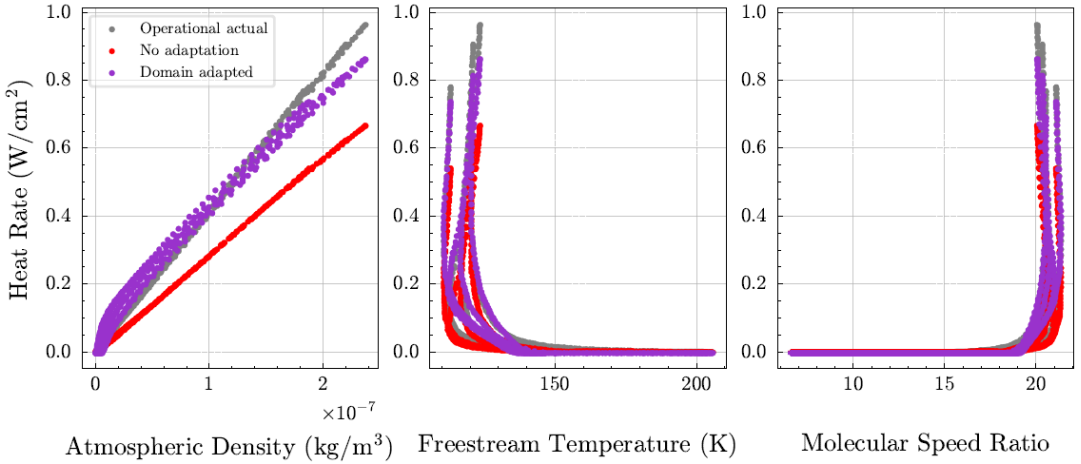




Extrapolatory mission (operational domain)







- Training in the aligned latent space allows the model to adapt to the distribution shift in the inputs
- Model predicts the output with lower error than a model w/o adaptation in the extrapolatory mission

Research Objective

Develop an unsupervised method to allow a system digital twin to retain predictive performance on out-of-distribution data in evolving, unknown domains, such as changing environmental conditions, operating conditions, or configurations.

Overarching Hypothesis

If subspace tracking and manifold alignment are used to learn a common low-dimensional latent space of time-delay embedded data, then a data-driven digital twin trained in this latent space can retain predictive performance in evolving, unknown domains.

How effective is the method?

Research Question 1	Research Question 2	Research Question 3
How does the size of the domain shift affect the performance of PSA?	How does PSA perform on sequential, streaming data?	How does PSA perform under different noise levels?
Experiment 1	Experiment 2	Experiment 3

Experiments Overview

Experiments

Effectiveness is tested in three scenarios:

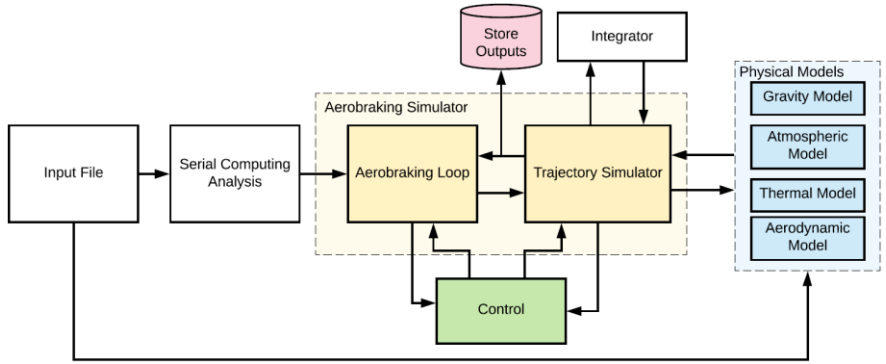
- *Generalization*: vary distance between domains by adjusting geometry of orbits
- *Online learning*: train model for increasing percentages of available data from the operational domain
- *Noisy data*: contaminate the operational inputs with noise, repeat for different levels of noise

Testing Details

- Simulated data is generated using Falcone and Putnam's ABTS and Mars-GRAM 2010
- PSA is compared against a baseline model without adaptation
- Tests are performed for three types of shifts in aerobraking orbits

Trajectory Simulation

- For all experiments, Falcone and Putnam's Aerobraking Trajectory Simulator (ABTS) [1] is run for the following parameters:
  - *Planet*: Mars
  - *Gravity*: Inverse-squared law with J2 effects
  - *Atmospheric Model*: Mars-GRAM 2010 [2]
  - *Aerodynamic Model*: Mach-dependent relation for rarefied gas
  - *Thermal Model*: Maxwellian heat transfer theory
  - *Initial Conditions*: Orbital elements
- Assume Mars Odyssey-like vehicle
  - Dry/prop mass, solar array area, geometry, thermal coefficient based on Odyssey mission [3]
  - Flow is assumed to be always normal to solar panel surface



ABTS architecture, from Falcone and Putnam [1].

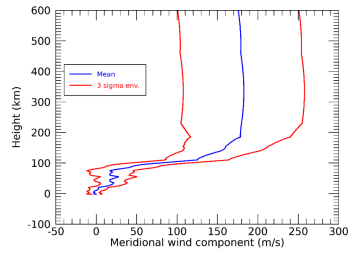


Figure 5. Three-sigma envelope of sample Mars-GRAM north (positive)/south (negative) meridional wind outputs for 32.333° N latitude and -117.8° E longitude.

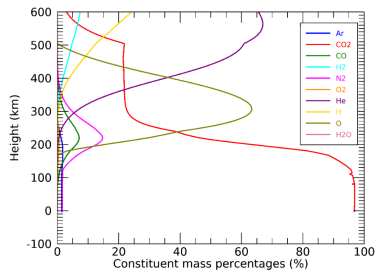


Figure 6. Sample Mars-GRAM mean constituent contributions by percent.

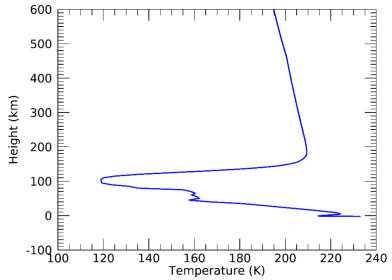


Figure 2. Height versus temperature from a sample Mars-GRAM output.

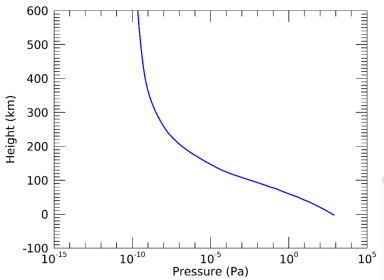
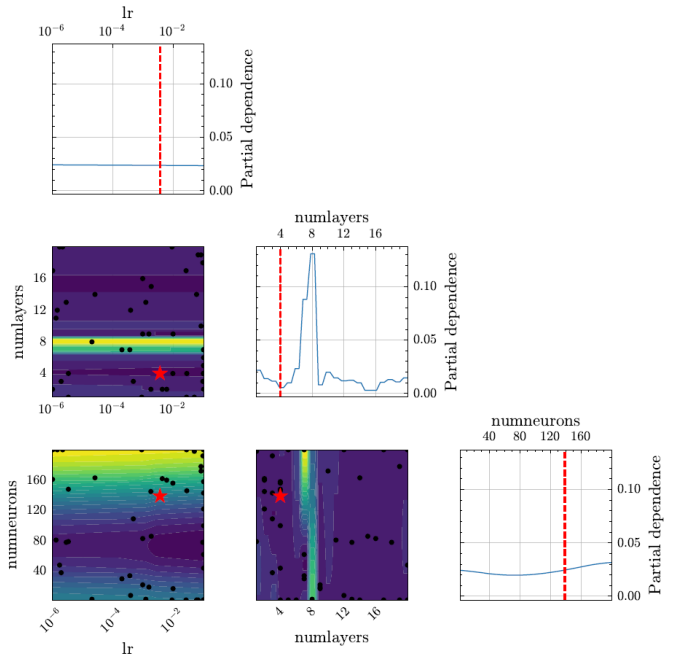


Figure 3. Height versus pressure from a sample Mars-GRAM output.

Mars-GRAM 2010 sample output, from Justh et al. [2].

Baseline Model and Hyperparameters

- **PSA is compared against a baseline model without adaptation**
- Both baseline and PSA models will be deep neural networks with identical architecture and hyperparameters
  - NN has reputation as “universal function approximator”
  - Results should be valid across other model choices because PSA is independent of training and model architecture
- Hyperparameters determined by Bayesian optimization [1-3]
  - Manual tuning performed afterwards
  - Plotted loss landscape to check training consistency/difficulty [4,5]

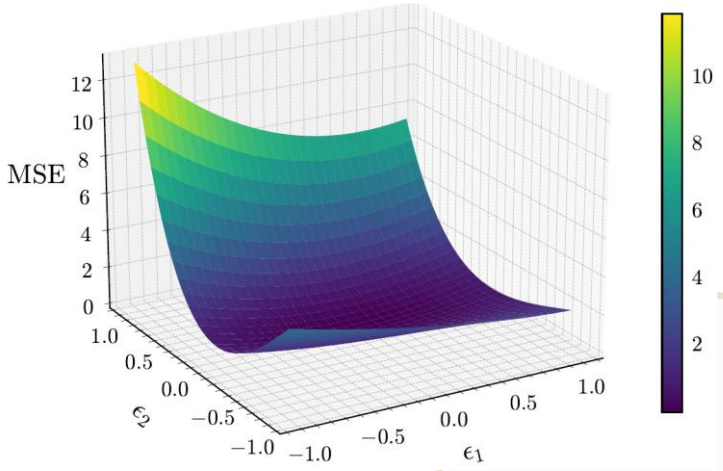


Bayesian optimization to determine learning rate, number of hidden layers, and number of neurons in network.

Calculate the Hessian of the loss function $\frac{\partial \mathcal{L}}{\partial \tilde{\boldsymbol{\theta}}} \in \mathbb{R}^N$ $\boldsymbol{H} = \frac{\partial^2 \mathcal{L}}{\partial \tilde{\boldsymbol{\theta}}^2} \in \mathbb{R}^{N \times N}$ $\boldsymbol{H} = \boldsymbol{E}\boldsymbol{\Lambda}\boldsymbol{E}^{-1}$

$$g(a,b) = \mathcal{L}(\tilde{\boldsymbol{\theta}}^* + a\boldsymbol{\epsilon}_1 + b\boldsymbol{\epsilon}_2)$$

Perturb the trained model parameters along the first and second eigenvector of the Hessian



Loss landscape of baseline model.

Types of Domain Shifts

To test the model, I run PSA for aerobraking orbits with different geometries

- Described by eccentricity e, semi-major axis a, semi-latus rectum p, periapsis r_p, and apoapsis r_a
- *Number of orbits remains constant*
- *DRM is the same across all experiments*

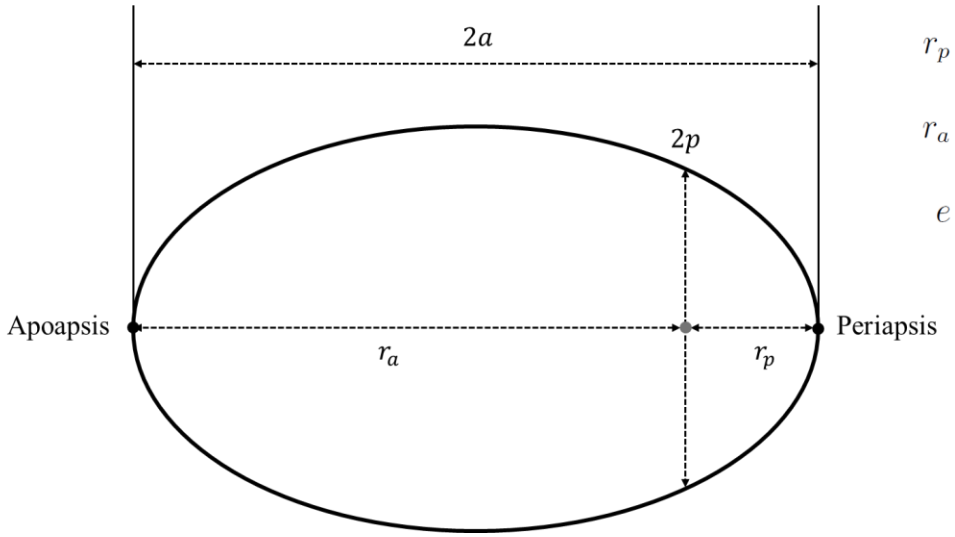
Test for three types of shifts:

$$p = a(1 - e^2)$$

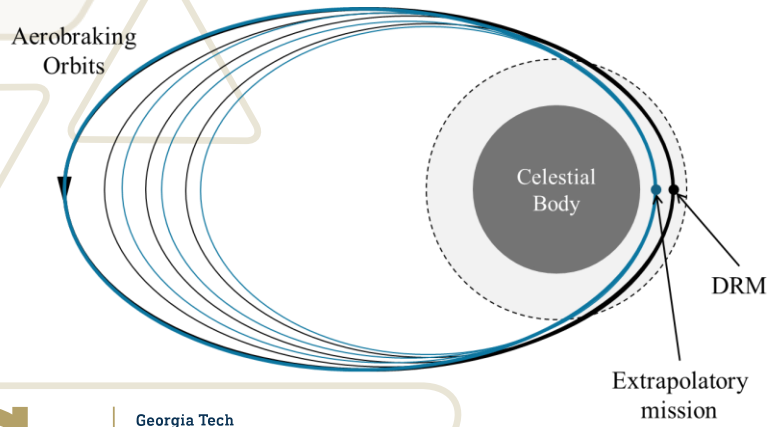
$$r_p = \frac{p}{1+e} = a(1-e)$$

$$r_a = \frac{p}{1-e} = a(1+e)$$

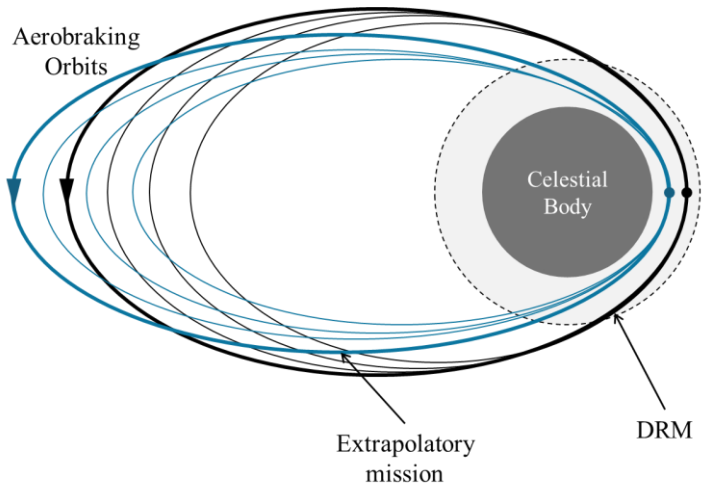
$$e = \frac{r_a - r_p}{r_p + r_a}$$



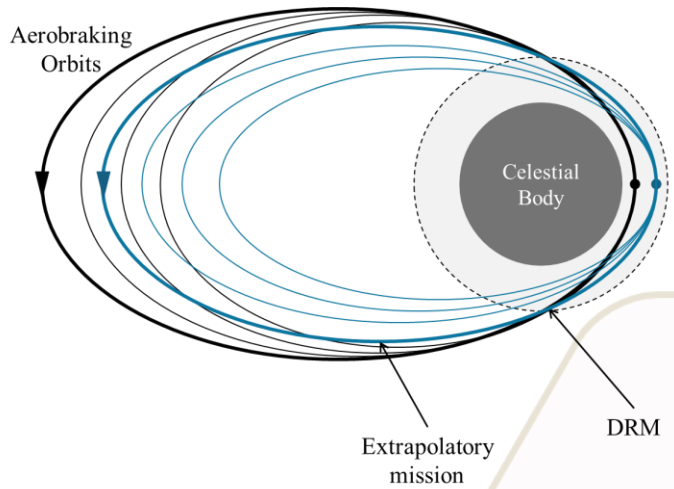
I. Periapsis shift



II. Eccentricity shift with fixed semi-major axis



III. Eccentricity shift with fixed semi-latus rectum



Experiment 1: Generalization

How does the size of the domain shift affect the performance of PSA?

Hypothesis: PSA should improve model accuracy on out-of-distribution data if the distance between domain subspaces is sufficiently small, but its effectiveness will decrease as the subspace distance increases.

To test this:

- Track distance between the subspaces in each domain by:

Pre-adaptation distance $$d(\boldsymbol{H}_{X_t,\text{sub}}, \boldsymbol{H}_{Z_t,\text{sub}}) = \|\tilde{\boldsymbol{H}}_{X_t,\text{proj}} - \tilde{\boldsymbol{H}}_{Z_t,\text{proj}}\|_2$$

Post-adaptation distance $$d(\boldsymbol{H}_{X_t,\text{sub}}, s\boldsymbol{Q}\boldsymbol{H}^T_{Z_t,\text{sub}}) = \|\tilde{\boldsymbol{H}}_{X_t,\text{proj}} - s\boldsymbol{Q}\tilde{\boldsymbol{H}}_{Z_t,\text{proj}}\|_2$$

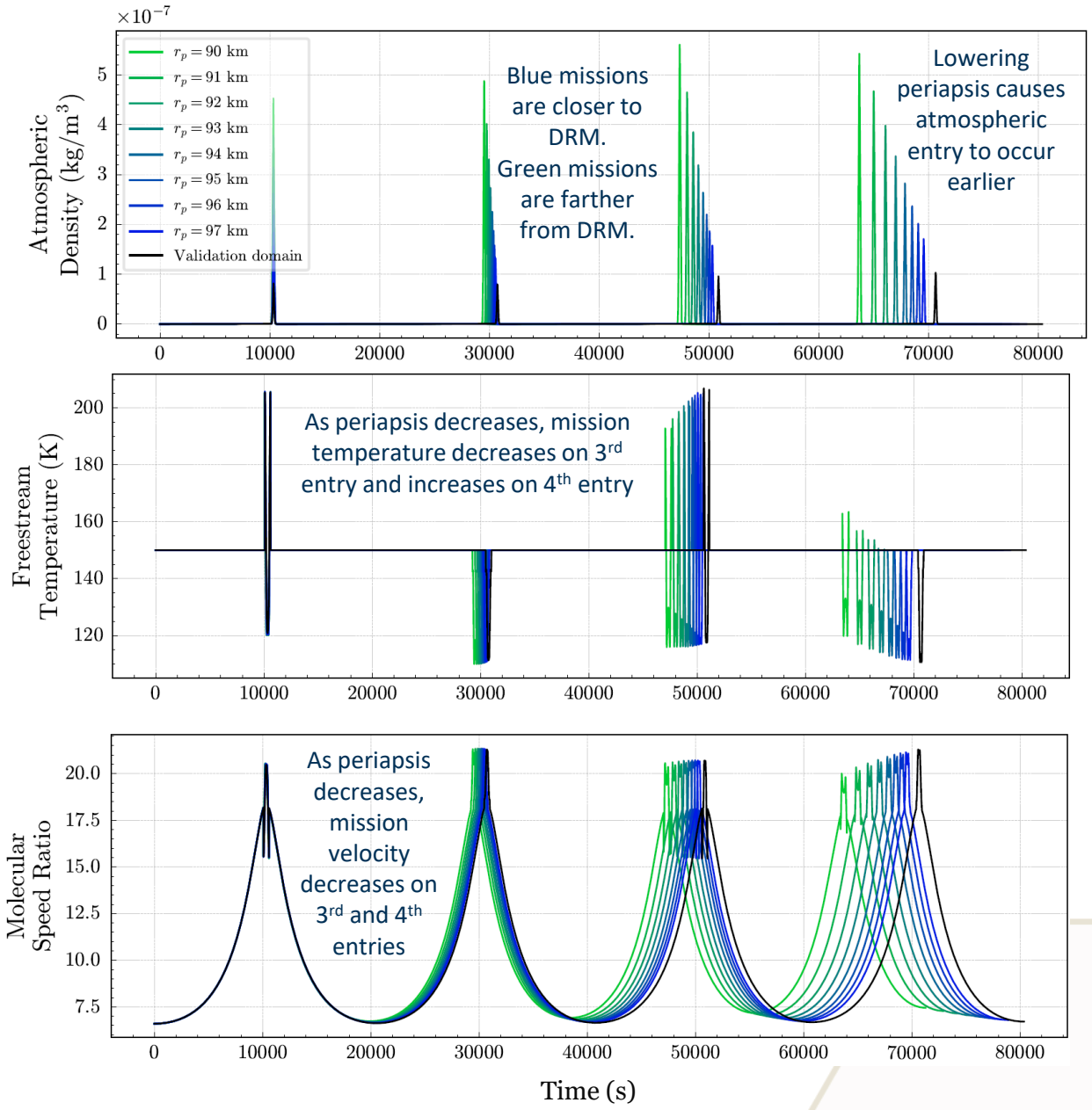
- Run PSA for each type of shifted orbit, and for increasingly larger shifts
- Track subspace distance, RMSE, and R^2 for all shifted missions

1. Specify list of mission parameters
2. Generate trajectory data for each mission
3. Run PSA on each mission
4. Return RMSE and R^2 of adapted model

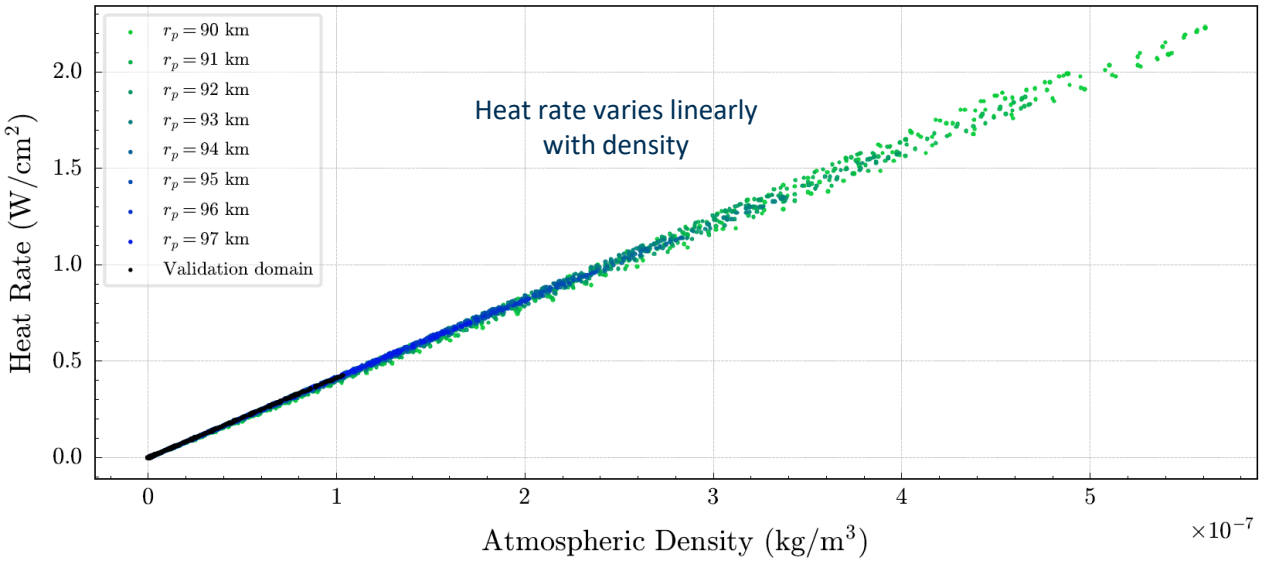
I. Periapsis Shift: Time-Series Data

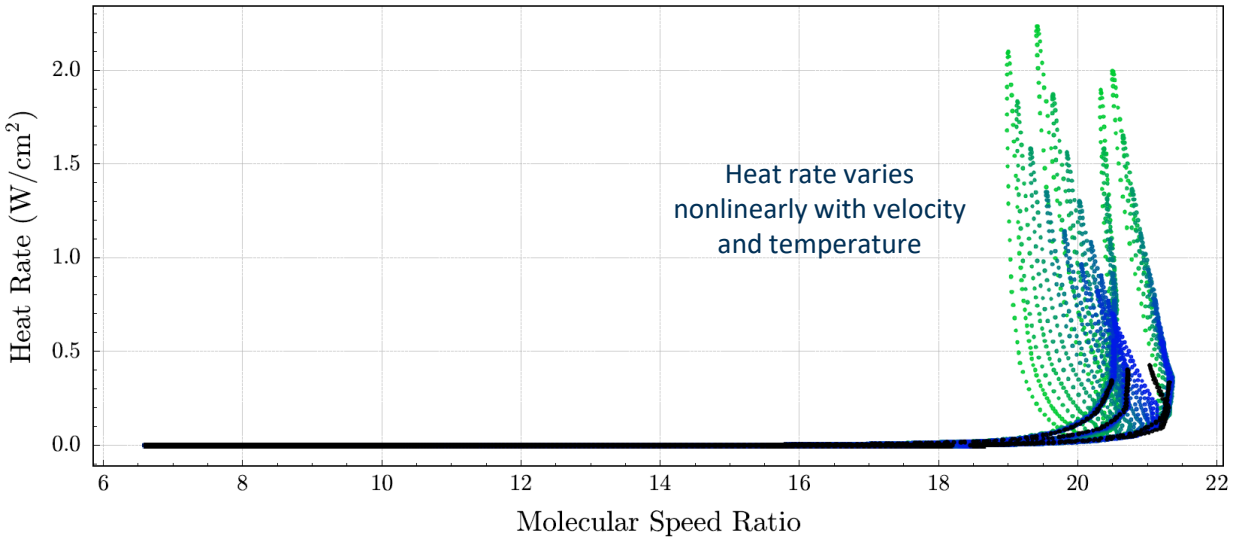
Mission Name	Number of Orbits	r_a (km)	r_p (km)	a (km)	p (km)	e
DRM	4	12000.0	100.00	6050.0	198.35	0.98347
Mission 1	4	12000.0	97.000	6048.5	192.44	0.98396
Mission 2	4	12000.0	96.000	6048.0	190.48	0.98413
Mission 3	4	12000.0	95.000	6047.5	188.51	0.98429
Mission 4	4	12000.0	94.000	6047.0	186.54	0.98446
Mission 5	4	12000.0	93.000	6046.5	184.57	0.98462
Mission 6	4	12000.0	92.000	6046.0	182.60	0.98478
Mission 7	4	12000.0	91.000	6045.5	180.63	0.98495
Mission 8	4	12000.0	90.000	6045.0	178.66	0.98511

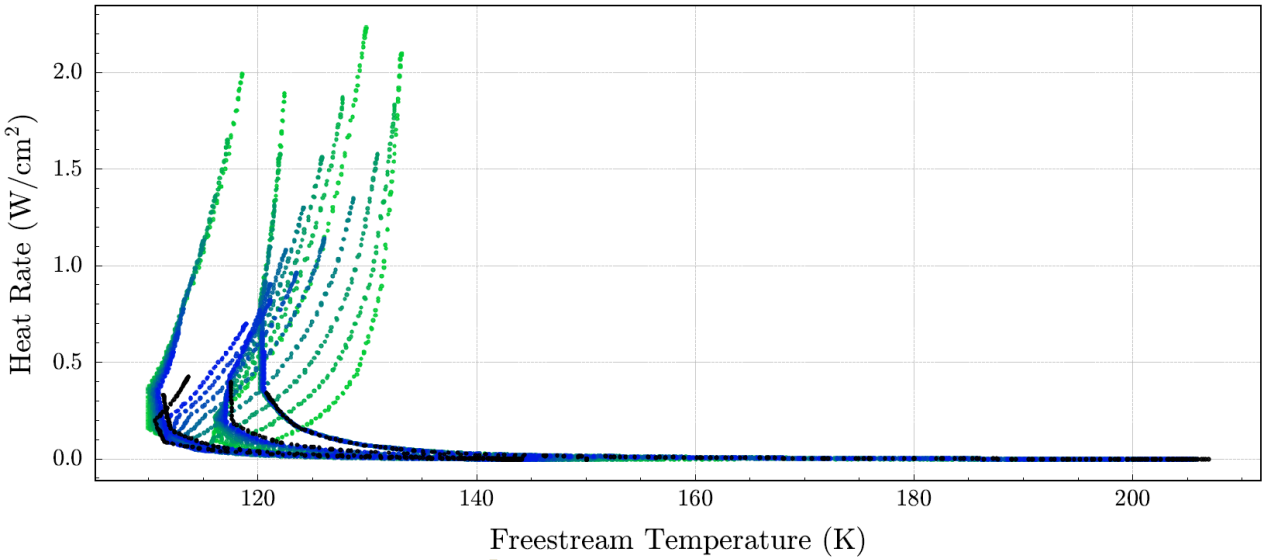
- Periapsis is lowered from 100 km to 90 km in Missions 1-8
- Model inputs vs. time are shown on right
  - Higher densities and heat rates
  - Atmospheric entry occurs earlier
- Peaks/periodicities in data are caused by drag passages

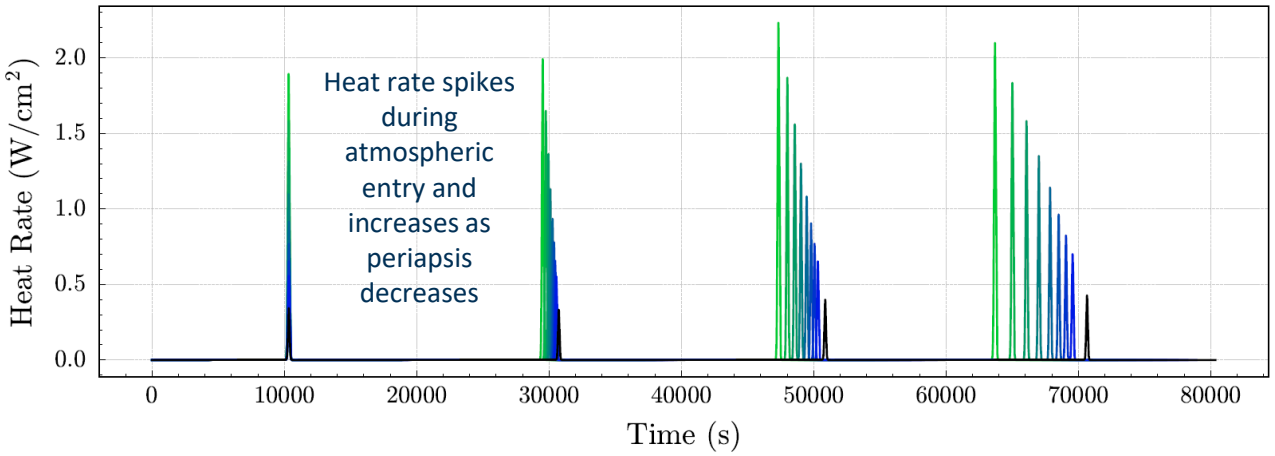


I. Periapsis Shift: State Space

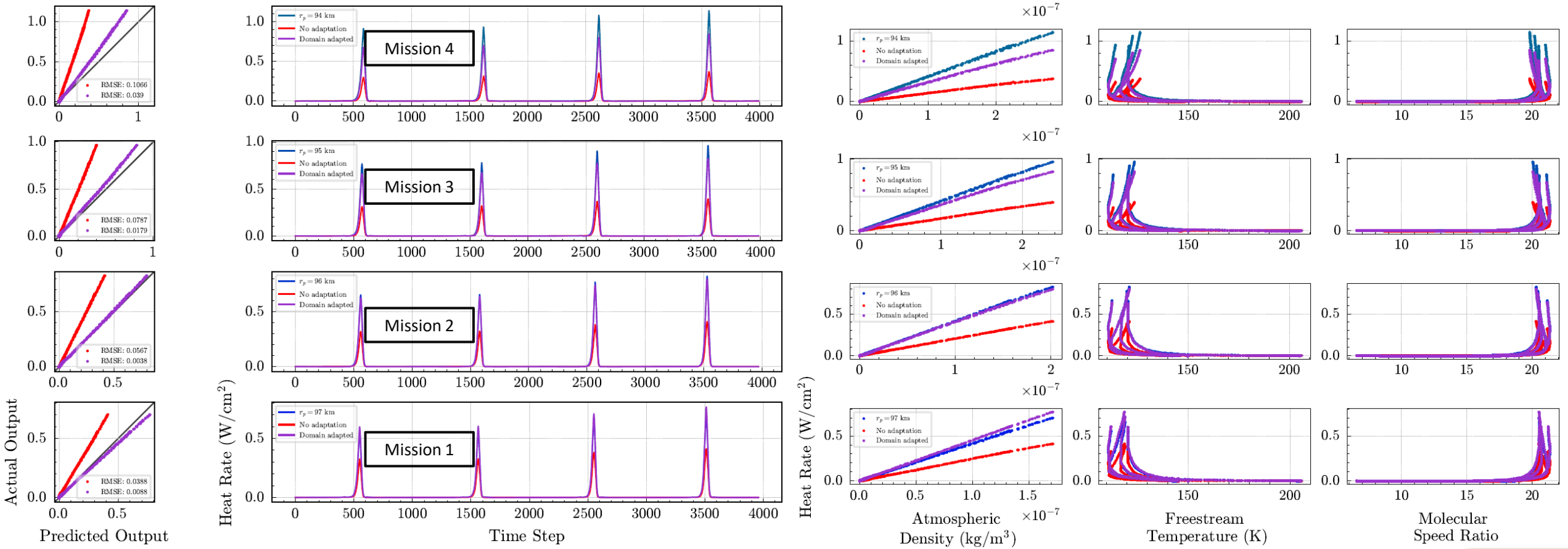






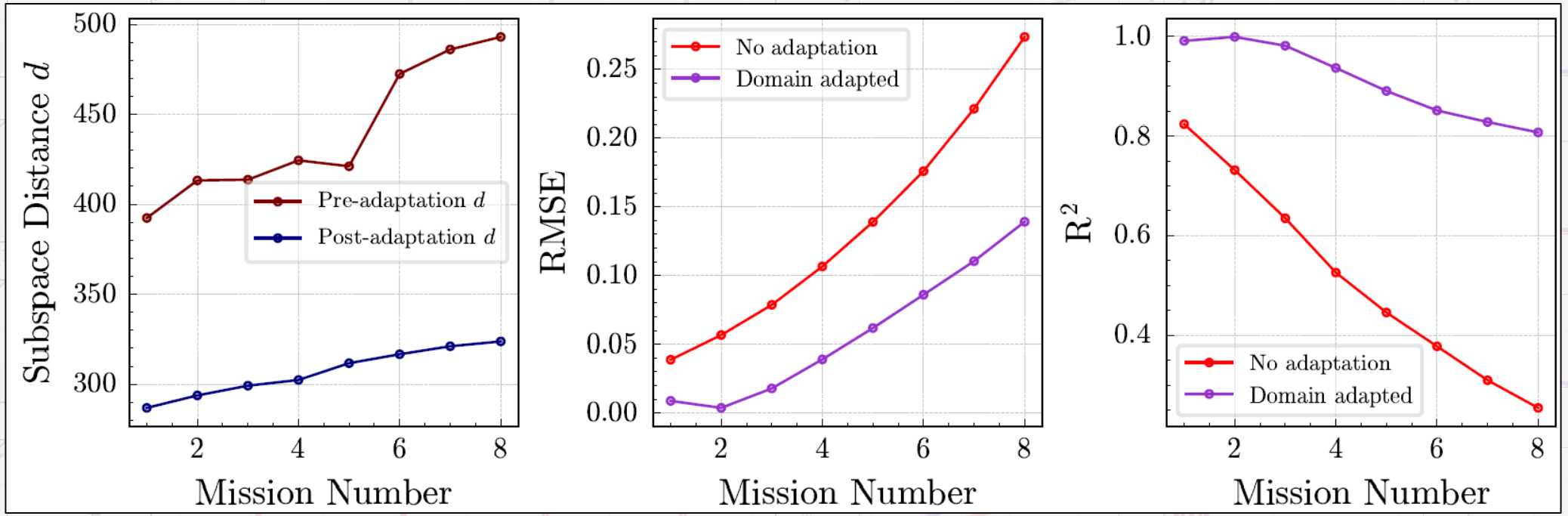


I. Periapsis Shift: Results



- Model predicts the heat rate with lower error than the baseline model across all missions
- Performance degrades as the periapsis decreases

I. Periapsis Shift: Results

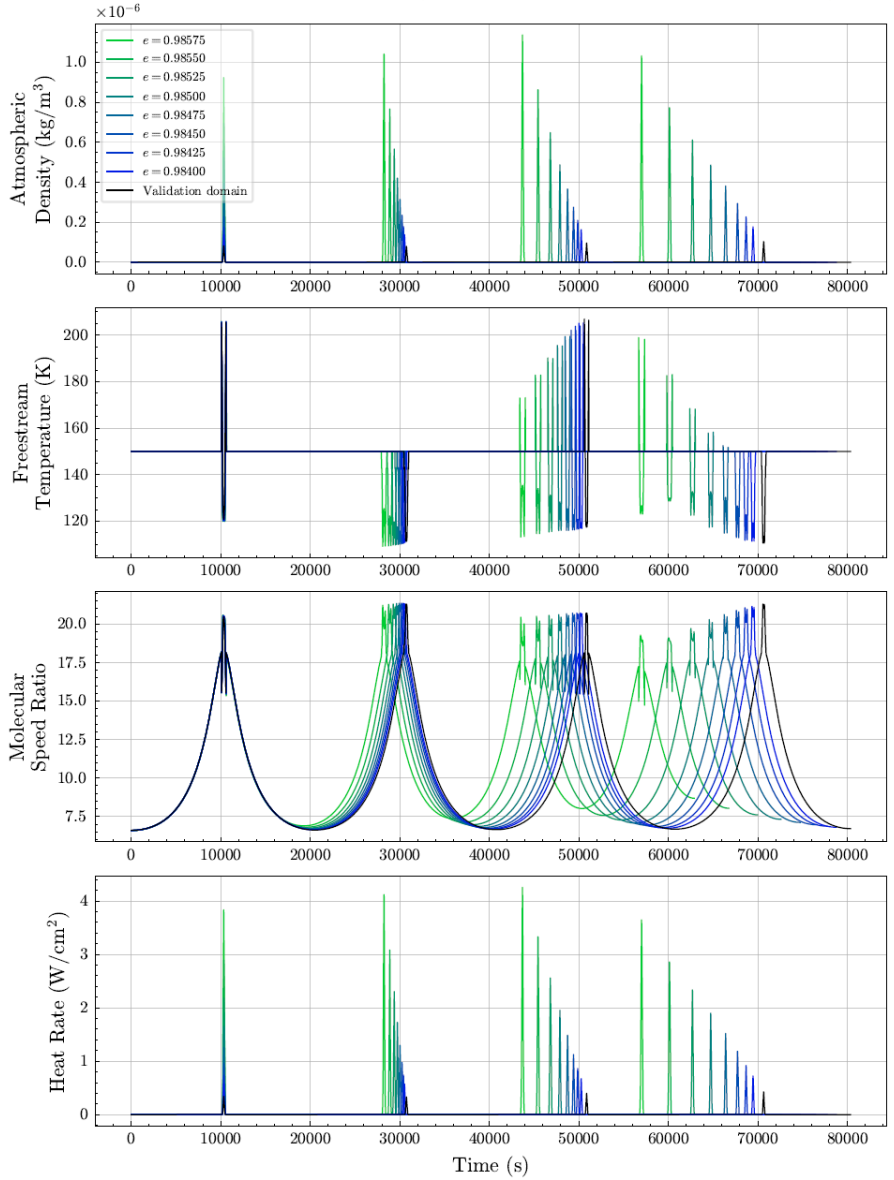


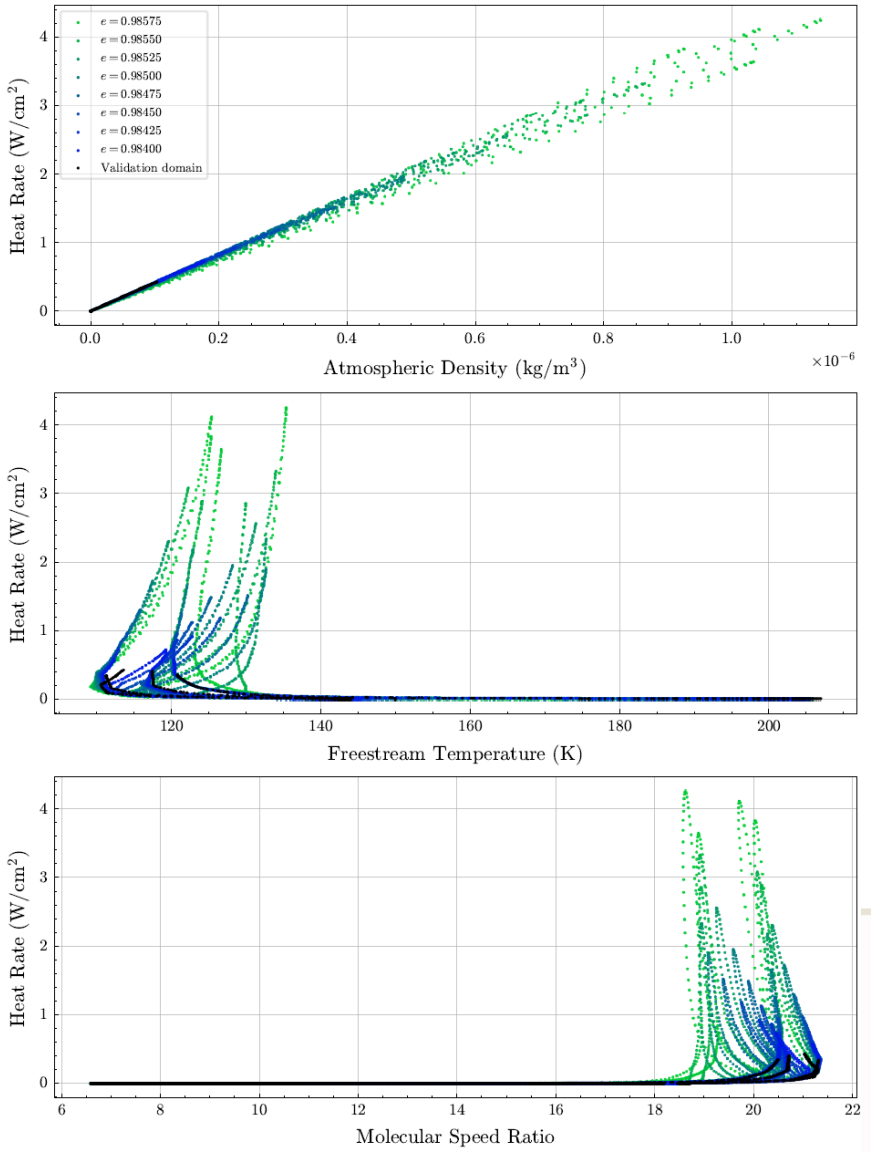
- RMSE increases and R^2 decreases as domain shift increases
- Missions with lower periapsis (and higher mission numbers) have larger subspace distances
- Manifold alignment struggles to scale the data correctly at higher shifts

II. Eccentricity Shift, Fixed Semi-Major Axis: Trajectory Data

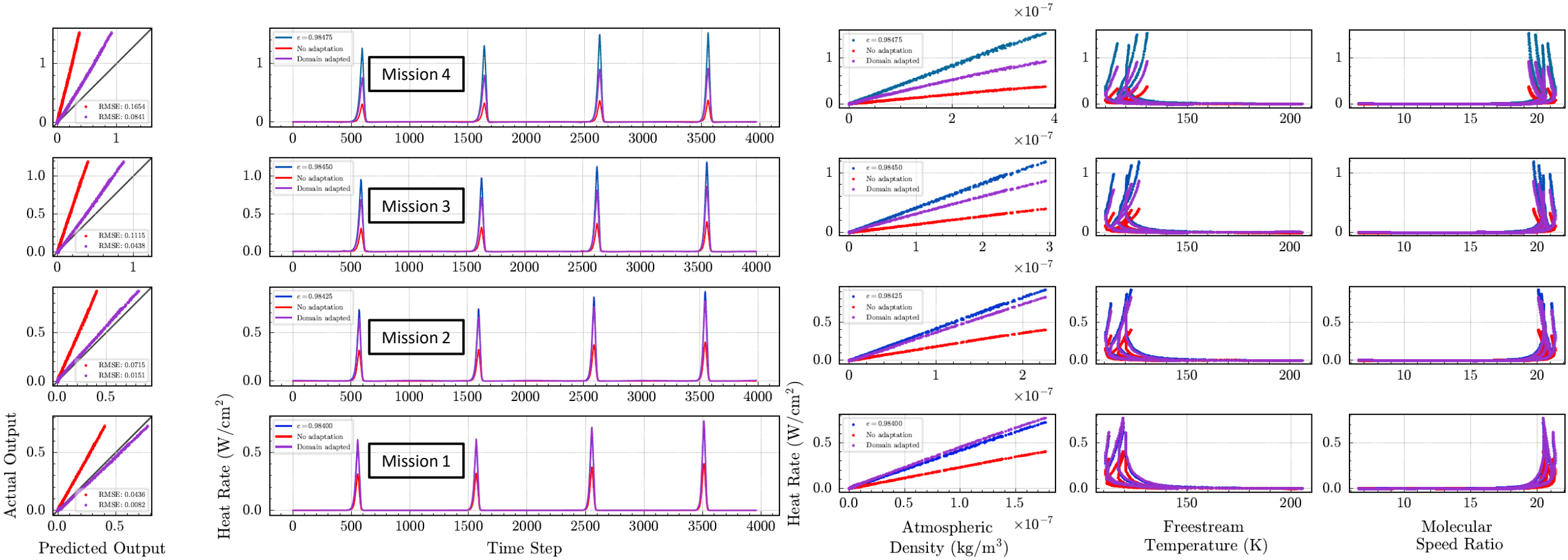
Mission Name	Number of Orbits	r_a (km)	r_p (km)	a (km)	p (km)	e
DRM	4	12000.0	100.00	6050.0	198.35	0.98347
Mission 1	4	12003.2	96.800	6050.0	192.05	0.98400
Mission 2	4	12004.7	95.288	6050.0	189.07	0.98425
Mission 3	4	12006.2	93.775	6050.0	186.10	0.98450
Mission 4	4	12007.7	92.262	6050.0	183.12	0.98475
Mission 5	4	12009.3	90.750	6050.0	180.14	0.98500
Mission 6	4	12010.8	89.238	6050.0	177.16	0.98525
Mission 7	4	12012.3	87.725	6050.0	174.18	0.98550
Mission 8	4	12013.8	86.212	6050.0	171.20	0.98575

- Eccentricity is raised from 0.98347 to 0.98575 in Missions 1-8
- Semi-major axis is fixed at 6050 km
- Data vs. time and heat rate vs. model inputs are shown on right
- Much higher densities, heat rates, and earlier entry times compared to periapsis shift



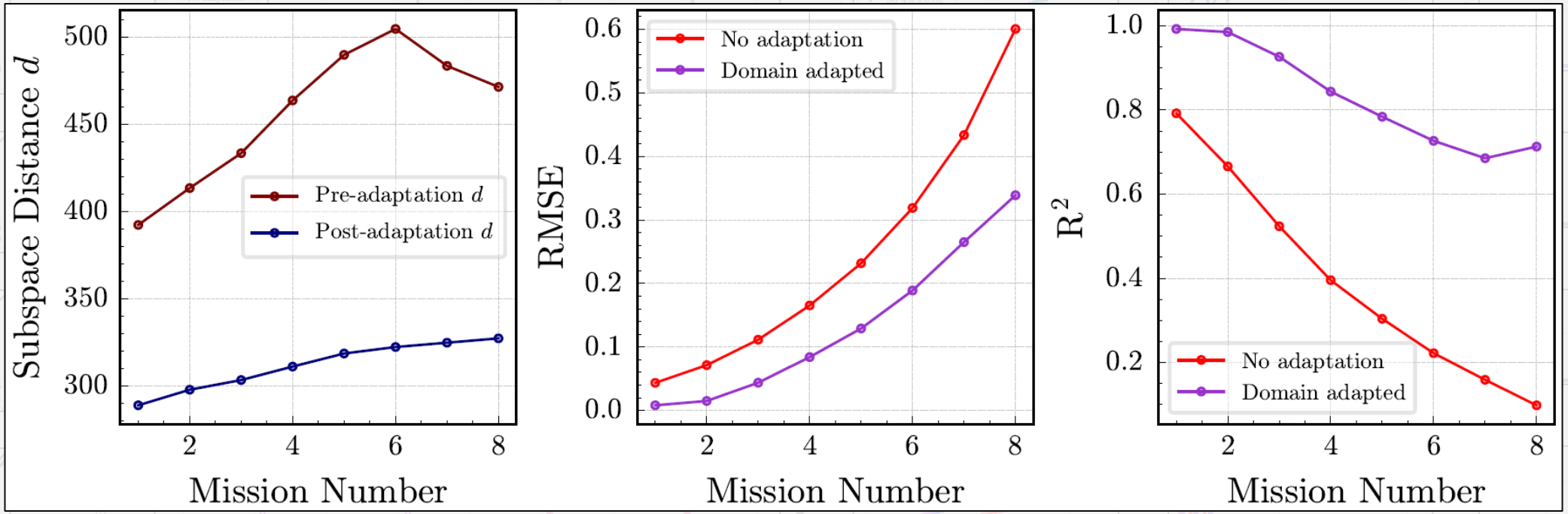


II. Eccentricity Shift, Fixed Semi-Major Axis: Results



- Model predicts the heat rate with lower error than the baseline model across all missions
- Performance degrades as eccentricity increases

II. Eccentricity Shift, Fixed Semi-Major Axis: Results

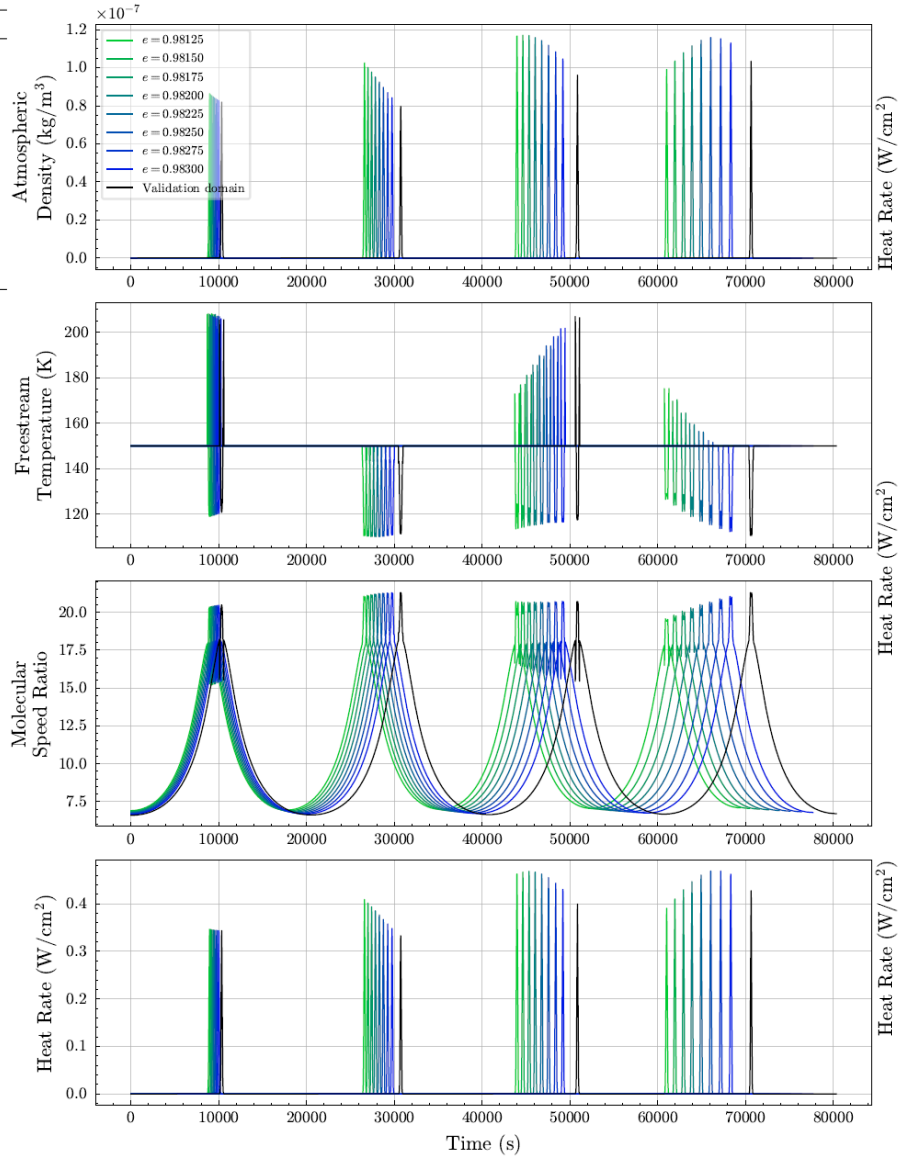


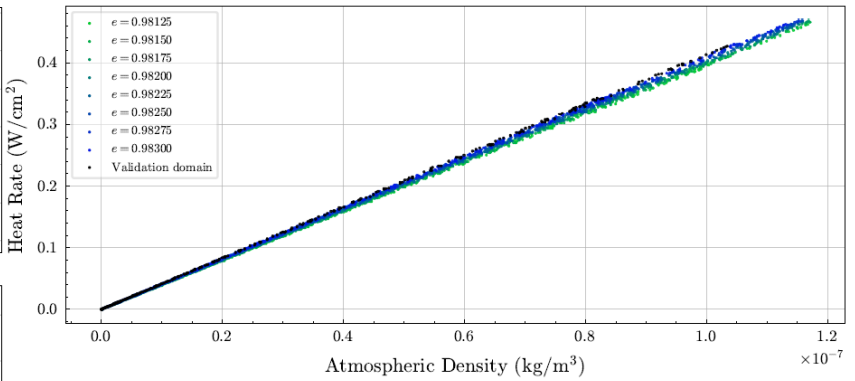
- RMSE increases and R^2 decreases as domain shift (mission number) increases
- As eccentricity increases (for fixed semi-major axis), the subspace distance increases

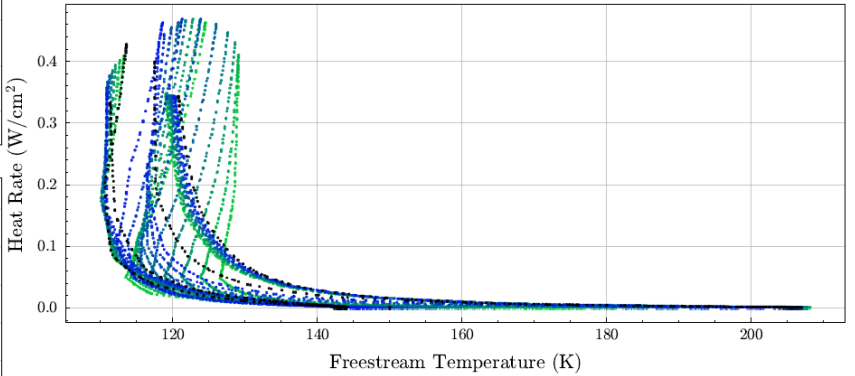
III. Eccentricity Shift, Fixed Semi-Latus Rectum: Trajectory Data

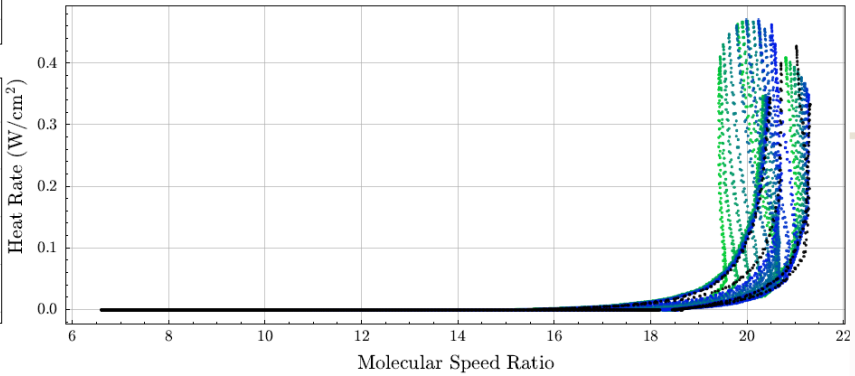
Mission Name	Number of Orbits	r_a (km)	r_p (km)	a (km)	p (km)	e
DRM	4	12000.0	100.00	6050.0	198.35	0.98347
Mission 1	4	11667.5	100.02	5883.8	198.35	0.98300
Mission 2	4	11498.4	100.04	5799.2	198.35	0.98275
Mission 3	4	11334.1	100.05	5717.1	198.35	0.98250
Mission 4	4	11174.5	100.06	5637.3	198.35	0.98225
Mission 5	4	11019.3	100.07	5559.7	198.35	0.98200
Mission 6	4	10868.3	100.09	5484.2	198.35	0.98175
Mission 7	4	10721.5	100.10	5410.8	198.35	0.98150
Mission 8	4	10578.5	100.11	5339.3	198.35	0.98125

- Eccentricity is lowered from 0.98347 to 0.98125 in Missions 1-8
- Semi-latus rectum is fixed at 198.35 km
- Data vs. time and heat rate vs. model inputs are shown on right
- Density, heat rate, similar in magnitude between missions
- Much earlier entry times, decrease in temperature at last drag passage

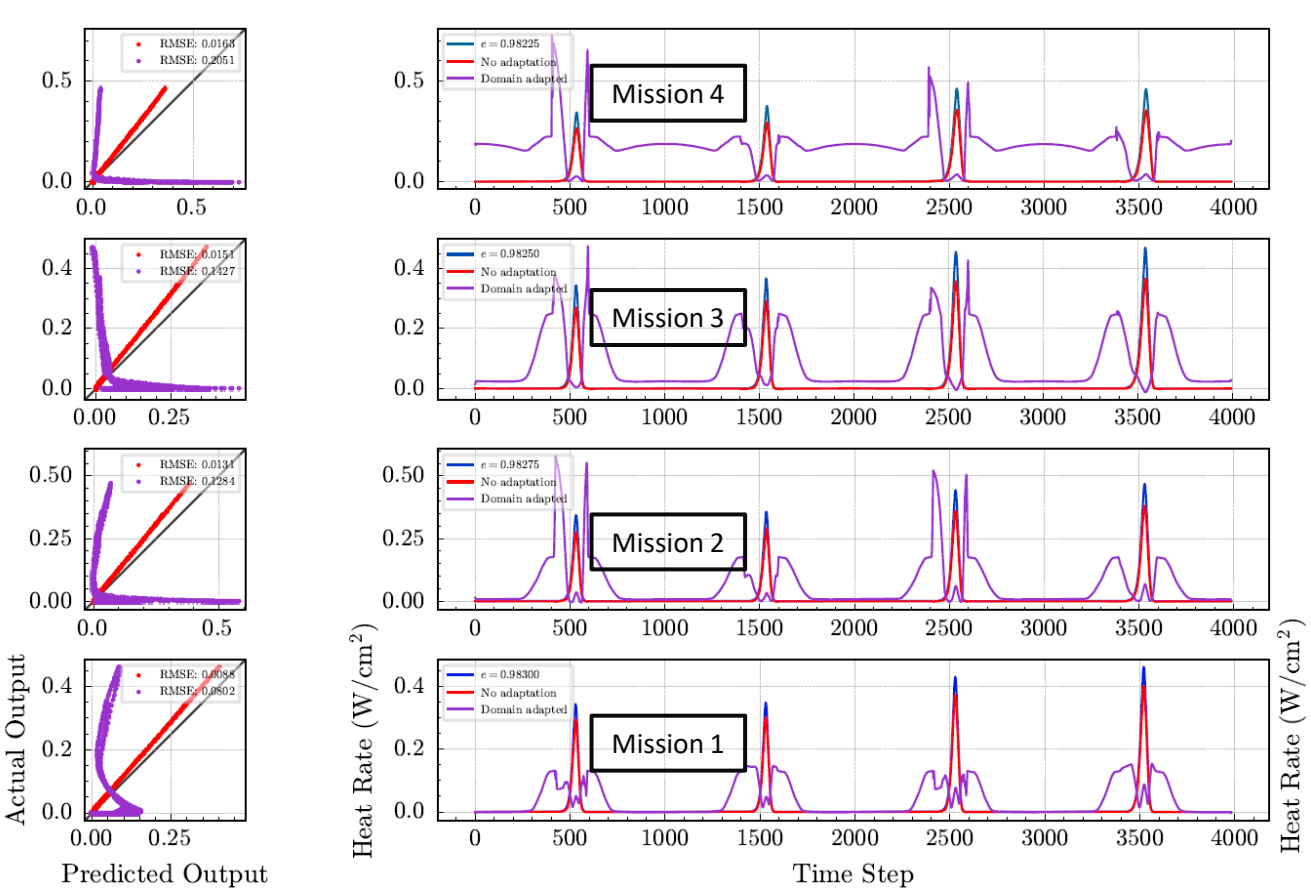


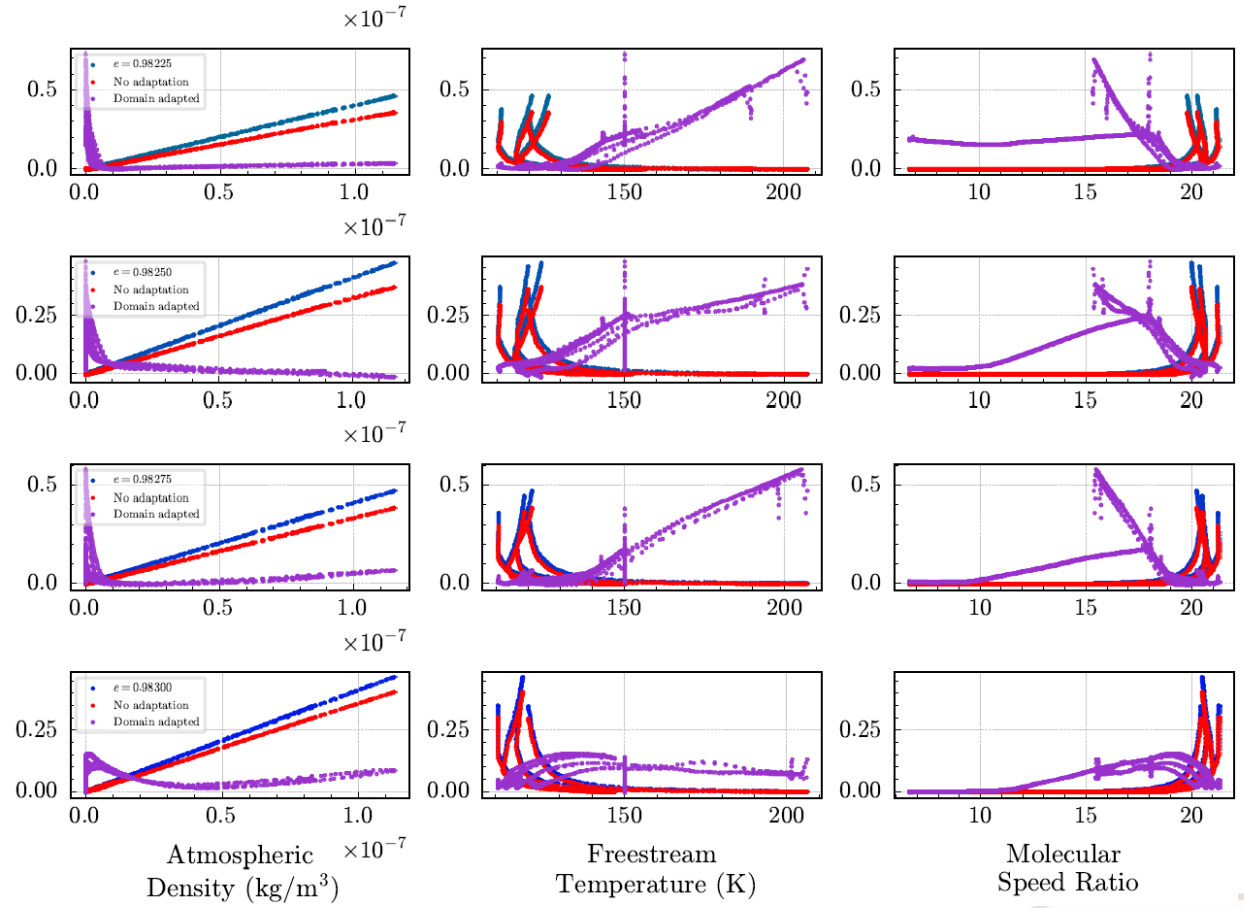






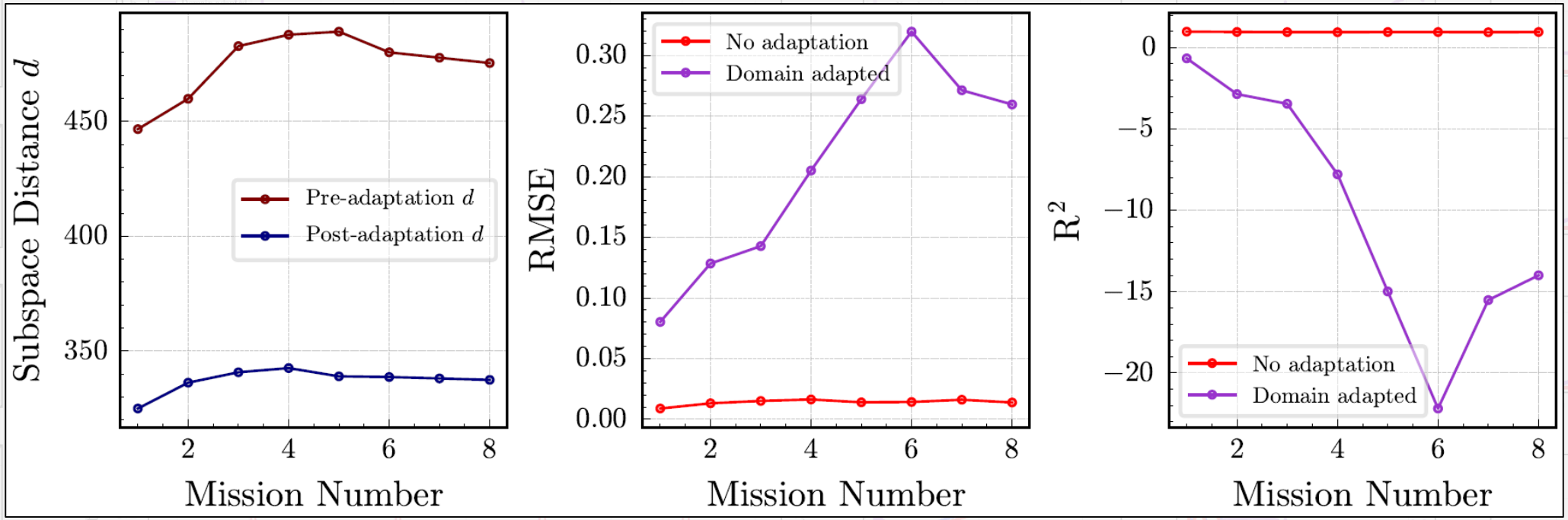
III. Eccentricity Shift, Fixed Semi-Latus Rectum: Results





- Model fails to adapt to any mission
- Results are so poor, the R^2 is negative

III. Eccentricity Shift, Fixed Semi-Latus Rectum: Results

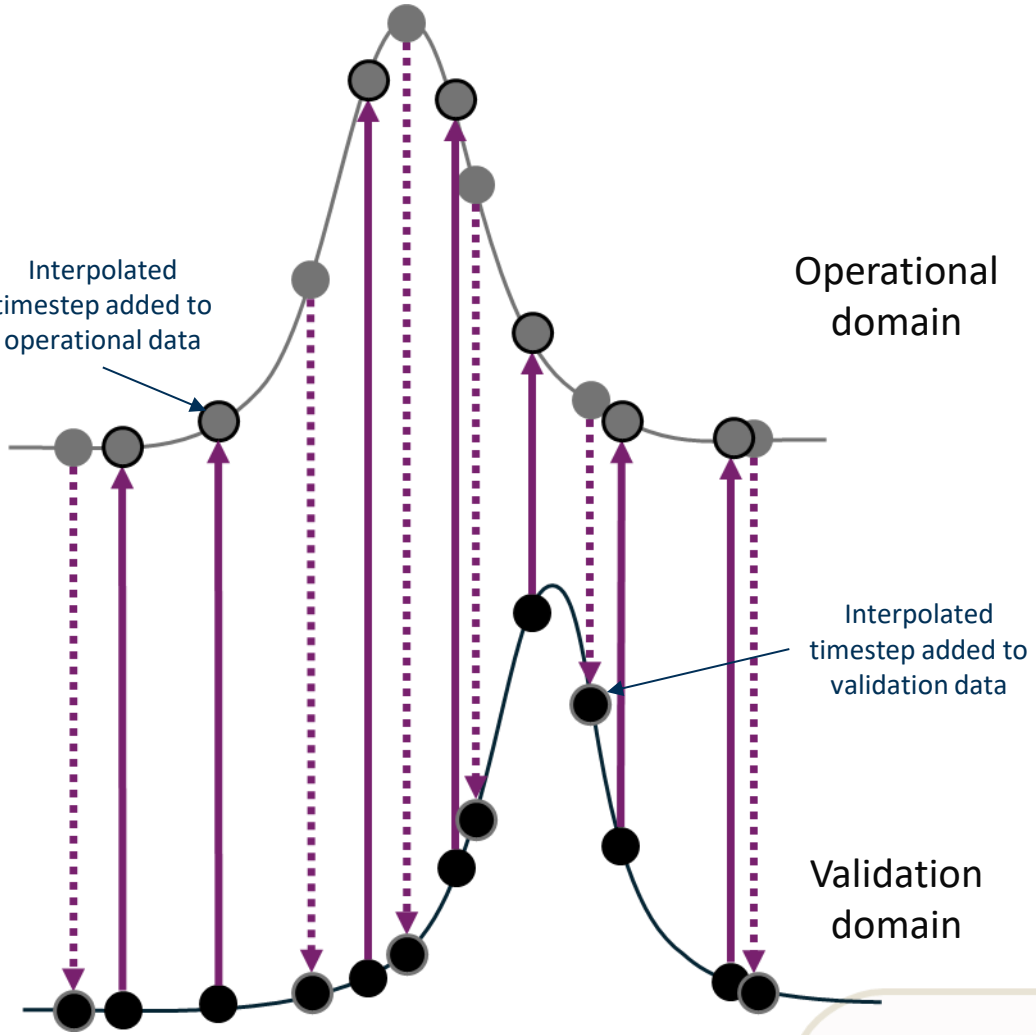


- Model fails to adapt to any mission
- Results are so poor, the R^2 is negative

Pairwise Correspondence and Interpolation

Why does adaptation fail on this type of shift?

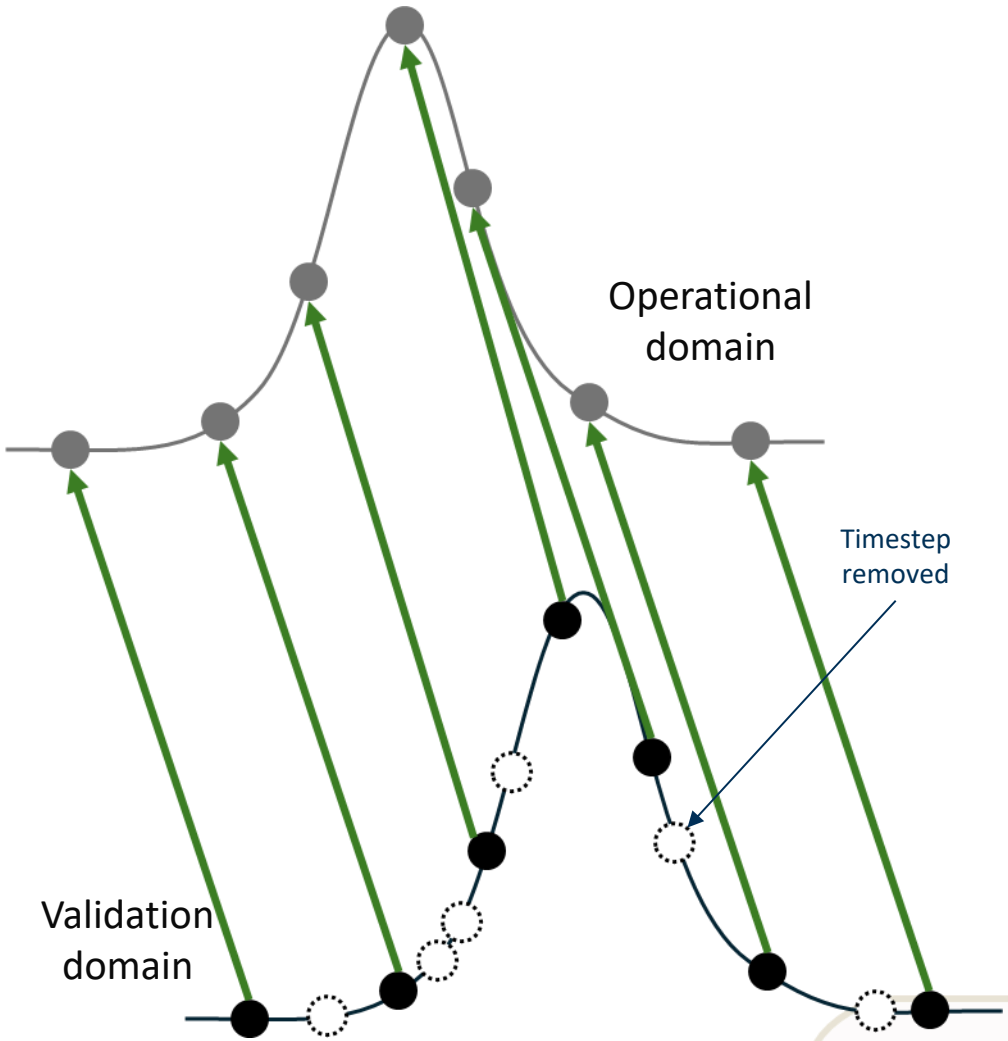
- Procrustes analysis requires "pairwise correspondence" between the data
  - i.e., each domain must have an equal number of points
  - **To establish correspondence, PSA interpolates the data to a common set of timesteps**
- Interpolating in time introduces spurious correspondence between points
  - First drag passage occurs earlier than all other shifts
  - Entire data set is shifted in time, rather than just the last 3 passages
  - Data in drag passages are linked to points where spacecraft is in vacuum
- Interpolating in time also introduces new points, which affects the sampling
  - Potential bias towards zero/repeated values because the spacecraft is in vacuum for majority of mission



Interpolating to a common time to establish correspondence. In the interpolated inputs, a new data point is added to the domain if the other domain has a timestep that is absent.

Pairwise Correspondence and Interpolation

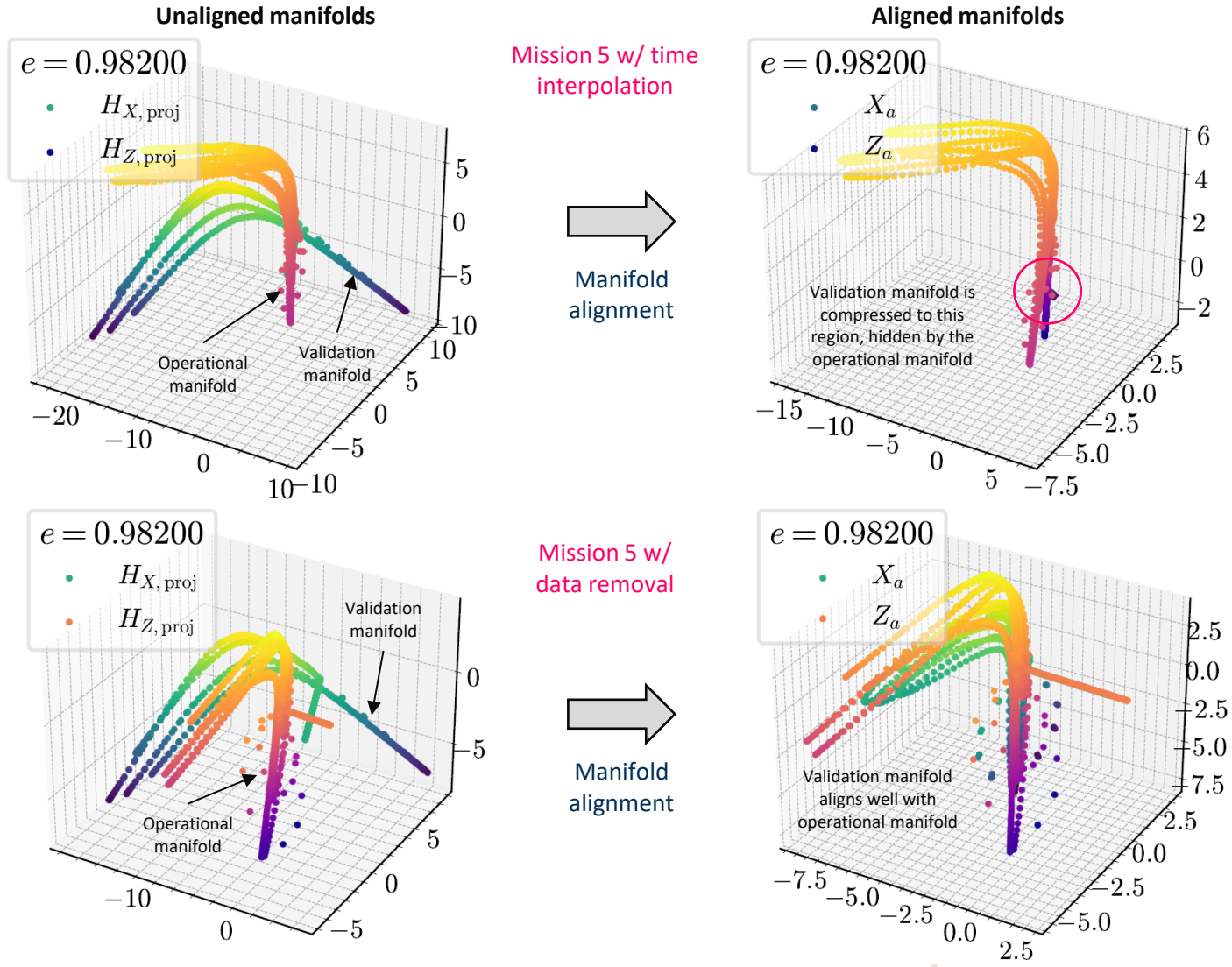
- To address this issue, assume the data has:
  - Similar number of points
  - Sampling rate and distribution are very similar in both domains
- Instead of interpolating the data, identify timesteps that correspond to common points in mission
- Remove any timestep that does not have a nearby corresponding timestep in the other domain
- With this, data corresponding to drag passages and vacuum should be linked better



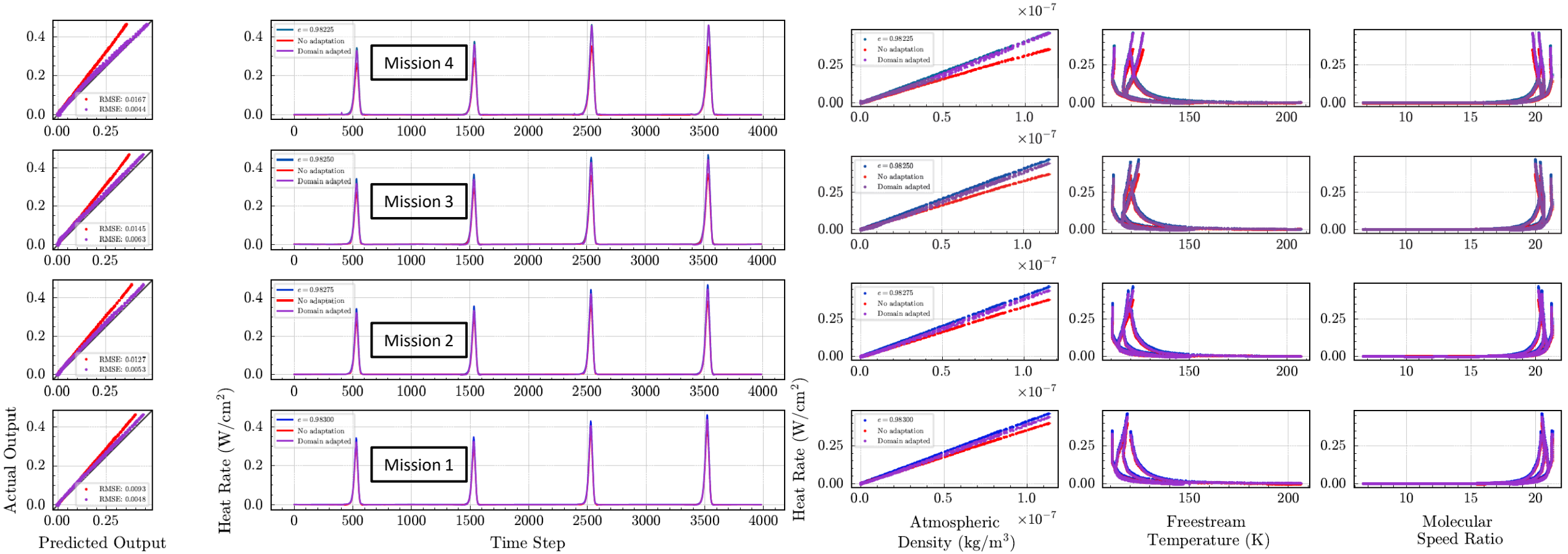
Remove data points that do not have a neighboring timestep in both domains to establish correspondence.

III. Eccentricity Shift, Fixed Semi-Latus Rectum: Manifolds

- Can verify if this correspondence approach works by plotting the manifolds
- Two correspondence approaches:
  - Interpolate data in time (top row)
  - Remove extraneous data (bottom row)
- Incorrect manifold alignment when data is interpolated
- Successful manifold alignment when data is removed

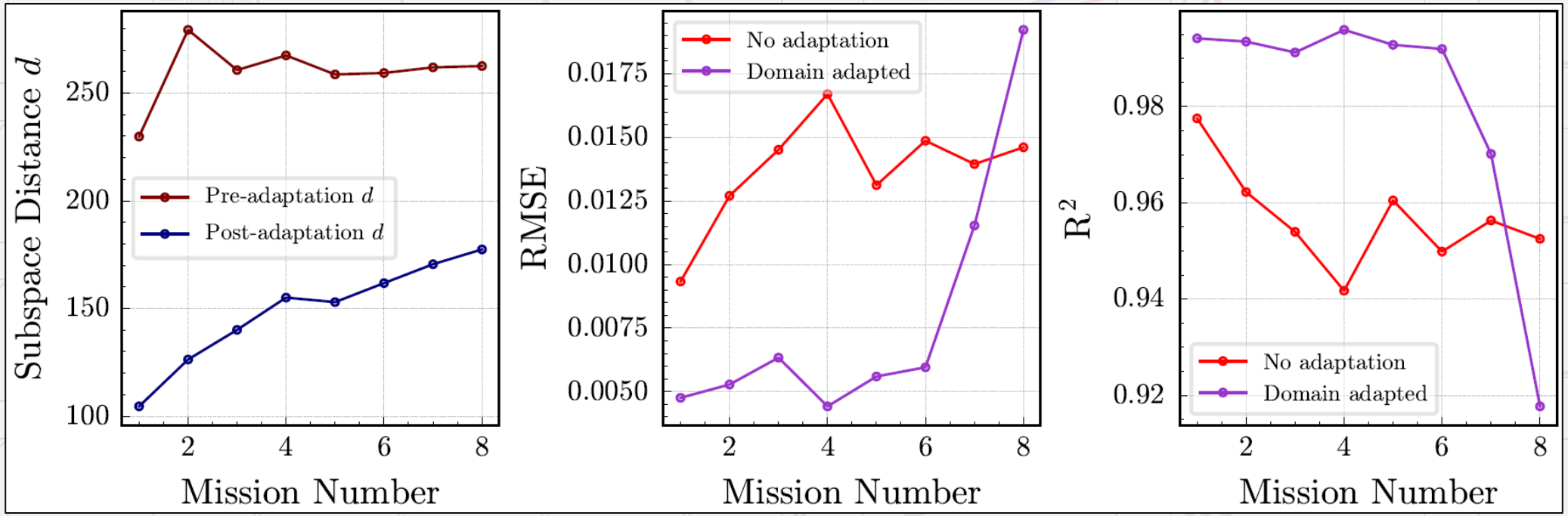


III. Eccentricity Shift, Fixed Semi-Latus Rectum: Results w/ Data Removal



- RMSE increases and R^2 decreases as domain shift increases
- Domain adapted model performs worse than baseline at Mission 8
- Subspace distance is significantly smaller than with interpolation

III. Eccentricity Shift, Fixed Semi-Latus Rectum: Results w/ Data Removal



- RMSE increases and R^2 decreases as domain shift increases
- Domain adapted model performs worse than baseline at Mission 8
- Subspace distance is significantly smaller than with interpolation

Experiment 1: Summary

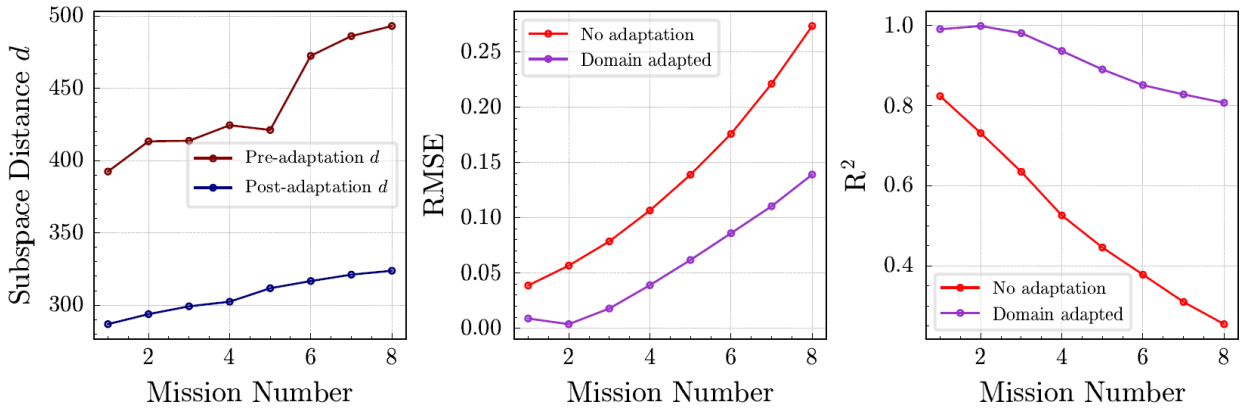
How does the size of the domain shift affect the performance of PSA?

Hypothesis: PSA should improve model accuracy on out-of-distribution data if the distance between domain subspaces is sufficiently small, but its effectiveness will decrease as the subspace distance increases.

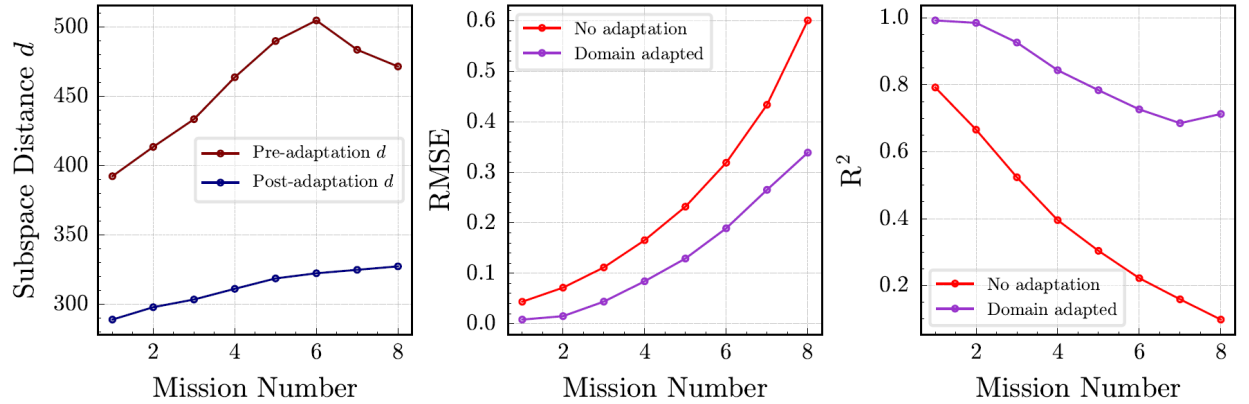
- RMSE is low at smaller domain shifts, increases as subspace distance (size of shift increases)

Hypothesis 1 validated

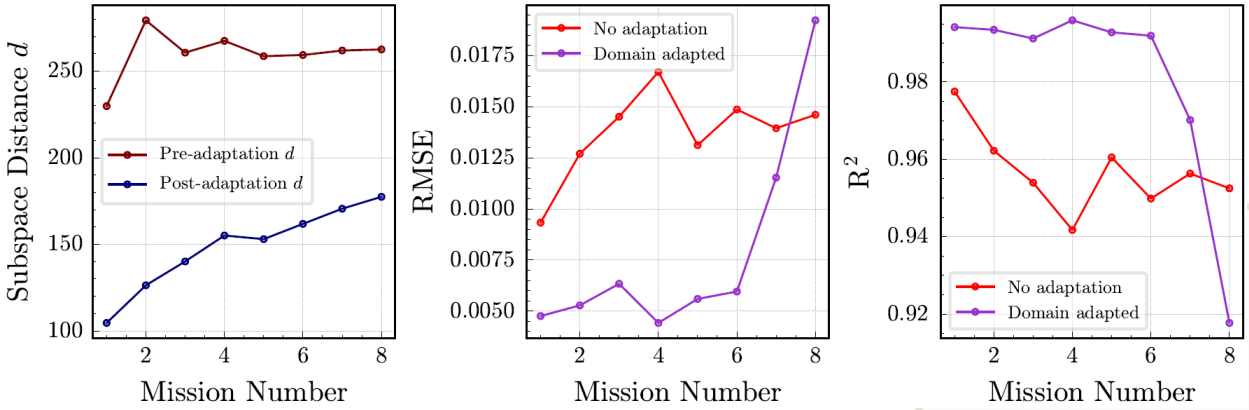
I. Periapsis shift



II. Eccentricity shift, fixed semi-major axis



III. Eccentricity shift, fixed semi-latus rectum



Research Objective

Develop an unsupervised method to allow a system digital twin to retain predictive performance on out-of-distribution data in evolving, unknown domains, such as changing environmental conditions, operating conditions, or configurations.

Overarching Hypothesis

If subspace tracking and manifold alignment are used to learn a common low-dimensional latent space of time-delay embedded data, then a data-driven digital twin trained in this latent space can retain predictive performance in evolving, unknown domains.

Research Question 1

How does the size of the domain shift affect the performance of PSA?

PSA performs well for small shifts, and its performance degrades for larger shifts

Research Question 2

How does PSA perform on sequential, streaming data?

Experiment 2

Research Question 3

How does PSA perform under different noise levels?

Experiment 3

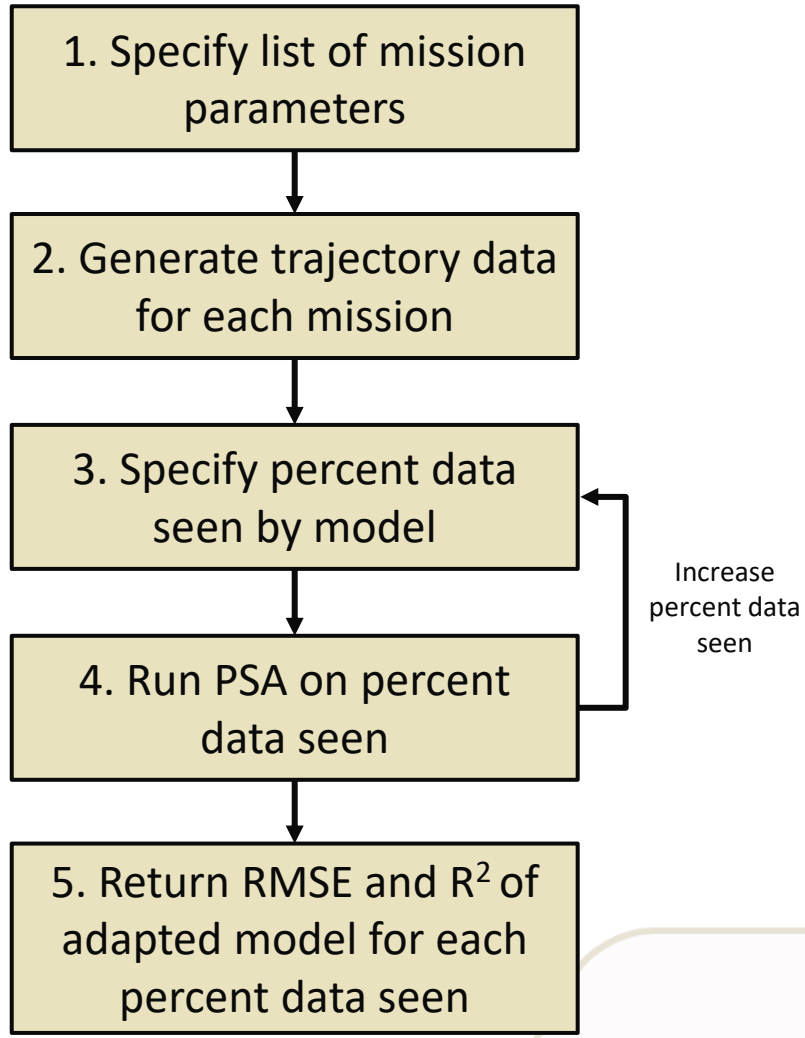
Experiment 2: Online Learning

How does PSA perform on sequential, streaming data?

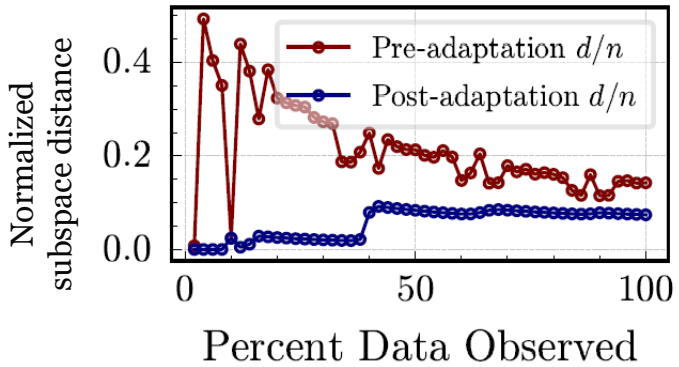
Hypothesis: PSA should always decrease the subspace distance between domains and incrementally decrease a model's prediction error.

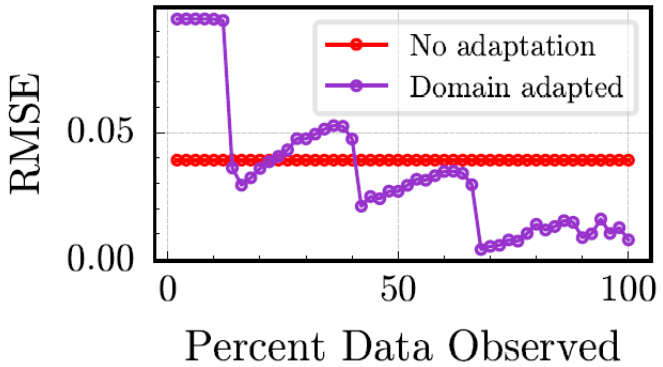
To test this:

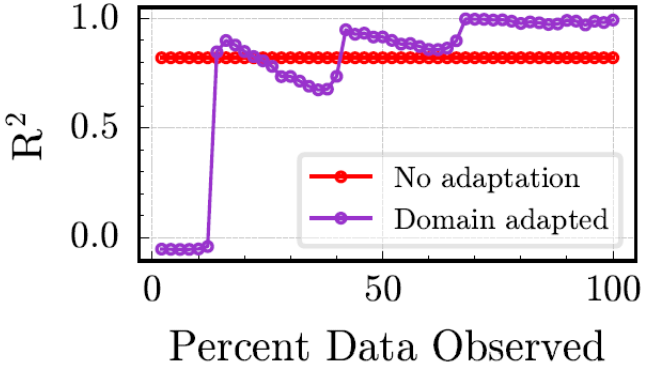
- Run PSA for different percent data seen from the operational domain to mimic streaming data
  - e.g., the model can only observe the first 5% of the operational inputs, then 10%, etc.
- Track subspace distance, RMSE, and R^2 as percent data seen increases
- **Note the same missions are used from last experiment**



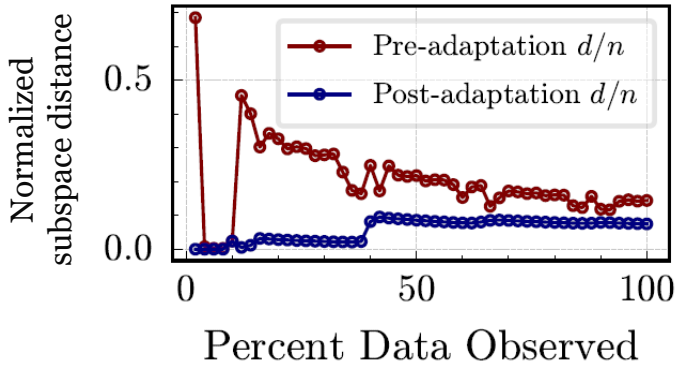
I. Periapsis shift

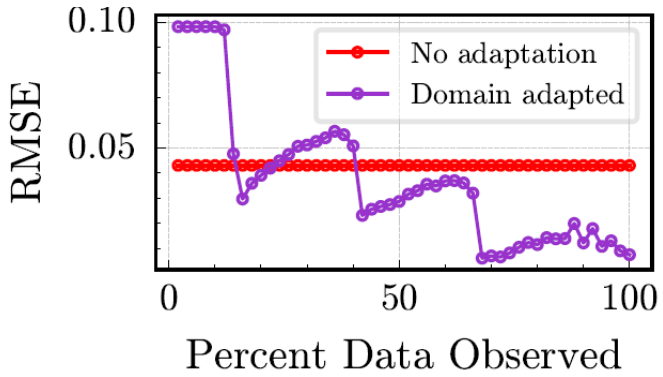


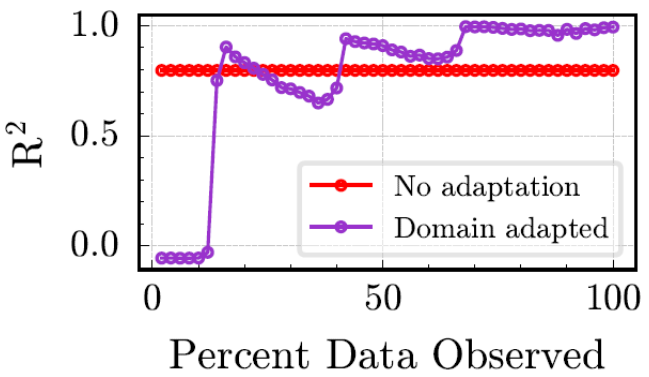




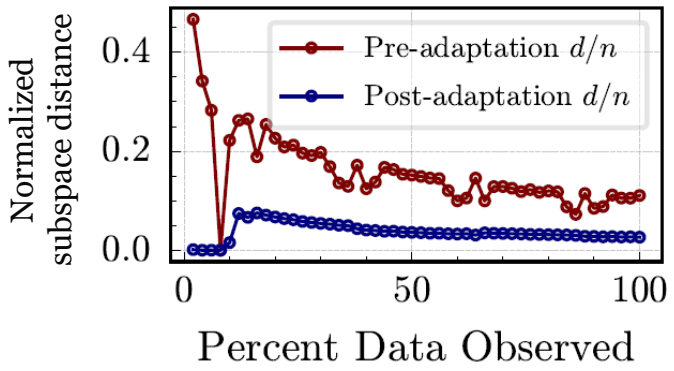
II. Eccentricity shift, fixed semi-major axis

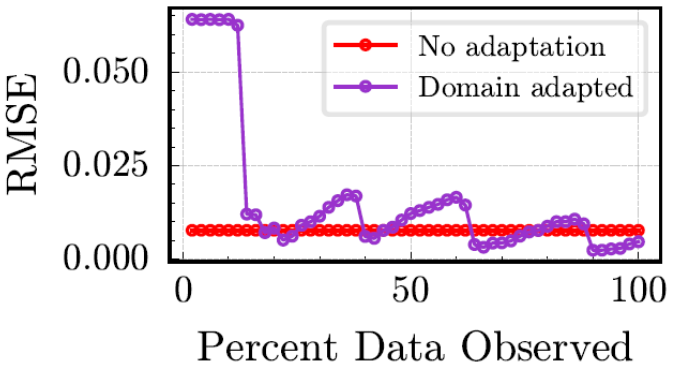


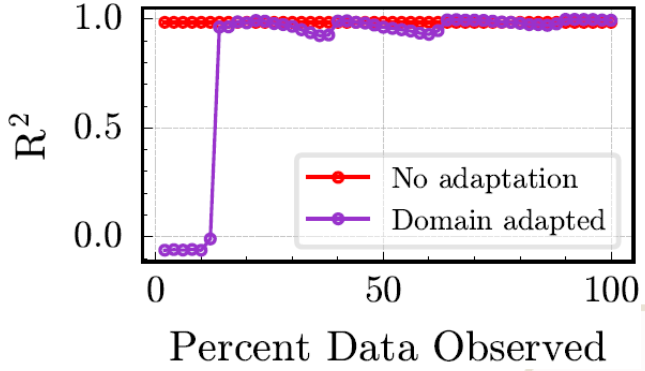




III. Eccentricity shift, fixed semi-latus rectum







- All results shown for Mission 1 of the corresponding shift type
- Online training results are identical in behavior for all other missions

How does PSA perform on sequential, streaming data?

Hypothesis: PSA should always decrease the subspace distance between domains and incrementally decrease a model's prediction error.

- From the plots, subspace distance always decreases from pre- to post-adaptation
- RMSE incrementally decreases with each subsequent drag passage

Hypothesis 2 validated

Research Objective

Develop an unsupervised method to allow a system digital twin to retain predictive performance on out-of-distribution data in evolving, unknown domains, such as changing environmental conditions, operating conditions, or configurations.

Overarching Hypothesis

If subspace tracking and manifold alignment are used to learn a common low-dimensional latent space of time-delay embedded data, then a data-driven digital twin trained in this latent space can retain predictive performance in evolving, unknown domains.

Research Question 1	Research Question 2	Research Question 3
How does the size of the domain shift affect the performance of PSA?	How does PSA perform on sequential, streaming data?	How does PSA perform under different noise levels?
PSA performs well for small shifts, and its performance degrades for larger shifts	PSA decreases the model error as more data is received	Experiment 3

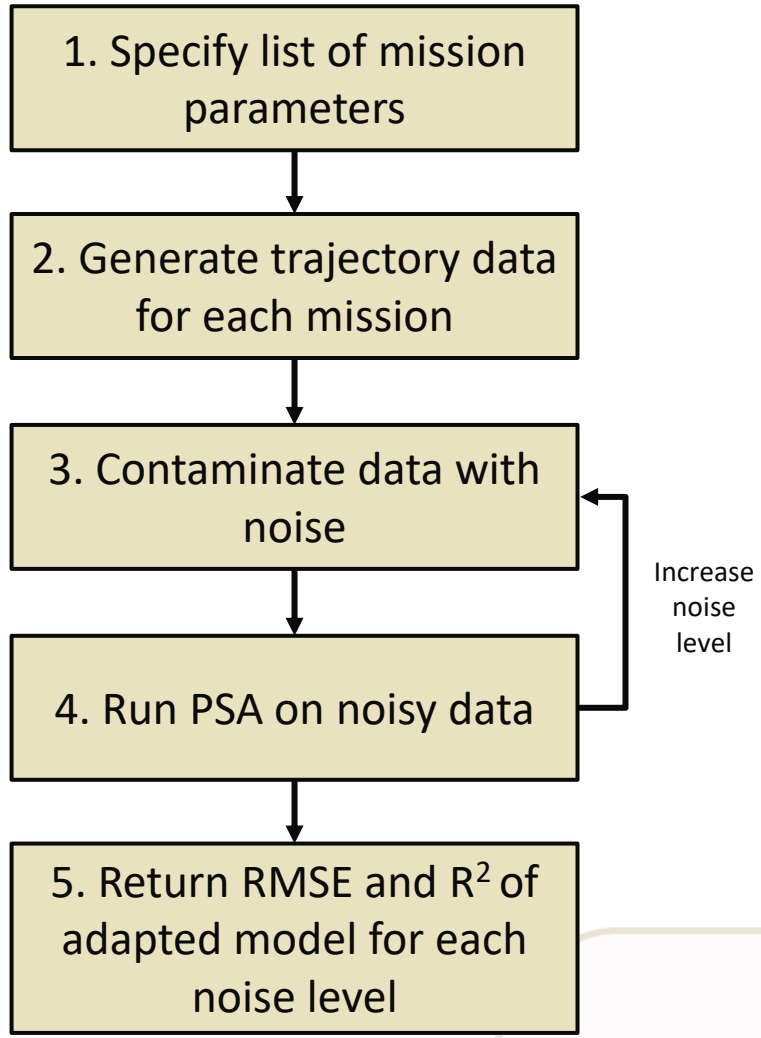
Experiment 3: Noisy Data

How does PSA perform under different noise levels?

Hypothesis: PSA should handle small amounts of noise in the data but its effectiveness will degrade for larger domain shifts and higher noise levels.

To test this:

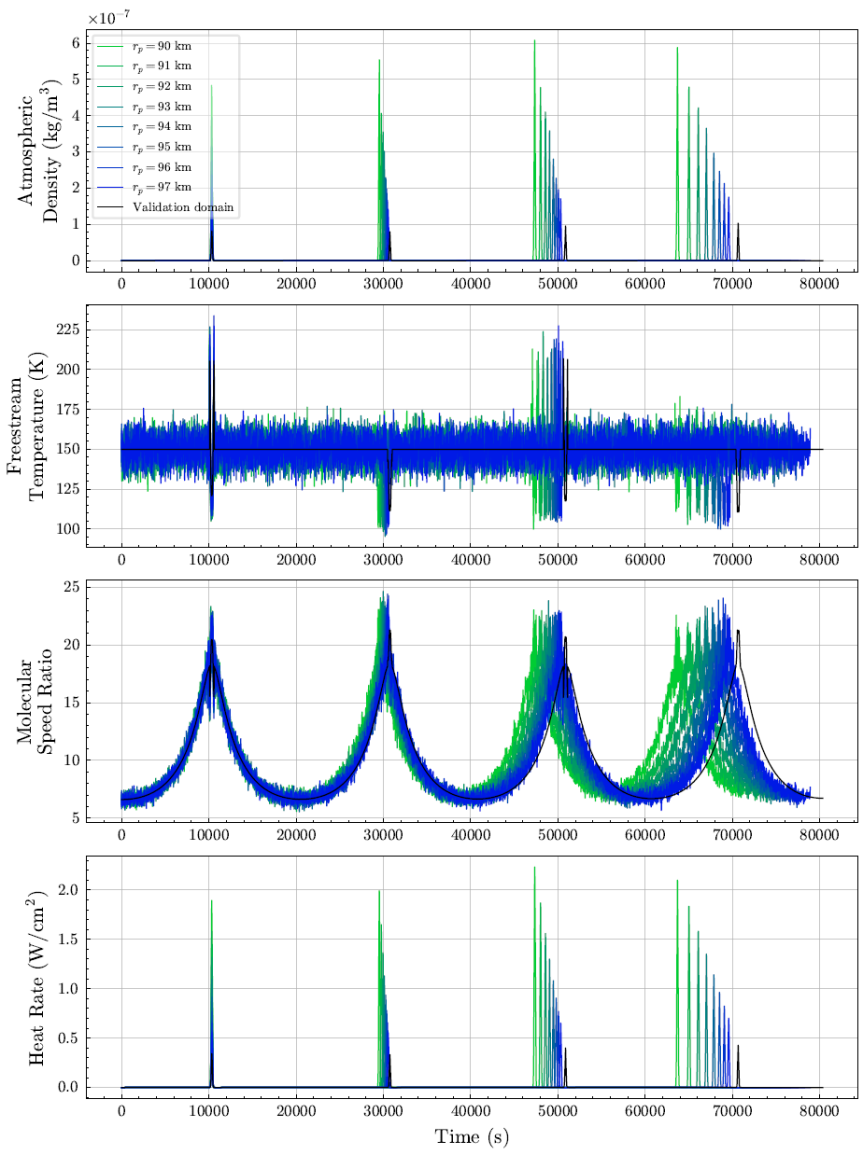
- Contaminate operational domain inputs with additive white Gaussian noise (AWGN)
  - DRM is noiseless, only operating mission has noise
- Run PSA on noisy data for all domain shifts
- Track subspace distance, RMSE, and R^2 as noise level increases for same missions from Experiment 1
- **Experiment tests for 1%, 5%, and 10% AWGN, but only 5% results shown (see Appendix H for other noise levels)**

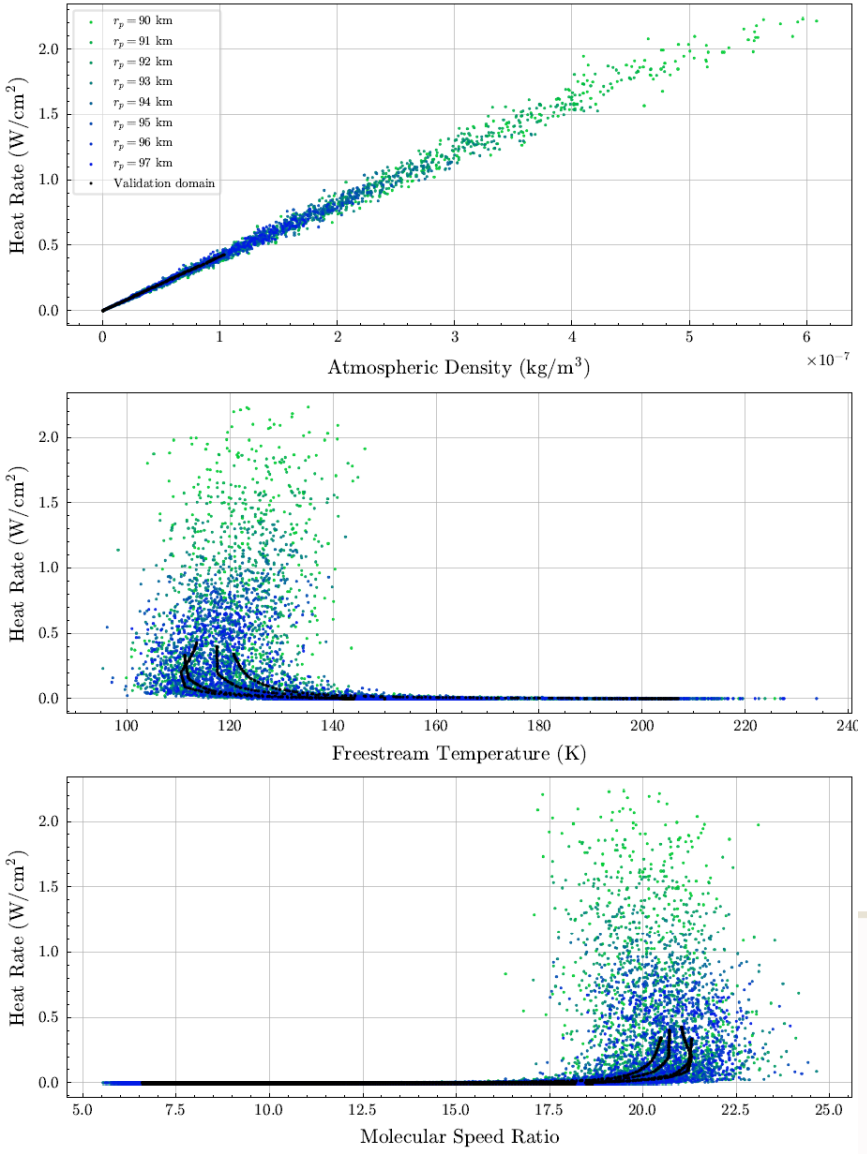


I. Periapsis Shift: Trajectory Data

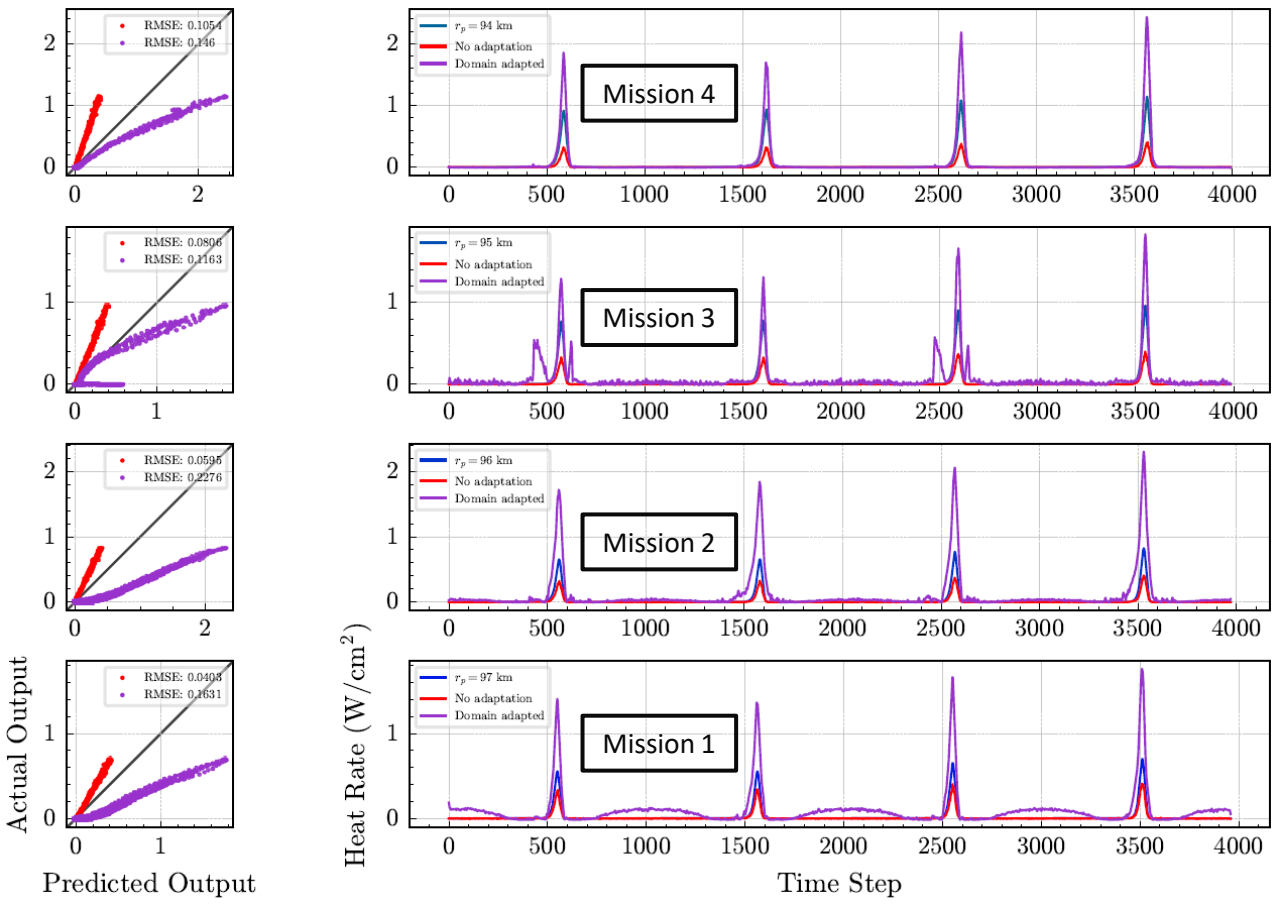
Mission Name	Number of Orbits	r_a (km)	r_p (km)	a (km)	p (km)	e
DRM	4	12000.0	100.00	6050.0	198.35	0.98347
Mission 1	4	12000.0	97.000	6048.5	192.44	0.98396
Mission 2	4	12000.0	96.000	6048.0	190.48	0.98413
Mission 3	4	12000.0	95.000	6047.5	188.51	0.98429
Mission 4	4	12000.0	94.000	6047.0	186.54	0.98446
Mission 5	4	12000.0	93.000	6046.5	184.57	0.98462
Mission 6	4	12000.0	92.000	6046.0	182.60	0.98478
Mission 7	4	12000.0	91.000	6045.5	180.63	0.98495
Mission 8	4	12000.0	90.000	6045.0	178.66	0.98511

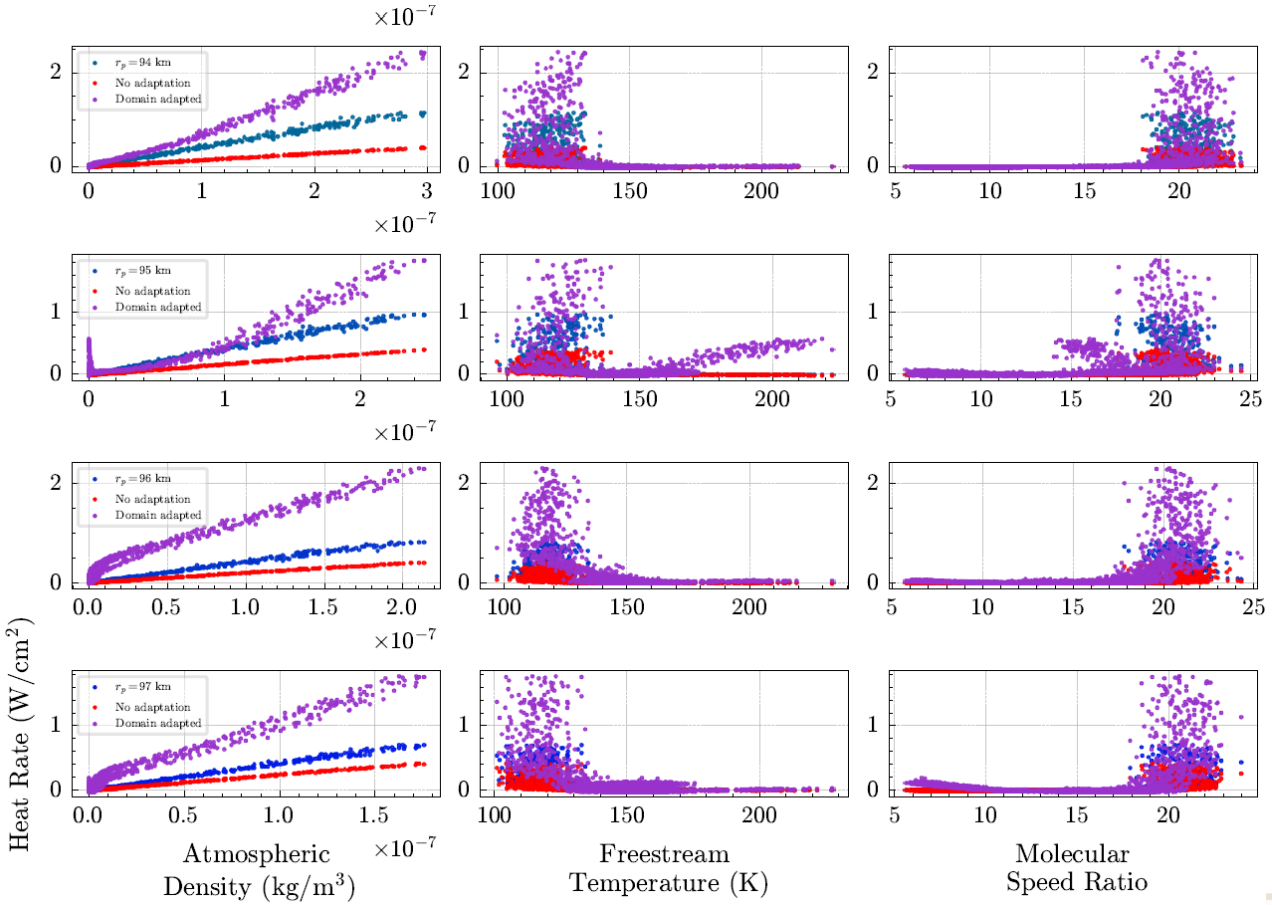
- Same data from Experiment 1
  - Periapsis is lowered from 100 km to 90 km in Missions 1-8
  - **Density, temperature, and speed ratio/velocity are contaminated with 5% AWGN**





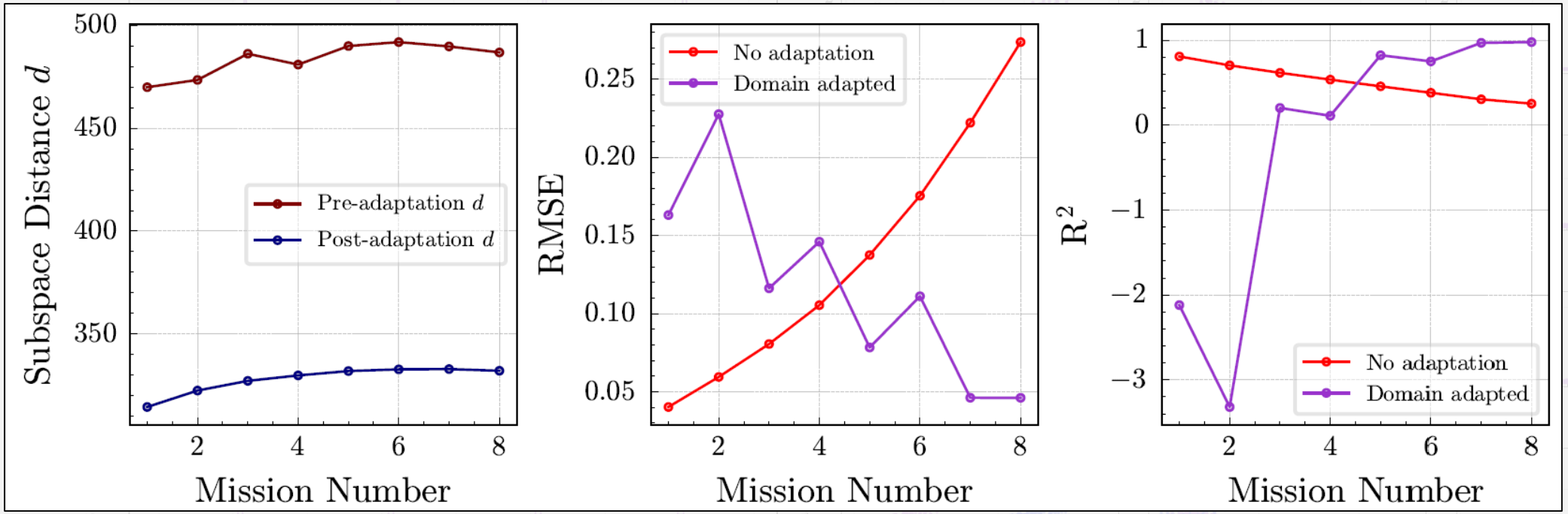
I. Periapsis Shift: Results





- Adapted model performs poorly on all missions
- Adapted model overpredicts heat rate

I. Periapsis Shift: Results

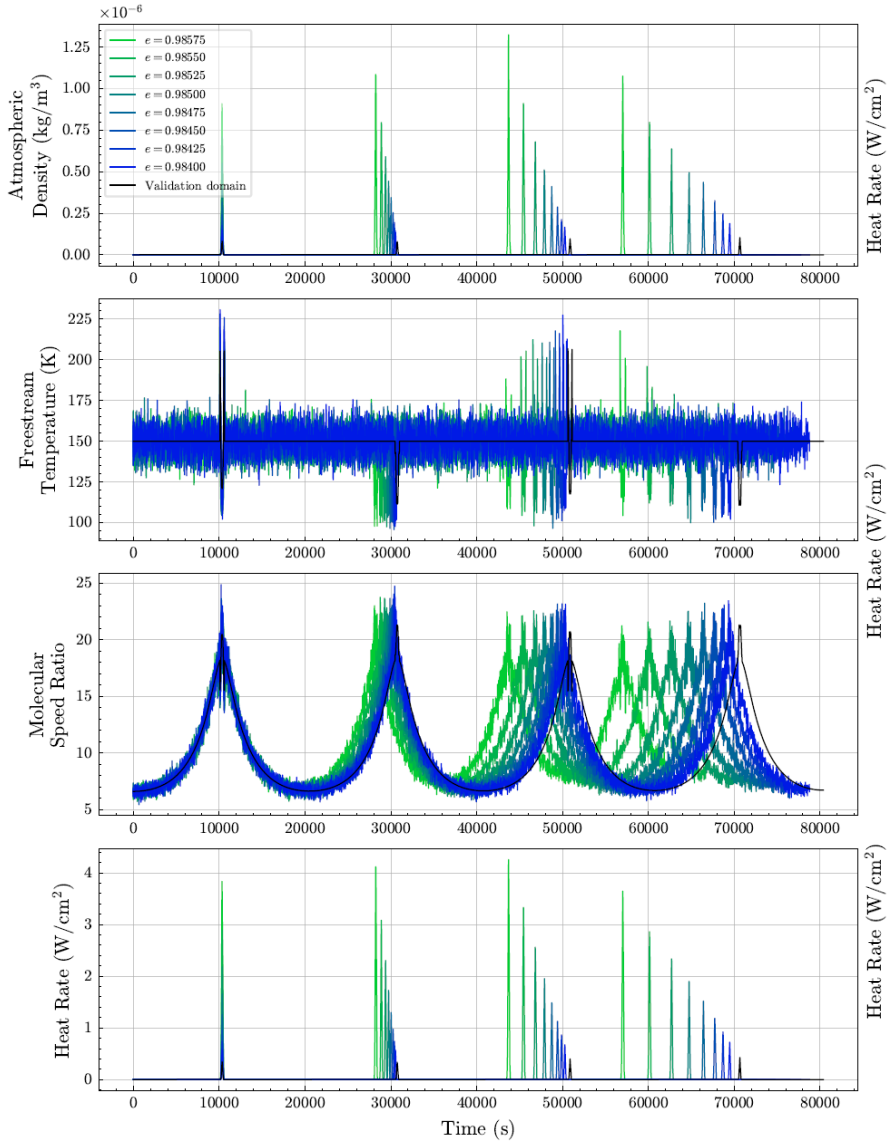


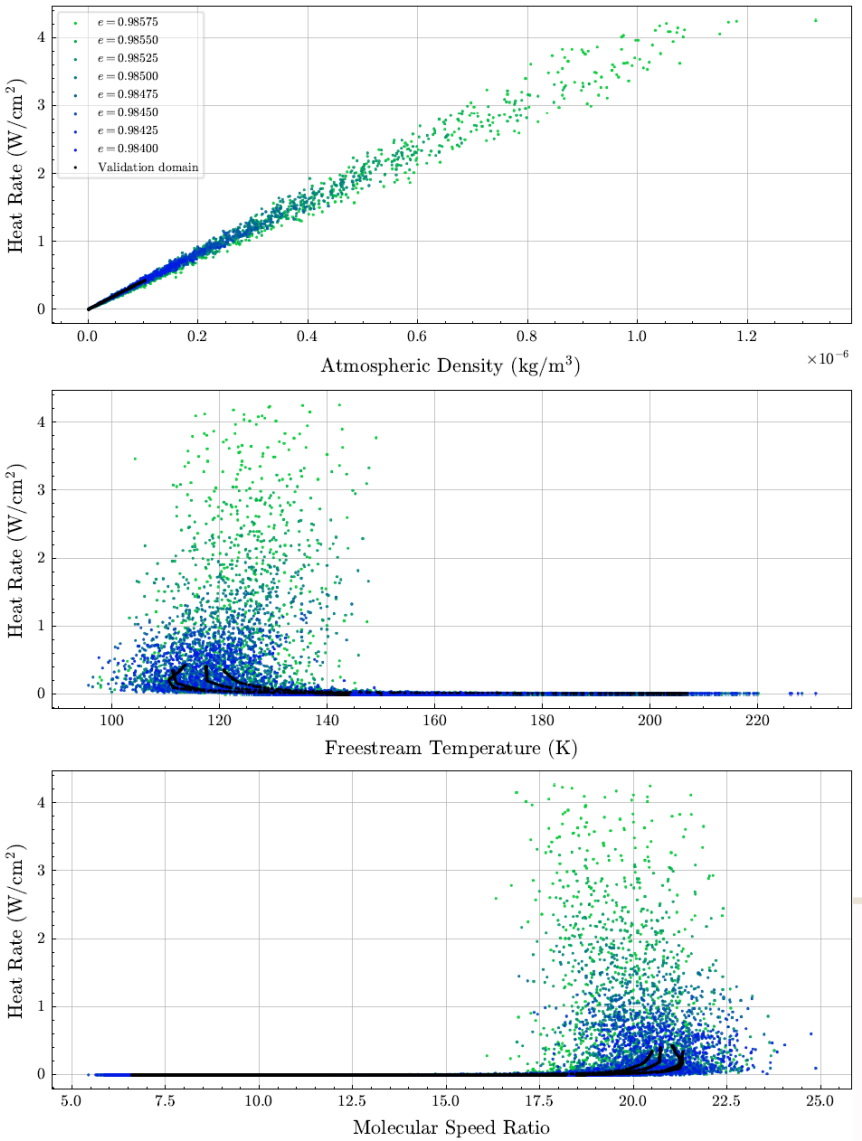
- Model appears performs better at higher domain shifts
- This contradicts the behavior on noiseless data, where the model worsens as the domain shift increases

II. Eccentricity Shift, Fixed Semi-Major Axis: Trajectory Data

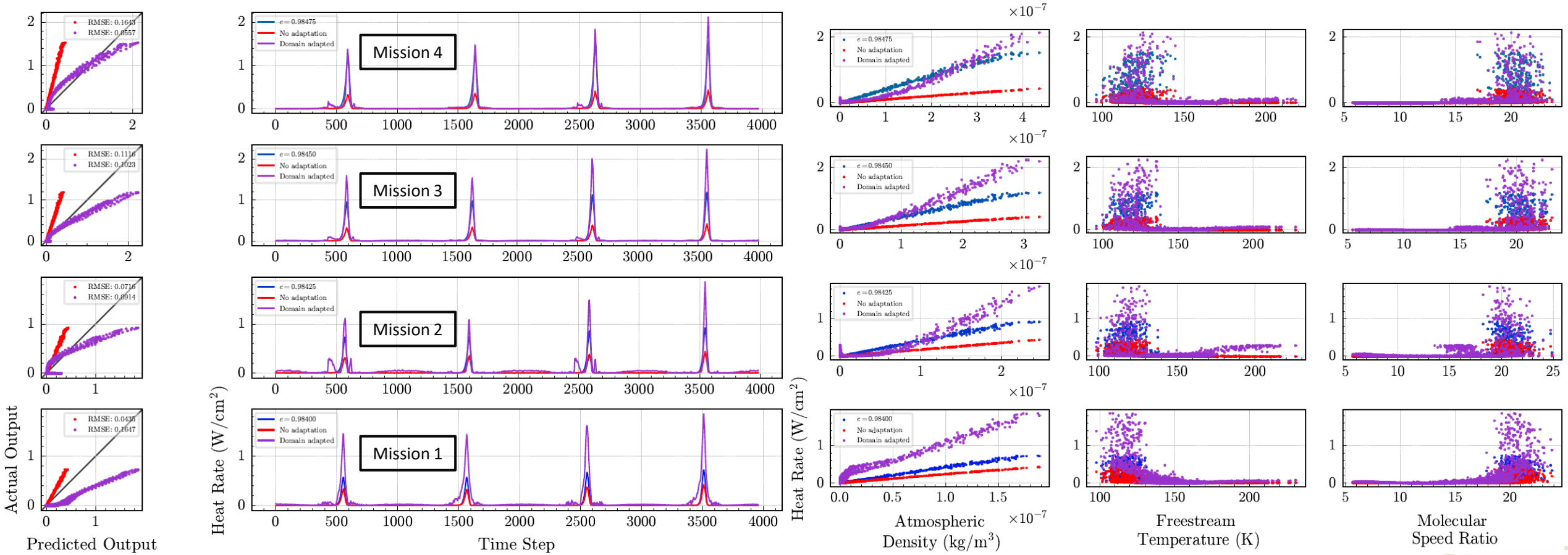
Mission Name	Number of Orbits	r_a (km)	r_p (km)	a (km)	p (km)	e
DRM	4	12000.0	100.00	6050.0	198.35	0.98347
Mission 1	4	12003.2	96.800	6050.0	192.05	0.98400
Mission 2	4	12004.7	95.288	6050.0	189.07	0.98425
Mission 3	4	12006.2	93.775	6050.0	186.10	0.98450
Mission 4	4	12007.7	92.262	6050.0	183.12	0.98475
Mission 5	4	12009.3	90.750	6050.0	180.14	0.98500
Mission 6	4	12010.8	89.238	6050.0	177.16	0.98525
Mission 7	4	12012.3	87.725	6050.0	174.18	0.98550
Mission 8	4	12013.8	86.212	6050.0	171.20	0.98575

- Same data from Experiment 1
  - Eccentricity is raised from 0.98347 km to 0.98575 km in Missions 1-8, fixed semi-major axis
  - **Density, temperature, and speed ratio/velocity are contaminated with 5% AWGN**



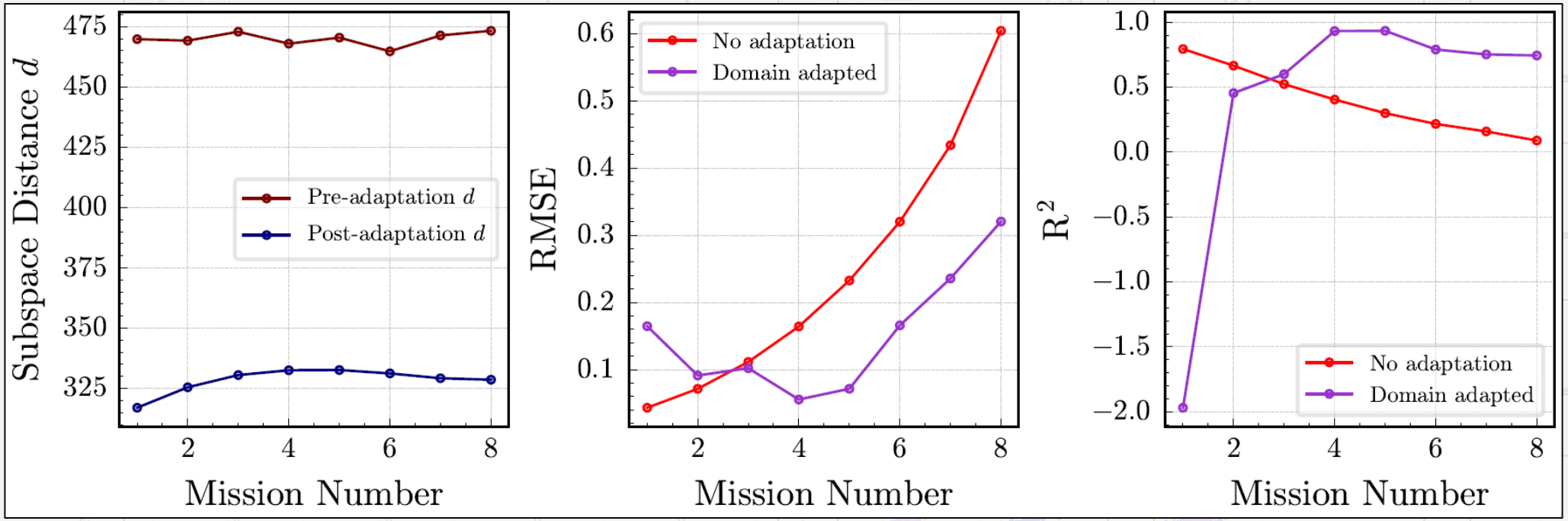


II. Eccentricity Shift, Fixed Semi-Major Axis: Results



- Same results as periapsis shift; adapted model performs poorly on all missions
- Adapted model overpredicts heat rate

II. Eccentricity Shift, Fixed Semi-Major Axis: Results

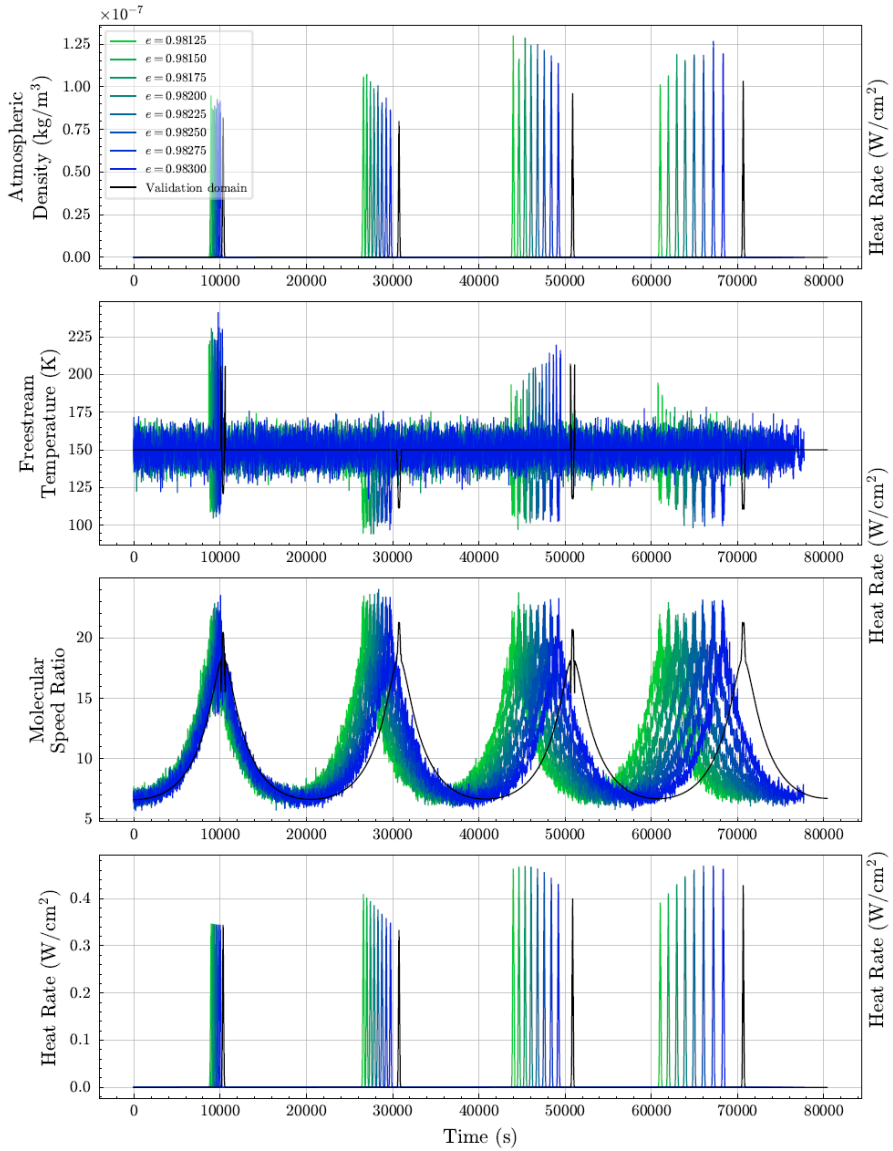


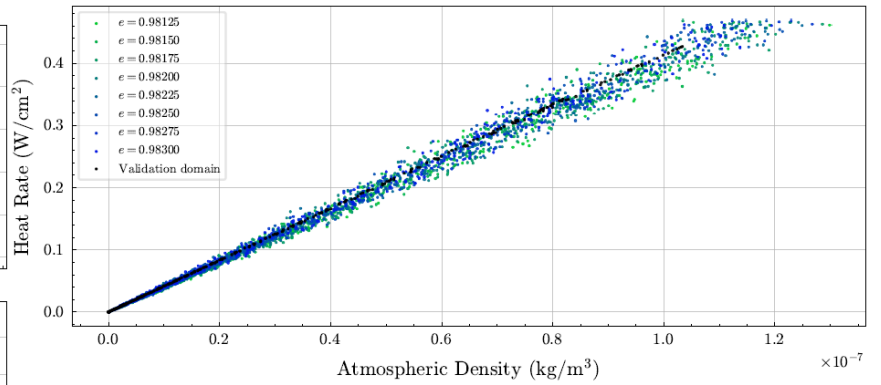
- As before, model appears to perform better at higher domain shifts
- Best performance on Mission 4

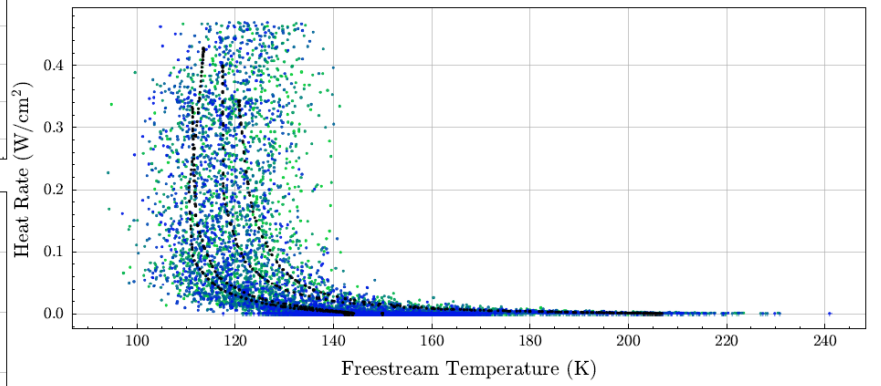
III. Eccentricity Shift, Fixed Semi-Latus Rectum: Trajectory Data

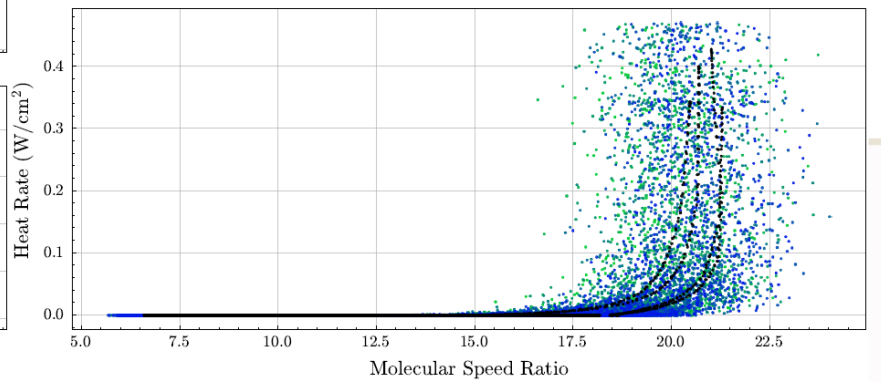
Mission Name	Number of Orbits	r_a (km)	r_p (km)	a (km)	p (km)	e
DRM	4	12000.0	100.00	6050.0	198.35	0.98347
Mission 1	4	11667.5	100.02	5883.8	198.35	0.98300
Mission 2	4	11498.4	100.04	5799.2	198.35	0.98275
Mission 3	4	11334.1	100.05	5717.1	198.35	0.98250
Mission 4	4	11174.5	100.06	5637.3	198.35	0.98225
Mission 5	4	11019.3	100.07	5559.7	198.35	0.98200
Mission 6	4	10868.3	100.09	5484.2	198.35	0.98175
Mission 7	4	10721.5	100.10	5410.8	198.35	0.98150
Mission 8	4	10578.5	100.11	5339.3	198.35	0.98125

- Same data from Experiment 1
  - Eccentricity is lowered from 0.98347 km to 0.98125 km in Missions 1-8, fixed semi-latus rectum
  - **Density, temperature, and speed ratio/velocity are contaminated with 5% AWGN**

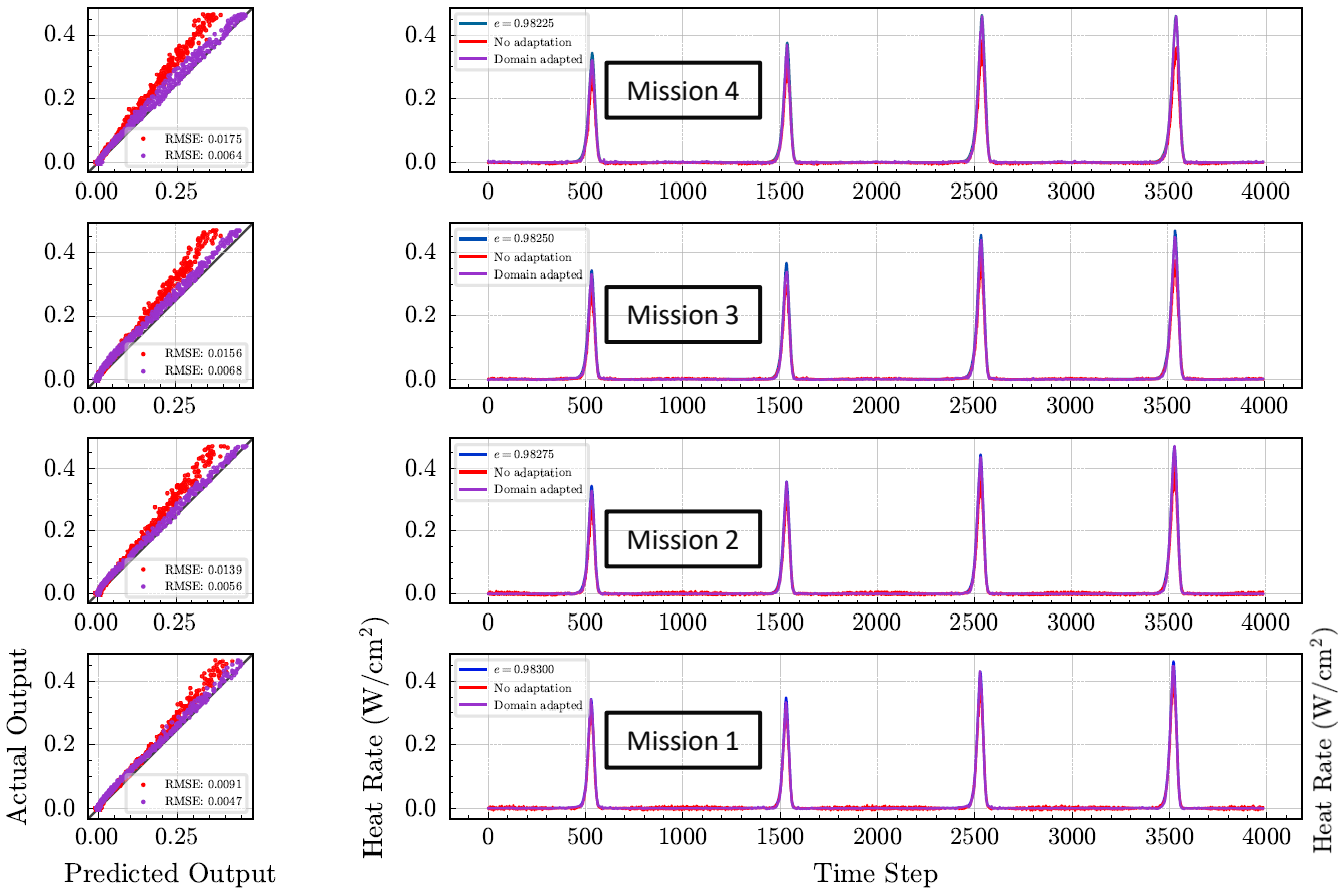


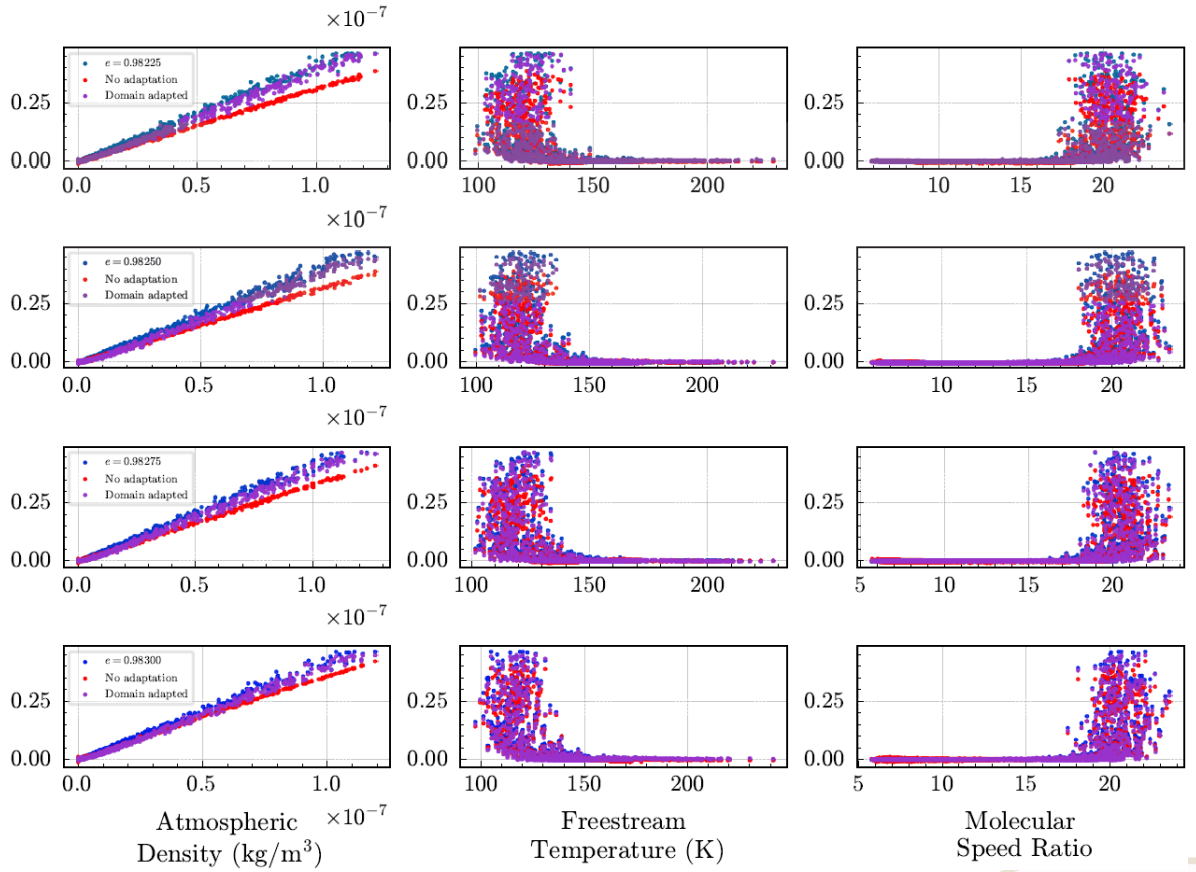






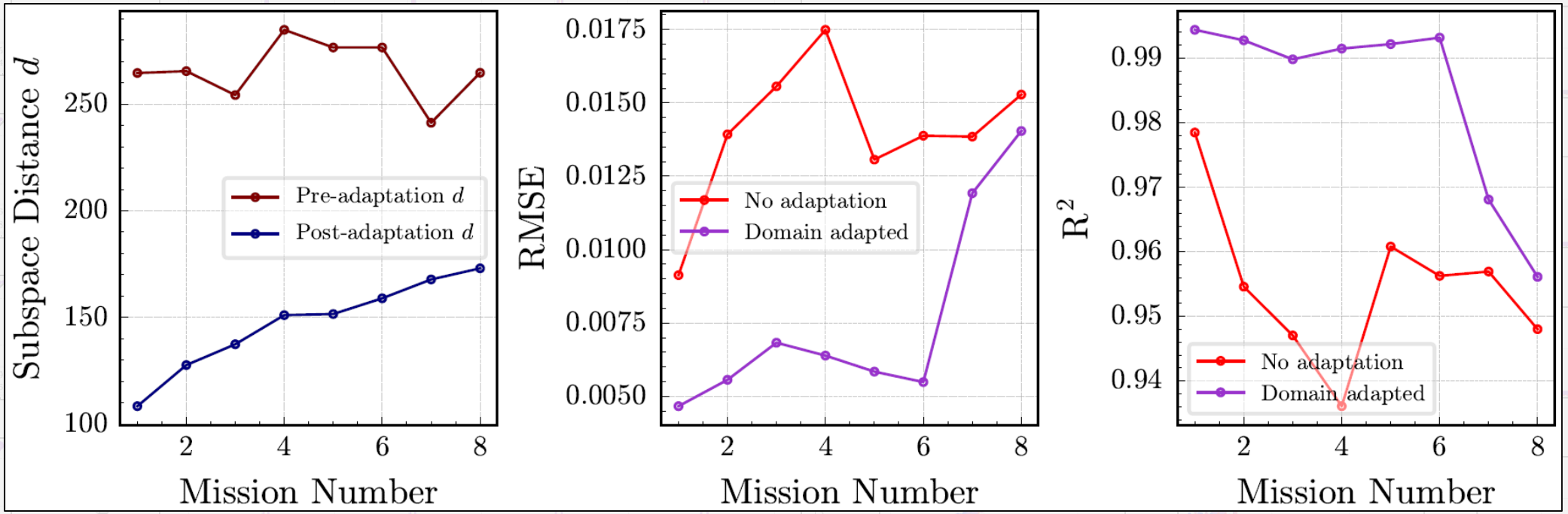
III. Eccentricity Shift, Fixed Semi-Latus Rectum: Results





- **The model performs well**
  - Best performance on Mission 1
  - Domain adapted model performs better than baseline across all missions

III. Eccentricity Shift, Fixed Semi-Latus Rectum: Results



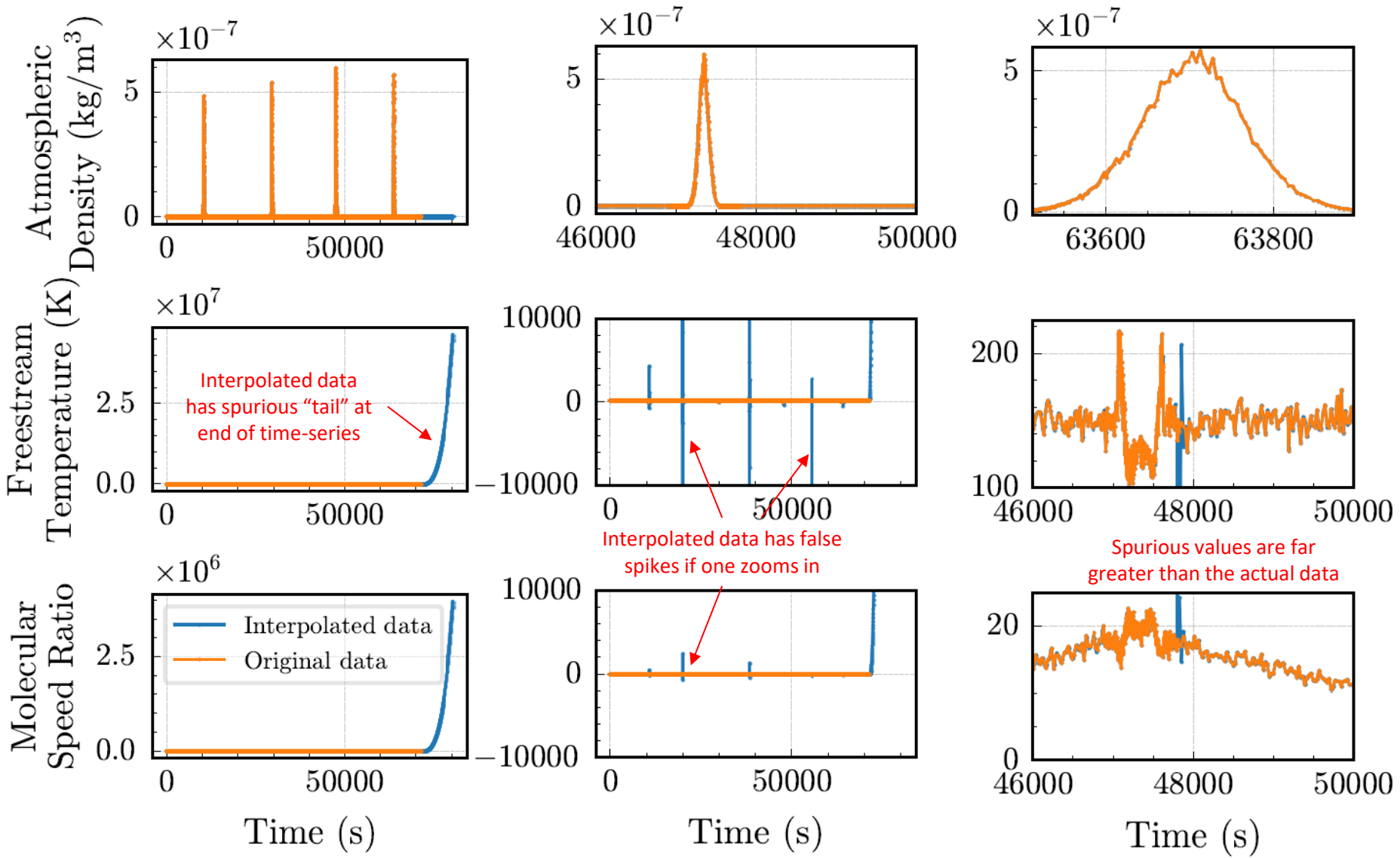
- **The model performs well**
  - Best performance on Mission 1
  - Domain adapted model performs better than baseline across all missions

Spurious Interpolation on Noisy Data

Why did the model perform so poorly on the first two shift types?

- Recall that the data is interpolated in time for the first two shift types
- Interpolation badly fits the data and spurious data is introduced
  - Manifold alignment fails as a result
- **Model superficially appears to be performing better at higher shifts for types I and II, but manifolds are actually scaled incorrectly**

Interpolated inputs for Mission 1



Experiment 3: Summary

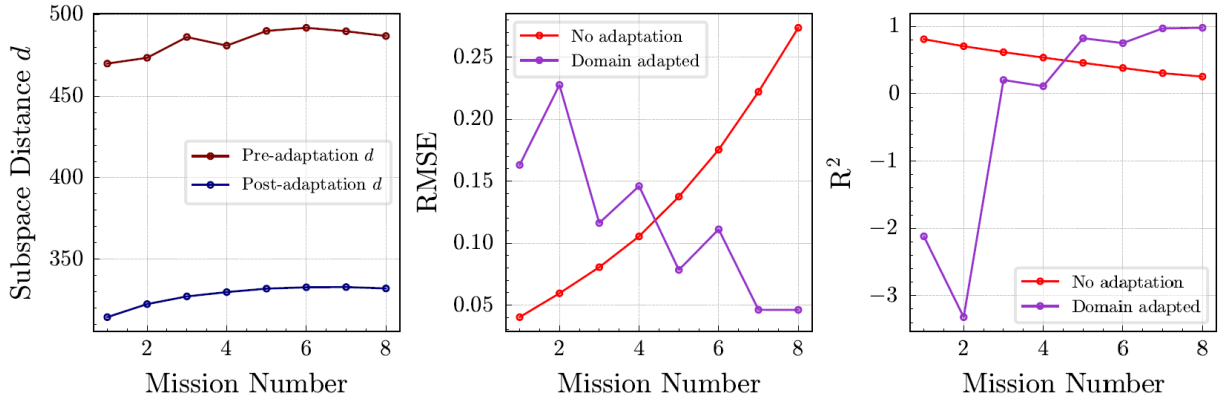
How does PSA perform under different noise levels?

Hypothesis: PSA should handle small amounts of noise in the data but its effectiveness will degrade for larger domain shifts and higher noise levels.

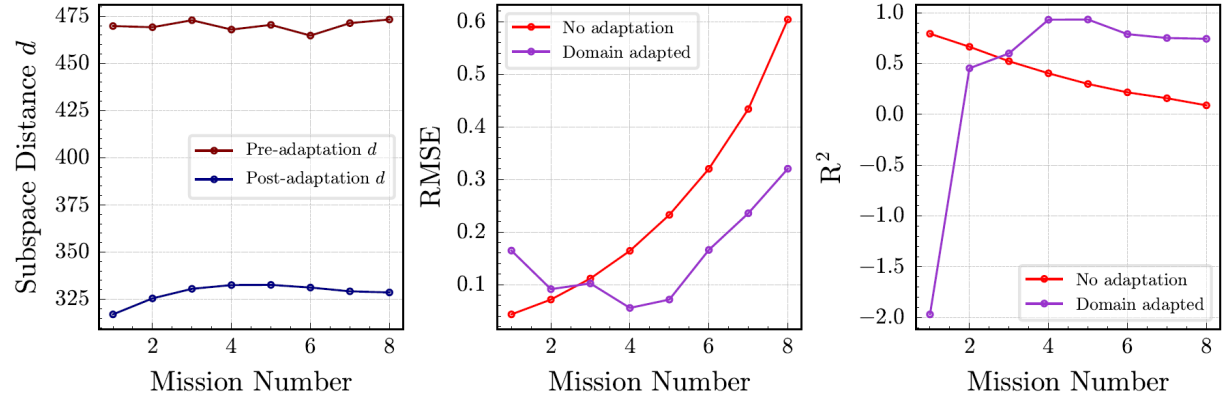
- Interpolating the data causes manifold alignment to fail (see shift types I and II)
- Removing data for correspondence yields accurate manifold alignment
- **When testing for 1% and 10% noise, results demonstrate effectiveness degrades when noise level increases**
- **Noise does not affect data differently across different sizes of domain shift**

Hypothesis 3 partially validated (if correspondence is correct)

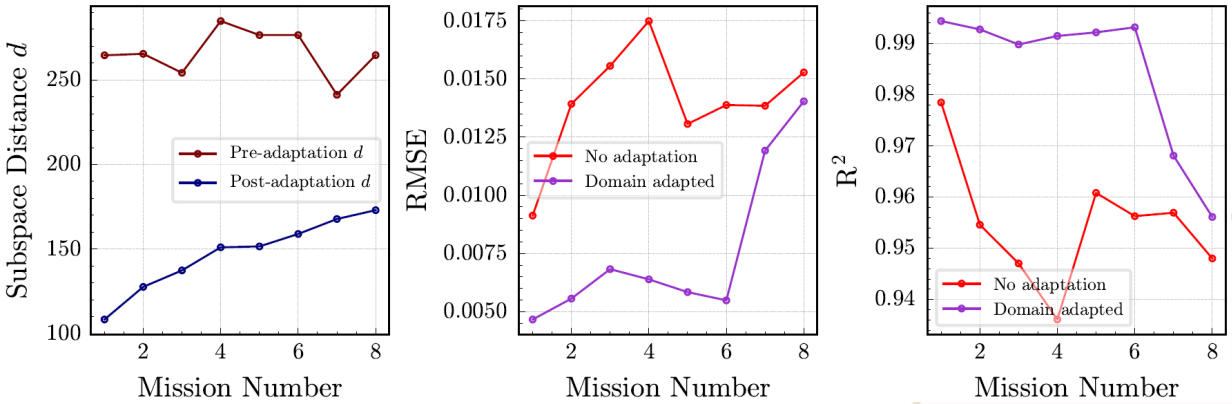
I. Periapsis shift w/ 5% AWGN



II. Eccentricity shift, fixed semi-major axis w/ 5% AWGN



III. Eccentricity shift, fixed semi-latus rectum w/ 5% AWGN



Research Objective

Develop an unsupervised method to allow a system digital twin to retain predictive performance on out-of-distribution data in evolving, unknown domains, such as changing environmental conditions, operating conditions, or configurations.

Overarching Hypothesis

If subspace tracking and manifold alignment are used to learn a common low-dimensional latent space of time-delay embedded data, then a data-driven digital twin trained in this latent space can retain predictive performance in evolving, unknown domains.

Research Question 1

How does the size of the domain shift affect the performance of PSA?

PSA performs well for small shifts, and its performance degrades for larger shifts

Research Question 2

How does PSA perform on sequential, streaming data?

PSA decreases the model error as more data is received

Research Question 3

How does PSA perform under different noise levels?

PSA can handle small amounts of noise in the data, but its performance degrades for higher noise levels

Conclusion

Concluding Remarks

Summary

- Data-driven and hybrid digital twin models are only accurate within a sampled set of data and degrade in accuracy outside this data
- Developed unsupervised method of adapting models to extrapolatory data by exploiting low-dimensional time-dependent features between missions

Contributions

- Many existing calibration/adaptation methods are supervised
- New unsupervised adaptation method for regression models; vast majority of existing methods are intended for classifiers in ML and digital twin literature
- Adaptation can be performed online, unlike many existing works that assume the complete domain is available for adaptation

Usages/Applications

- PSA is beneficial in scenarios with unobservable model outputs
- PSA is beneficial in scenarios with limited data (e.g., only one trajectory is available for training)

Limitations and Future Work

Limitations

- Method is only valid if the two domains have similar temporal features
- Method depends on accurate pairwise correspondence to be effective

Future Work

- Research new correspondence methods and/or methods that do not require correspondence
- Combine adaptation with new model architectures (e.g., domain adversarial neural networks)
- Test new use cases

Publications and Conferences

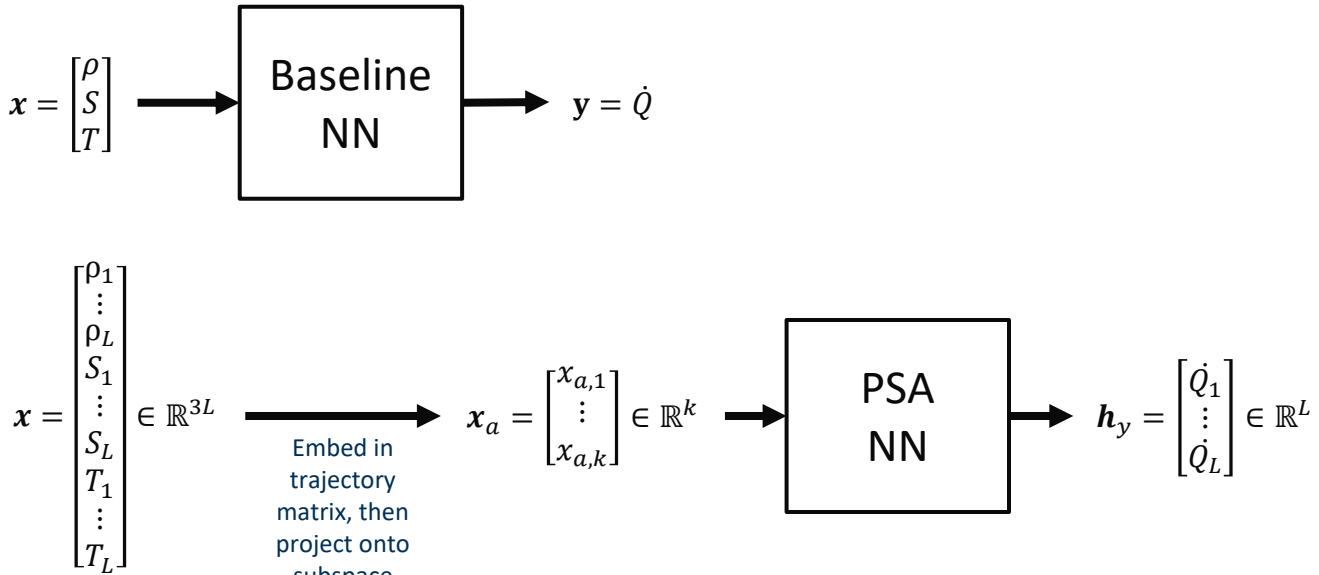
- "*Adaptive Digital Twins: Continuous Subspace Learning for Dynamic Domains*," AIAA SciTech Forum 2026
- "*Unsupervised Adaptive Data-Driven Thermal Models for Aerobraking Campaigns*," 23rd International Planetary Probe Workshop
- "*Unsupervised Online Subspace Adaptation for Dynamic Data-Driven Digital Twins*," AIAA Journal (in preparation)
- "*Domain Adaptive Mission Design by Optimal Transport*," Acta Astronautica (in preparation)

Thank you

Questions?

Appendix A.1: Model Architecture and Learning Algorithm

- PSA changes the input dimension such that the model predicts on a window of L points
- Both the baseline model and the PSA model have 4 layers, 200 neurons per hidden layer, a learning rate of 0.006, and were trained for 200 epochs for all results
- These hyperparameters were determined by Bayesian optimization to minimize the baseline model's loss on the validation domain



Algorithm 1 Procrustes Subspace Adaptation (PSA)

Input: $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]^T \in \mathbb{R}^{n \times D}$, $\boldsymbol{Y}_S = [y_S^{(1)}, \ldots, y_S^{(n)}] \in \mathbb{R}^n$, timestamps $\boldsymbol{t}_S \in \mathbb{R}^n$, subspace dimension k, window length L, learning rate η

Output: Model $\tilde{f} : \boldsymbol{x}_a \mapsto \tilde{\boldsymbol{h}}_Y$, embedding space $\boldsymbol{H}_{Z_t,\text{sub}}$

Find $\boldsymbol{H}_X = \mathcal{H}(\boldsymbol{X}) \in \mathbb{R}^{DL \times (n-L+1)}$ and $\boldsymbol{H}_{Y_S} = \mathcal{H}(\boldsymbol{Y}_S) \in \mathbb{R}^{DL \times (n-L+1)}$
$\boldsymbol{H}_{X,\text{sub}} \in \mathbb{R}^{DL \times k} \leftarrow \text{pca}(\boldsymbol{H}_X)$
Set initial guess for $\boldsymbol{H}_{Z_t,\text{sub}}(t=0) = \boldsymbol{H}_{X,\text{sub}}$
repeat
  Receive observations of $\boldsymbol{z}$ and append to $\boldsymbol{Z}_t$
  Record corresponding timestamps of $\boldsymbol{Z}_t$ as $\boldsymbol{t}_T$
  Truncate $\boldsymbol{X}$ to $\boldsymbol{X}_t$, containing $\boldsymbol{x}$ seen up to time t
  $\boldsymbol{X}_t = (\boldsymbol{X}_t - \bar{\boldsymbol{X}}_t)/\sigma_{X_t}$ ▷ Mean $\bar{\boldsymbol{X}}_t$, standard deviation σ_{X_t}
  $\boldsymbol{Z}_t = (\boldsymbol{Z}_t - \bar{\boldsymbol{Z}}_t)/\sigma_{Z_t}$ ▷ Mean $\bar{\boldsymbol{Z}}_t$, standard deviation σ_{Z_t}
  $\boldsymbol{H}_{X_t} = \mathcal{H}(\boldsymbol{X}_t)$
  $\boldsymbol{H}_{Z_t} = \mathcal{H}(\boldsymbol{Z}_t)$
  Define common timesteps for interpolation $\mathbf{t} = \text{sort}\left([\boldsymbol{t}_S | \boldsymbol{t}_T]^T\right)$
  $\tilde{\boldsymbol{X}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{X}_t)$
  $\tilde{\boldsymbol{Z}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{Z}_t)$
  $\tilde{\boldsymbol{H}}_{X_t} = \mathcal{H}(\tilde{\boldsymbol{X}}_t)$
  $\tilde{\boldsymbol{H}}_{Z_t} = \mathcal{H}(\tilde{\boldsymbol{Z}}_t)$
  $\boldsymbol{H}_{X_t,\text{sub}} \leftarrow \text{pca}(\boldsymbol{H}_{X_t})$
  $\boldsymbol{H}_{Z_t,\text{sub}}(t) = \boldsymbol{H}_{Z_t,\text{sub}}(t-1) + \eta \tilde{\boldsymbol{h}}_{Z_t}(t) \left(\tilde{\boldsymbol{h}}_{Z_t}(t)\right)^T \boldsymbol{H}_{Z_t,\text{sub}}(t-1)$
  $\tilde{\boldsymbol{H}}_{X_t,\text{proj}} = \boldsymbol{H}_{X_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{X_t}$
  $\tilde{\boldsymbol{H}}_{Z_t,\text{proj}} = \boldsymbol{H}_{Z_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{Z_t}$
  $\boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T = \tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{Z_t,\text{proj}}^T$
  $\boldsymbol{Q} = \boldsymbol{V}\boldsymbol{U}^T$
  $s = \dfrac{\text{tr}(\boldsymbol{S})}{\text{tr}(\tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{X_t,\text{proj}}^T)}$
  $\boldsymbol{X}_a = s\boldsymbol{Q}(\boldsymbol{H}_{X_t,\text{sub}}^T \boldsymbol{H}_{X_t})$
  $\boldsymbol{Z}_a = \boldsymbol{H}_{Z_t,\text{sub}}^T \boldsymbol{H}_{Z_t}$
  $\tilde{f} = \arg\min_{\tilde{\theta}} \sum_{i=1}^{n} \mathcal{L}\left(\tilde{f}(\boldsymbol{x}_a^{(i)} | \tilde{\boldsymbol{\theta}}), \boldsymbol{h}_{Y_S}^{(i)}\right)$
  $t = t + 1$
until termination or all $\boldsymbol{z}$ seen
return $\tilde{f}$, $\boldsymbol{H}_{Z_t,\text{sub}}$

Appendix A.2: Details on Training

Algorithm has two phases:

- Adaptation
  1) Stack inputs into trajectory matrices (using operator $\mathcal{H}$)
  2) Find linked data for correspondence by interpolating the data in time (interpolated data has tilde hat)
  3) Stack interpolated data into trajectory matrices
  4) Find subspaces of all trajectory matrices
  5) Find manifolds of interpolated trajectory matrices by projecting them onto their subspaces
  6) Perform Procrustes analysis to solve for rotation $\boldsymbol{Q}$ and scaling s terms
  7) Align non-interpolated validation manifold using $\boldsymbol{Q}$ and s. Find non-interpolated operational manifold.
- Regression (training)
  - Stack outputs of the validation domain into their own trajectory matrix (done outside the loop)
  8) Perform empirical risk minimization using the columns of the validation manifold and the columns of the output trajectory matrix

Algorithm 1 Procrustes Subspace Adaptation (PSA)

Input: $\boldsymbol{X} = [\boldsymbol{x}_1, \ldots, \boldsymbol{x}_n]^T \in \mathbb{R}^{n \times D}$, $\boldsymbol{Y}_S = [y_S^{(1)}, \ldots, y_S^{(n)}] \in \mathbb{R}^n$, timestamps $\boldsymbol{t}_S \in \mathbb{R}^n$, subspace dimension k, window length L, learning rate η

Output: Model $\tilde{f} : \boldsymbol{x}_a \mapsto \tilde{\boldsymbol{h}}_Y$, embedding space $\boldsymbol{H}_{Z_t,\text{sub}}$

Find $\boldsymbol{H}_X = \mathcal{H}(\boldsymbol{X}) \in \mathbb{R}^{DL \times (n-L+1)}$ and $\boldsymbol{H}_{Y_S} = \mathcal{H}(\boldsymbol{Y}_S) \in \mathbb{R}^{DL \times (n-L+1)}$

$\boldsymbol{H}_{X,\text{sub}} \in \mathbb{R}^{DL \times k} \leftarrow \text{pca}(\boldsymbol{H}_X)$

Set initial guess for $\boldsymbol{H}_{Z_t,\text{sub}}(t=0) = \boldsymbol{H}_{X,\text{sub}}$

repeat

  Receive observations of $\boldsymbol{z}$ and append to $\boldsymbol{Z}_t$

  Record corresponding timestamps of $\boldsymbol{Z}_t$ as $\boldsymbol{t}_T$

  Truncate $\boldsymbol{X}$ to $\boldsymbol{X}_t$, containing $\boldsymbol{x}$ seen up to time t

  $\boldsymbol{X}_t = (\boldsymbol{X}_t - \bar{\boldsymbol{X}}_t)/\sigma_{X_t}$ ▷ Mean $\bar{\boldsymbol{X}}_t$, standard deviation σ_{X_t}

  $\boldsymbol{Z}_t = (\boldsymbol{Z}_t - \bar{\boldsymbol{Z}}_t)/\sigma_{Z_t}$ ▷ Mean $\bar{\boldsymbol{Z}}_t$, standard deviation σ_{Z_t}

  $\boldsymbol{H}_{X_t} = \mathcal{H}(\boldsymbol{X}_t)$

  $\boldsymbol{H}_{Z_t} = \mathcal{H}(\boldsymbol{Z}_t)$

  Define common timesteps for interpolation $\mathbf{t} = \text{sort}\left([\boldsymbol{t}_S | \boldsymbol{t}_T]^T\right)$

  $\tilde{\boldsymbol{X}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{X}_t)$

  $\tilde{\boldsymbol{Z}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{Z}_t)$

  $\tilde{\boldsymbol{H}}_{X_t} = \mathcal{H}(\tilde{\boldsymbol{X}}_t)$

  $\tilde{\boldsymbol{H}}_{Z_t} = \mathcal{H}(\tilde{\boldsymbol{Z}}_t)$

  $\boldsymbol{H}_{X_t,\text{sub}} \leftarrow \text{pca}(\boldsymbol{H}_{X_t})$

  $\boldsymbol{H}_{Z_t,\text{sub}}(t) = \boldsymbol{H}_{Z_t,\text{sub}}(t-1) + \eta \tilde{\boldsymbol{h}}_{Z_t}(t) \left(\tilde{\boldsymbol{h}}_{Z_t}(t)\right)^T \boldsymbol{H}_{Z_t,\text{sub}}(t-1)$

  $\tilde{\boldsymbol{H}}_{X_t,\text{proj}} = \boldsymbol{H}_{X_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{X_t}$

  $\tilde{\boldsymbol{H}}_{Z_t,\text{proj}} = \boldsymbol{H}_{Z_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{Z_t}$

  $\boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T = \tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{Z_t,\text{proj}}^T$

  $\boldsymbol{Q} = \boldsymbol{V}\boldsymbol{U}^T$

  $s = \dfrac{\text{tr}(\boldsymbol{S})}{\text{tr}(\tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{X_t,\text{proj}}^T)}$

  $\boldsymbol{X}_a = s\boldsymbol{Q}(\boldsymbol{H}_{X_t,\text{sub}}^T \boldsymbol{H}_{X_t})$

  $\boldsymbol{Z}_a = \boldsymbol{H}_{Z_t,\text{sub}}^T \boldsymbol{H}_{Z_t}$

  $\tilde{f} = \arg\min_{\tilde{\theta}} \sum_{i=1}^{n} \mathcal{L}\left(\tilde{f}(\boldsymbol{x}_a^{(i)} | \tilde{\boldsymbol{\theta}}), \boldsymbol{h}_{Y_S}^{(i)}\right)$

  $t = t + 1$

until termination or all $\boldsymbol{z}$ seen

return $\tilde{f}$, $\boldsymbol{H}_{Z_t,\text{sub}}$

Appendix A.3: Details on Inference

The outputs of the algorithm are:

- Trained, domain adapted model $\tilde{f}$
- Subspace of the operational domain inputs $H_{Z_t,\text{sub}}$

To predict using the model:

1) Take the input time series Z, stack it into a trajectory matrix H_Z
2) Project the trajectory matrix onto the subspace
$$H_{Z,\text{proj}} = H_{Z_t,\text{sub}}^T H_Z$$
3) Predict on the columns of $H_{Z,\text{proj}} = [h_1, \dots, h_m]$.
$$\tilde{f}(h) = h_y$$
3) The model produces a vector h_y containing the corresponding predictions for each entry of the column vector h. Recall that these vectors are windows of the time series

Algorithm 1 Procrustes Subspace Adaptation (PSA)

Input: $\boldsymbol{X} = [\boldsymbol{x}_1, \dots, \boldsymbol{x}_n]^T \in \mathbb{R}^{n \times D}$, $\boldsymbol{Y}_S = [y_S^{(1)}, \dots, y_S^{(n)}] \in \mathbb{R}^n$, timestamps $\boldsymbol{t}_S \in \mathbb{R}^n$, subspace dimension k, window length L, learning rate η

Output: Model $\tilde{f} : \boldsymbol{x}_a \mapsto \tilde{\boldsymbol{h}}_Y$, embedding space $\boldsymbol{H}_{Z_t,\text{sub}}$

Find $\boldsymbol{H}_X = \mathcal{H}(\boldsymbol{X}) \in \mathbb{R}^{DL \times (n-L+1)}$ and $\boldsymbol{H}_{Y_S} = \mathcal{H}(\boldsymbol{Y}_S) \in \mathbb{R}^{DL \times (n-L+1)}$

$\boldsymbol{H}_{X,\text{sub}} \in \mathbb{R}^{DL \times k} \leftarrow \text{pca}(\boldsymbol{H}_X)$

Set initial guess for $\boldsymbol{H}_{Z_t,\text{sub}}(t=0) = \boldsymbol{H}_{X,\text{sub}}$

repeat

  Receive observations of $\boldsymbol{z}$ and append to $\boldsymbol{Z}_t$

  Record corresponding timestamps of $\boldsymbol{Z}_t$ as $\boldsymbol{t}_T$

  Truncate $\boldsymbol{X}$ to $\boldsymbol{X}_t$, containing $\boldsymbol{x}$ seen up to time t

  $\boldsymbol{X}_t = (\boldsymbol{X}_t - \bar{\boldsymbol{X}}_t)/\sigma_{X_t}$ ▷ Mean $\bar{\boldsymbol{X}}_t$, standard deviation σ_{X_t}

  $\boldsymbol{Z}_t = (\boldsymbol{Z}_t - \bar{\boldsymbol{Z}}_t)/\sigma_{Z_t}$ ▷ Mean $\bar{\boldsymbol{Z}}_t$, standard deviation σ_{Z_t}

  $\boldsymbol{H}_{X_t} = \mathcal{H}(\boldsymbol{X}_t)$

  $\boldsymbol{H}_{Z_t} = \mathcal{H}(\boldsymbol{Z}_t)$

  Define common timesteps for interpolation $\mathbf{t} = \text{sort}\left([\boldsymbol{t}_S | \boldsymbol{t}_T]^T\right)$

  $\tilde{\boldsymbol{X}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{X}_t)$

  $\tilde{\boldsymbol{Z}}_t \leftarrow \text{interpolate}(\mathbf{t}, \boldsymbol{Z}_t)$

  $\tilde{\boldsymbol{H}}_{X_t} = \mathcal{H}(\tilde{\boldsymbol{X}}_t)$

  $\tilde{\boldsymbol{H}}_{Z_t} = \mathcal{H}(\tilde{\boldsymbol{Z}}_t)$

  $\boldsymbol{H}_{X_t,\text{sub}} \leftarrow \text{pca}(\boldsymbol{H}_{X_t})$

  $\boldsymbol{H}_{Z_t,\text{sub}}(t) = \boldsymbol{H}_{Z_t,\text{sub}}(t-1) + \eta \tilde{\boldsymbol{h}}_{Z_t}(t) \left(\tilde{\boldsymbol{h}}_{Z_t}(t)\right)^T \boldsymbol{H}_{Z_t,\text{sub}}(t-1)$

  $\tilde{\boldsymbol{H}}_{X_t,\text{proj}} = \boldsymbol{H}_{X_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{X_t}$

  $\tilde{\boldsymbol{H}}_{Z_t,\text{proj}} = \boldsymbol{H}_{Z_t,\text{sub}}^T \tilde{\boldsymbol{H}}_{Z_t}$

  $\boldsymbol{U}\boldsymbol{S}\boldsymbol{V}^T = \tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{Z_t,\text{proj}}^T$

  $\boldsymbol{Q} = \boldsymbol{V}\boldsymbol{U}^T$

  $s = \dfrac{\text{tr}(\boldsymbol{S})}{\text{tr}(\tilde{\boldsymbol{H}}_{X_t,\text{proj}} \tilde{\boldsymbol{H}}_{X_t,\text{proj}}^T)}$

  $\boldsymbol{X}_a = s\boldsymbol{Q}(\boldsymbol{H}_{X_t,\text{sub}}^T \boldsymbol{H}_{X_t})$

  $\boldsymbol{Z}_a = \boldsymbol{H}_{Z_t,\text{sub}}^T \boldsymbol{H}_{Z_t}$

  $\tilde{f} = \arg\min_{\tilde{\theta}} \sum_{i=1}^{n} \mathcal{L}\left(\tilde{f}(\boldsymbol{x}_a^{(i)} | \tilde{\boldsymbol{\theta}}), \boldsymbol{h}_{Y_S}^{(i)}\right)$

  $t = t + 1$

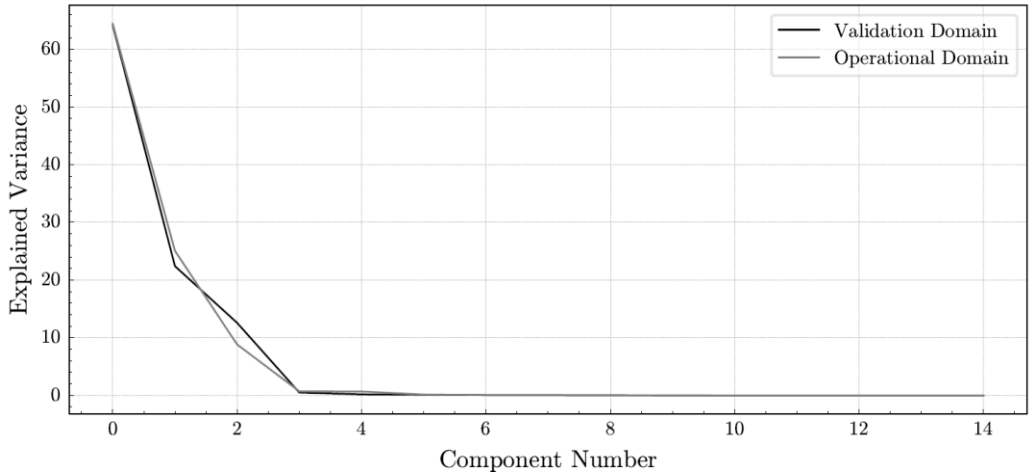
until termination or all $\boldsymbol{z}$ seen

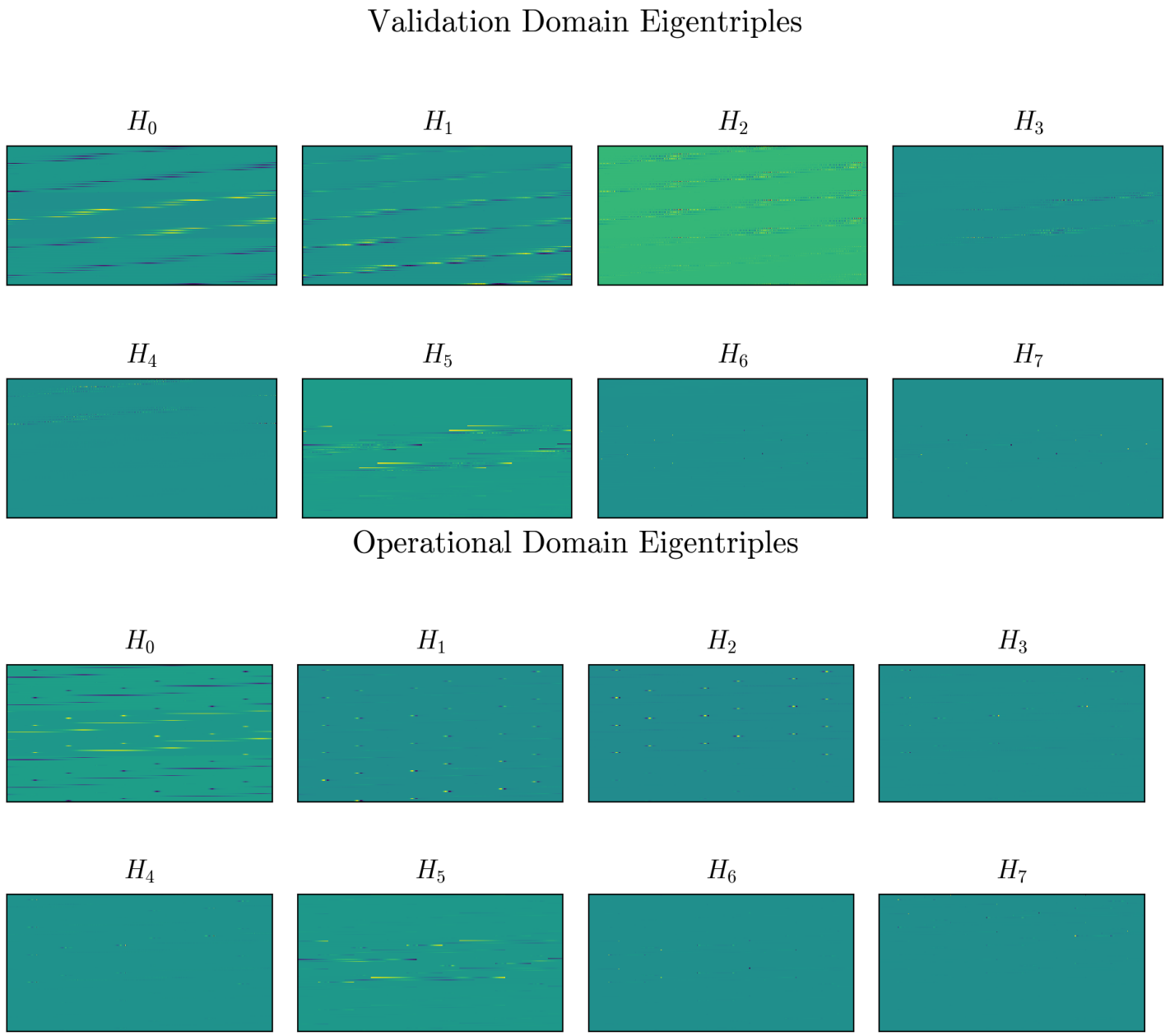
return $\tilde{f}$, $\boldsymbol{H}_{Z_t,\text{sub}}$

Appendix B: Principal Components and Eigentriples

- The validation domain/DRM mission subspace is found using PCA
- The operational domain/extrapolatory mission subspace is assumed to be streaming and found by Oja's algorithm
- Mission orbits
  - Validation domain: Apoapsis 12000 km, periapsis 97 km
  - Operational domain: Apoapsis 12000 km, periapsis 95 km
- **Subspace dimension was determined by taking the top left singular vectors that contributed to >95% of the explained variance (d=rank of matrix)**

$$\frac{\sigma_i^2}{\sum_{k=0}^{d-1} \sigma_k^2}$$





Appendix C: Theoretical Bounds in Domain Adaptation for Regression

Why do adaptation methods intended for image classification not extend to regression problems?

- Certain methods, like adversarial NNs, are built on learning theory that explicitly assumes discrete, finite/bounded loss functions to measure the discrepancy between domains. Applying these methods to a regression setting with a continuous valued loss function (like MSE) violates this assumption, which removes this theoretical guarantee of learning a good adaptation.
- This is not to say these methods won't work, but there's no way to gauge how adaptation performs for different domain discrepancies or know a priori if the method will be successful for a problem
- I alluded to this briefly, but a great example of how domain learning theory doesn't translate well to regression is in examining Ben-David et al.'s symmetric difference hypothesis distance [1], which can be approximated by what Mansour et al. call the discrepancy distance (this is the "proxy-A distance" in Ben-David et al.) [2]

$$\hat{d}_{\mathcal{H}\Delta\mathcal{H}} = 2 \max_{f \in \mathcal{H}} |\hat{R}(f, p_{\text{val}}) - \hat{R}(f, p_{\text{opr}})|$$

- Assuming an MSE loss and simple neural network (any model works, really), the optimization problem to solve $\hat{d}_{\mathcal{H}\Delta\mathcal{H}}$ is unbounded; there is no maxima because MSE increases indefinitely. Unless there is some constraint on the network weights, the weights would simply diverge to infinity
- This discrepancy distance can be used to establish a generalization bound (i.e., when a model can/cannot adapt to a domain) [1-2]. Evaluating the below expression gives a trivial result because the RHS is always greater than the LHS, meaning the bound is always true

$$\hat{R}(f, p_{\text{opr}}) \leq \hat{R}(f, p_{\text{val}}) + \frac{1}{2}\hat{d}_{\mathcal{H}\Delta\mathcal{H}} + \min_{f \in \mathcal{H}}[\hat{\mathrm{R}}(\mathrm{f}, \mathrm{p}_{\text{opr}}) + \hat{\mathrm{R}}(\mathrm{f}, \mathrm{p}_{\text{opr}})]$$

[1] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, "A theory of learning from different domains," *Mach Learn*, vol. 79, no. 1–2, pp. 151–175, May 2010, doi: 10.1007/s10994-009-5152-4.
[2] Y. Mansour, M. Mohri, and A. Rostamizadeh, Domain Adaptation: Learning Bounds and Algorithms, Nov. 2023. DOI: 10.48550/arXiv.0902.3430. arXiv: 0902.3430

Appendix D: Trajectory Simulation

- Data was generated using Falcone and Putnam's Aerobraking Trajectory Simulator (ABTS) (https://github.com/gfalcon2/Aerobraking-Trajectory-Simulator)
- Other software was considered, such as STK, Monte, GMAT, and AVS Basilisk, but ABTS was ultimately selected for its ability to efficiently simulate drag passages and ease of use
    - Falcone and Putnam simulate the trajectory using a closed-form approximate solution, rather than numerically integrating the equations of motion

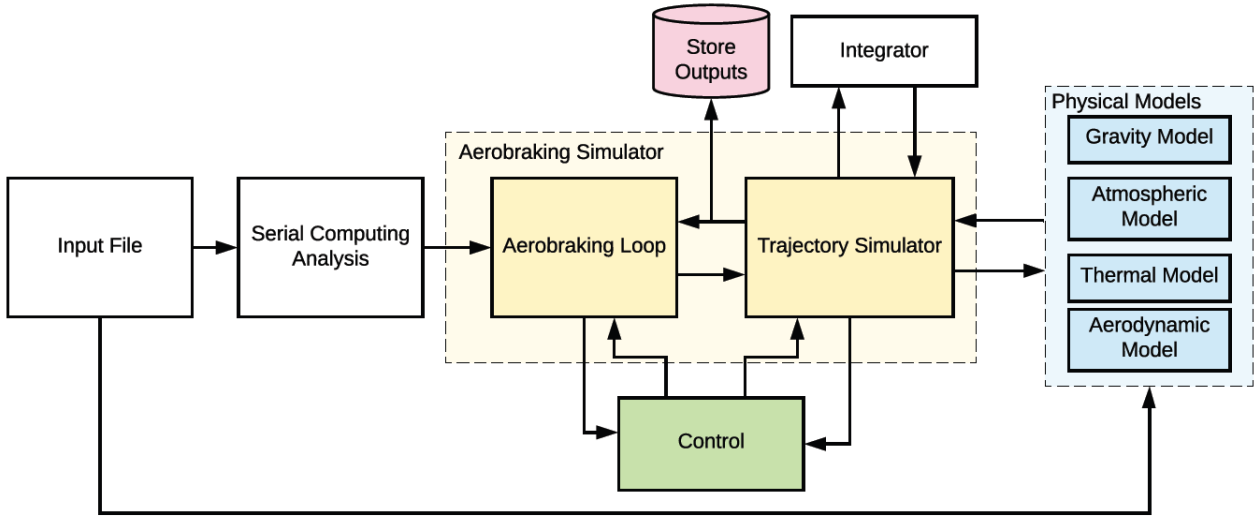


Fig. 3 Main architecture of ABTS.

Table 1 Input Parameters Categories

Mission Type	• Drag Passage • # Orbits • Aerobraking Campaign	**Aerodynamic Models**	• Constant • Mach-Dependent (Spacecraft) • Netwonian Flow Theory (Blunted-Cone)
Vehicle	• Spacecraft • Blunted Cone	**Thermal Models**	• Maxwellian Heat Transfer Theory (Spacecraft) • Sutton-Graves and Tauber-Sutton Approximation (Blunted-Cone)
Planet	• Mars • Earth • Venus	**Initial Conditions**	• Orbital Elements • Velocity, Flight-Path Angle, Altitude (if Drag-Passage)
Gravity Models	• Constant • Inverse-Squared Law • Inverse-Squared Law with J2 Effects	**Control Mode**	• Propulsive Maneuvers · Aerobraking Maneuvers · Drag Passage Thrust Maneuvers • Online Drag-Modulation Trajectory Control · Heat Rate Control · Heat Load Control · Heat Rate and Load Control
Atmospheric Models	• Constant • Exponential Law • Online Mars-GRAM 2010 (only for Mars)		
Monte Carlo Simulation	• Setting Parameters • Dispersion	**Miscellaneous**	• Plots Generation • Print Results • Save Results • Integrator Type • Integration Rate • Control Rate

Table 2 Mission Parameters

Mission Type	Required Parameters
Drag Passage	EI and AE vacuum altitude
Orbits	# of orbits
Aerobraking Campaign	-

Table 3 Vehicle Required Parameters

Vehicle	Required Parameters
General	• Dry Mass and Propellant Mass • I_{sp}, T and ϕ
Spacecraft	• Length and Height Main Bus • Length and Height Solar Panels • σ and α
Blunted Cone	• Cone Angle • Angle of Attack Angle • Node Radius • Base Radius

Table 4 Initial Condition Required Parameters

Initial Conditions	Required Parameters
Initial Conditions Type	• Apoaspsis, Periapsis • Velocity, Flight-Path Angle
General Initial Conditions	• Inclination • AOP • RAAN • Date (year, month, day, hours minutes, seconds)
Apoapsis/Periapsis/ Velocity/Flight-Path Angle	• Low Boundary • High Boundary • Step
Final Conditions	• Final Apoapsis Radius

Appendix E: Mars-GRAM 2010 Sample Output

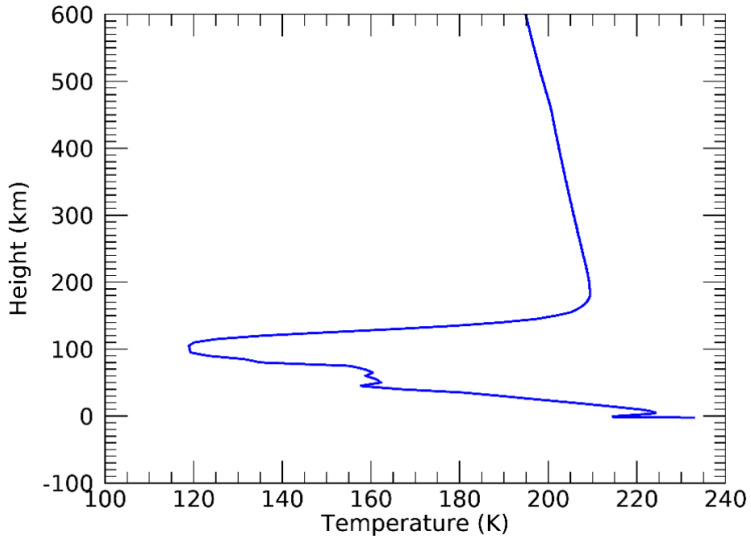


Figure 2. Height versus temperature from a sample Mars-GRAM output.

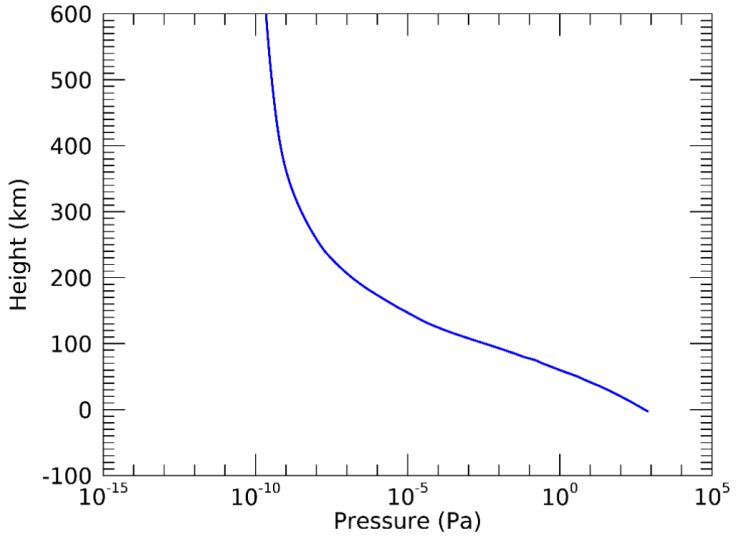


Figure 3. Height versus pressure from a sample Mars-GRAM output.

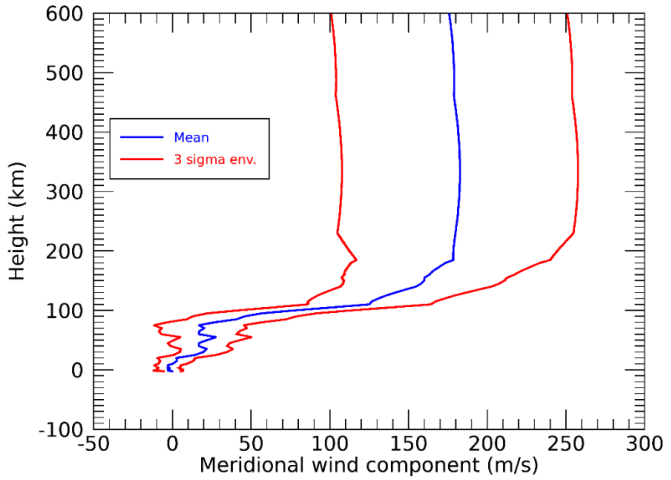


Figure 5. Three-sigma envelope of sample Mars-GRAM north (positive)/south (negative) meridional wind outputs for 32.333° N latitude and -117.8° E longitude.

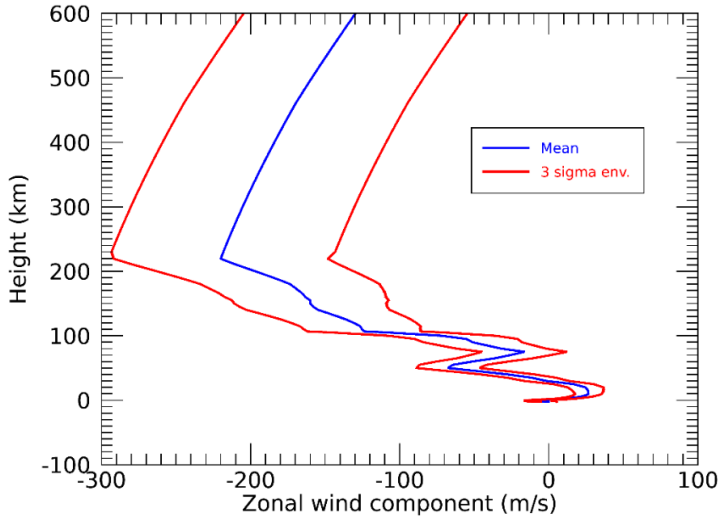


Figure 4. Three-sigma envelope of sample Mars-GRAM east (positive)/west (negative) zonal wind outputs for 32.333° N latitude and -117.8° E longitude.

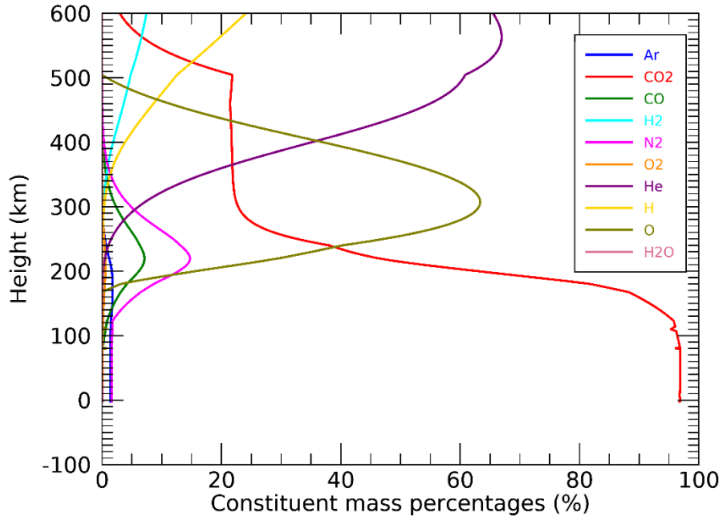


Figure 6. Sample Mars-GRAM mean constituent contributions by percent.

H. L. Justh, A. M. D. Cianciolo, and J. Hoffman, “Mars Global Reference Atmospheric Model (Mars-GRAM): User Guide”.

Appendix F: Free Molecule Flow and Heat Flux Prediction

Flow can be characterized as “continuum flow,” “slip flow,” “transition flow,” and “free molecule flow”, which correspond to density levels that are, respectively, ordinary, slightly rarefied, moderately rarefied, and highly rarefied [Schaaf and Chambre 1986]

ABTS assumes a highly rarefied flow and solves for the heat flux using the following expression:

$$\dot{Q} = \alpha\rho\mathcal{R}T\sqrt{\frac{\mathcal{R}T}{2\pi}}\left[\left(S^2 + \frac{\kappa+1}{2(\kappa-1)}\right)\left(e^{-\beta^2} + \beta\sqrt{\pi}(1+\operatorname{erf}\beta)\right) - \frac{1}{2}e^{-\beta^2}\right],$$

$$\beta = S\sin\gamma,$$

$$S = \frac{V}{\sqrt{2\mathcal{R}T}} = \sqrt{\frac{\kappa}{2}}M,$$

where α is the thermal accommodation coefficient, ρ is the atmospheric density, R is the specific gas constant, T is the freestream temperature, κ is the isentropic exponent, S is the molecular speed ratio, and M is the flow Mach number. The full derivation for this equation can be found in Schaaf and Chambre pp. 12-13

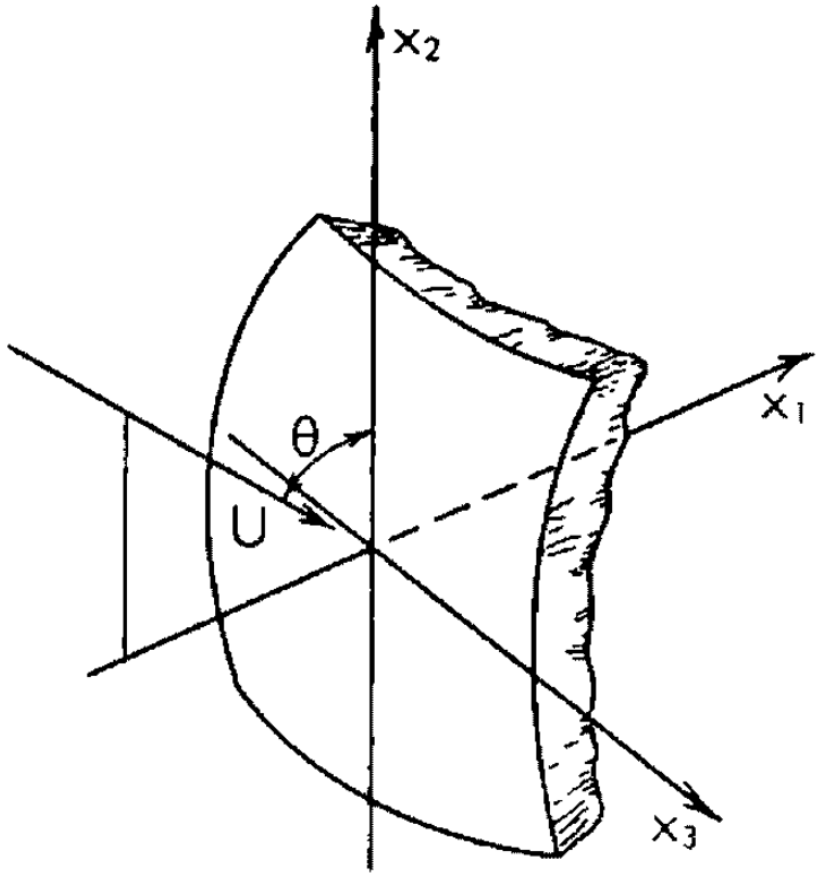
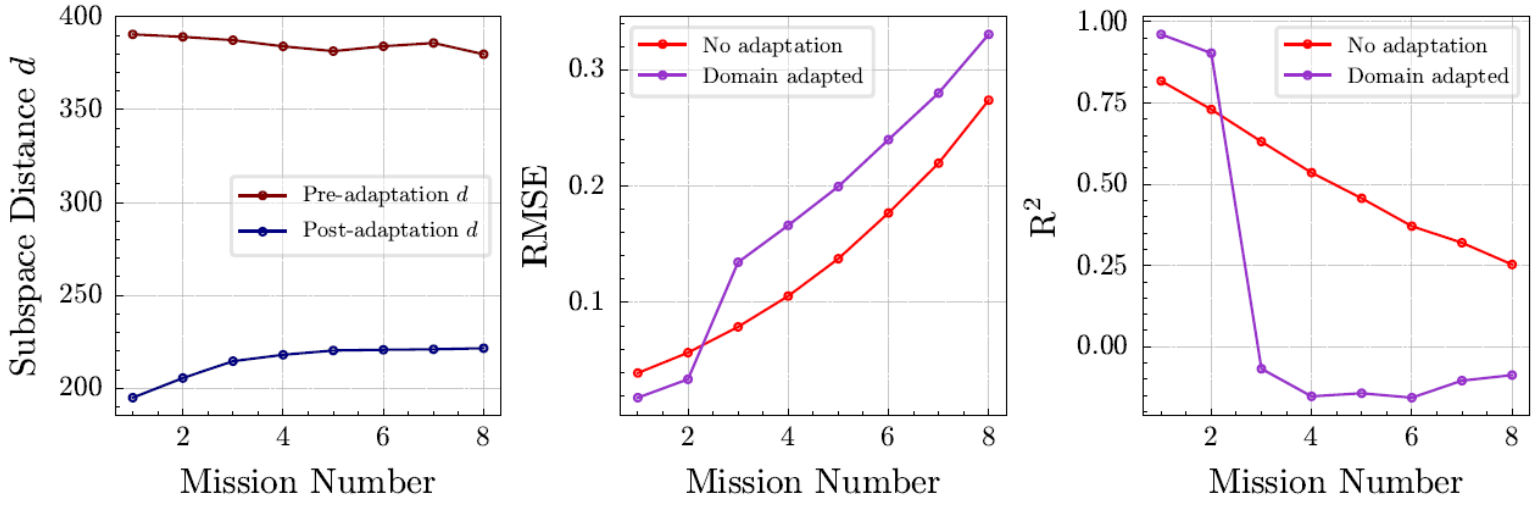


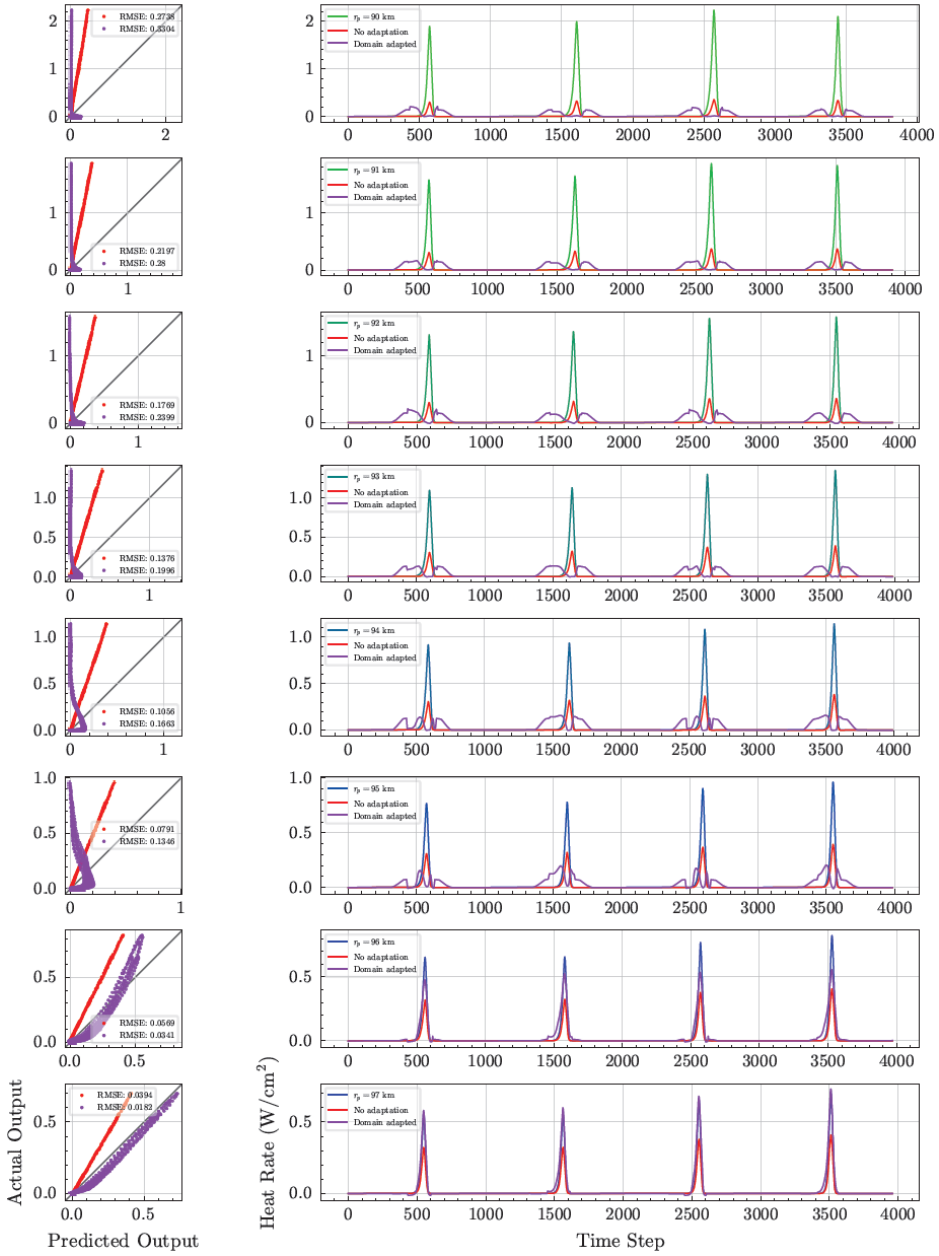
Fig. H,5. Coordinate system for free molecule flow.

S. A. Schaaf and P. L. Chambre, *Flow of Rarefied Gases*. Princeton University Press, 1961.

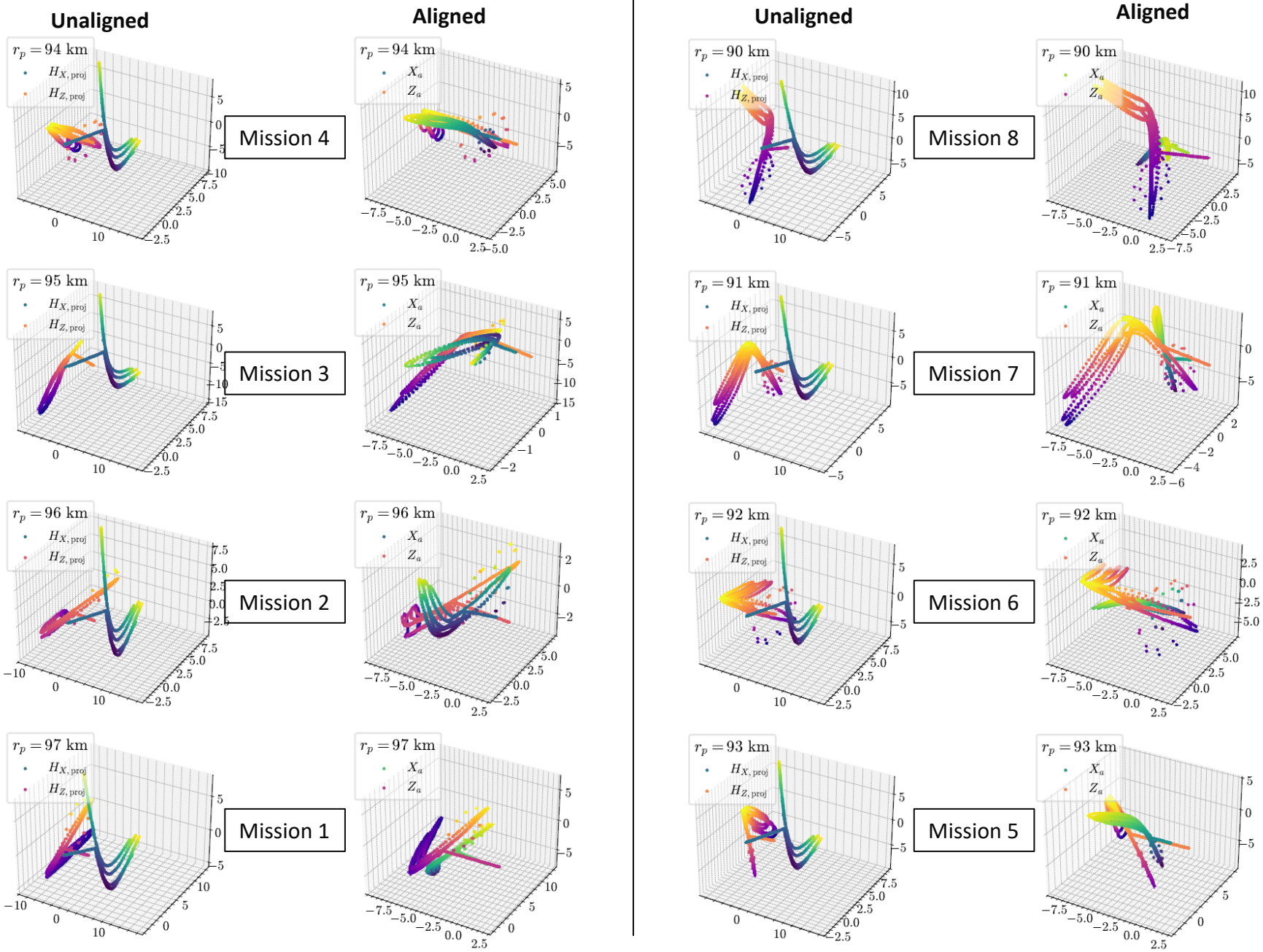
Appendix G.1: Data Removal Correspondence on Periapsis Shifted Missions

- Switching from time interpolation to removing data is not effective on the periapsis shifted missions
- At higher shifts, the manifold alignment struggles to orient the manifold correctly because the removal process assumes that similar timesteps, sampled at similar times in the mission, exist in both domains
- In reality, this is untrue because the sampling differs between missions as they become farther apart

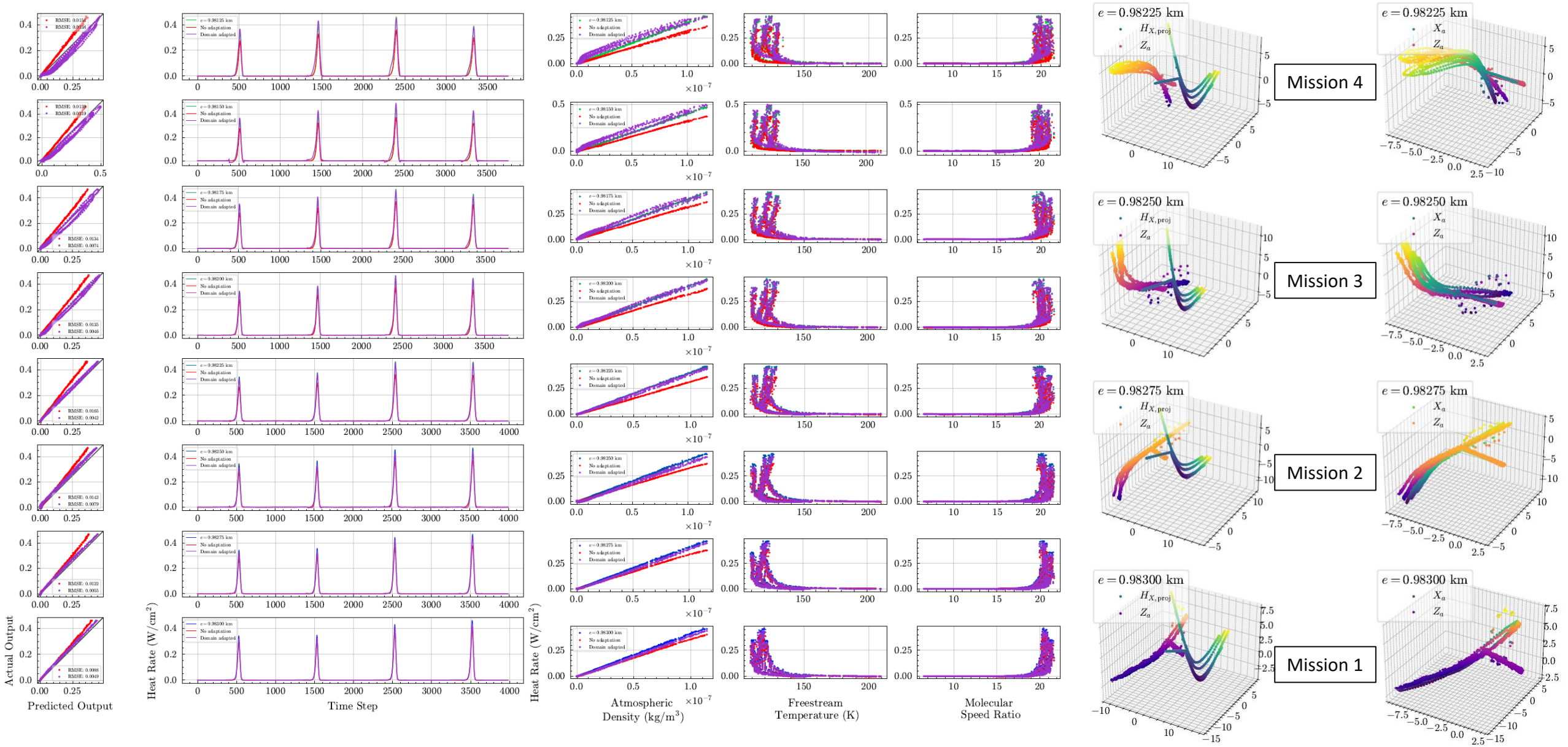




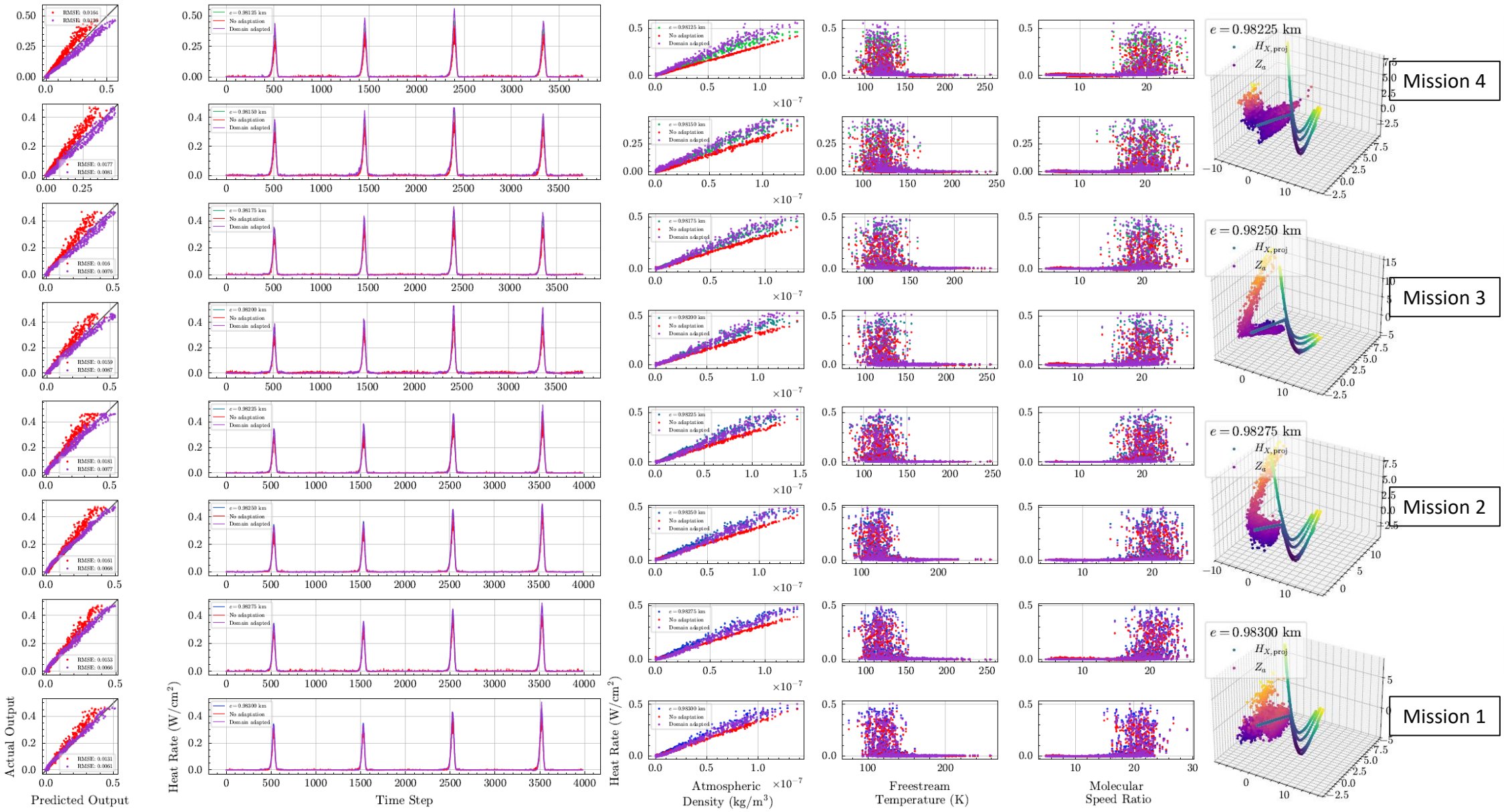
Appendix G.2: Data Removal Correspondence on Periapsis Shifted Missions: Manifolds



Appendix H.1: 1% AWGN, Eccentricity Shift with Fixed Semi-Latus Rectum



Appendix H.2: 10% AWGN, Eccentricity Shift with Fixed Semi-Latus Rectum



Slide 3

[1] E. Glaessgen and D. Stargel, “The digital twin paradigm for future nasa and u.s. air force vehicles,” in 53rd AIAA/ASME/ASCE/AHS/ASC Structures, Structural Dynamics and Materials Conference. AIAA, 2012. DOI: 10.2514/6.2012-1818.

[2] Office of the Army Chief Information Officer, Army Digital Transformation Strategy, Oct. 2021. 99

[3] J. M. Marlowe, C. L. Haymes, and P. L. Murphy, “NASA’s Digital Transformation Strategic Framework & Implementation Approach,” NASA, Technical Memorandum 20220018538, Dec. 2022.

[4] Space Force Chief Technology and Innovation Officer, “U.S. Space Force Vision for a Digital Service,” U.S. Space Force, Tech. Rep., 2021.

[5] Office of the Deputy Assistant Secretary of Defense for Systems Engineering, “U.S. Department of Defense Digital Engineering Strategy,” U.S. Department of Defense, Tech. Rep., 2018.

[6] P. Zimmerman, T. Gilbert, and F. Salvatore, “Digital engineering transformation across the Department of Defense,” The Journal of Defense Modeling and Simulation: Applications, Methodology, Technology, vol. 16, no. 4, pp. 325–338, Oct. 2019. DOI: 10.1177/1548512917747050.

[7] Office of the Under Secretary of Defense for Research and Engineering, DOD INSTRUCTION 5000.97 DIGITAL ENGINEERING.

Slide 4

[1] National Academies of Sciences, Engineering, and Medicine, Foundational Research Gaps and Future Directions for Digital Twins. Washington, D.C.: National Academies Press, Mar. 2024, p. 26 894, ISBN: 978-0-309-70042-9. DOI: 10.17226/ 26894.
[2] AIAA Digital Engineering Integration Committee, Digital Thread: Definition, Value, and Reference Model, 2023.
[3] AIAA Digital Engineering Integration Committee, DIGITAL TWIN: REFERENCE MODEL, REALIZATIONS & RECOMMENDATIONS, 2023.
[4] A. Thelen et al., "A comprehensive review of digital twin — part 1: Modeling and twinning enabling technologies," Structural and Multidisciplinary Optimization, vol. 65, no. 12, p. 354, Dec. 2022. DOI: 10.1007/s00158-022-03425-4.
[5] A. Thelen et al., "A comprehensive review of digital twin—part 2: Roles of uncertainty quantification and optimization, a battery digital twin, and perspectives," Structural and Multidisciplinary Optimization, vol. 66, no. 1, p. 1, Jan. 2023. DOI: 10.1007/s00158-022-03410-x.

Slide 5

[1] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, P. Perdikaris, S. Wang, and L. Yang, "Physics-informed machine learning," Nature Reviews Physics, vol. 3, no. 6, pp. 422– 440, May 2021. DOI: 10.1038/s42254-021-00314-5.
[2] National Academies of Sciences, Engineering, and Medicine, Foundational Research Gaps and Future Directions for Digital Twins. Washington, D.C.: National Academies Press, Mar. 2024, p. 26 894, ISBN: 978-0-309-70042-9. DOI: 10.17226/ 26894.
[3] H. Wang *et al.*, "Scientific discovery in the age of artificial intelligence," *Nature*, vol. 620, no. 7972, pp. 47–60, Aug. 2023, doi: 10.1038/s41586-023-06221-2.

Slide 6

[1] M. Mohri, A. Rostamizadeh, and A. Talwalkar, Foundations of Machine Learning (Adaptive Computation and Machine Learning Series), Second edition. Cambridge, MA: MIT Press, 2018.

[2] W. L. Oberkampf, T. G. Trucano, and C. Hirsch, “Verification, validation, and predictive capability in computational engineering and physics,” Applied Mechanics Reviews, vol. 57, no. 5, pp. 345–384, Dec. 2004. DOI: 10.1115/1.1767847.

[3] B. Thacker and S. Doebling, “Concepts of Model Verification and Validation,” Los Alamos National Laboratory, Tech. Rep. LA-14167-MS, Oct. 2004.

[4] R. G. Easterling, “Measuring the Predictive Capability of Computational Methods: Principles and Methods, Issues and Illustrations,” Sandia National Laboratories, Tech. Rep. SAND2001-0243, Feb. 2001.

Slide 9

[1] M. M. Munk and S. A. Moon, “Aerocapture Technology Development Overview,” in 2008 IEEE Aerospace Conference, Big Sky, MT, USA: IEEE, Mar. 2008, pp. 1– 7. DOI: 10.1109/AERO.2008.4526545.

[2] NASA, “NASA DRAFT Space Technology Roadmap - Modeling and Simulation,”National Aeronautics and Space Administration, Tech. Rep., 2011.

[3] J. L. Prince, J. A. Dec, and R. H. Tolson, “Autonomous Aerobraking Using Thermal Response Surface Analysis,” Journal of Spacecraft and Rockets, vol. 46, no. 2, pp. 292–298, Mar. 2009. DOI: 10.2514/1.32793.

[4] R. H. Tolson et al., “Application of Accelerometer Data to Mars Odyssey Aerobraking and Atmospheric Modeling,” Journal of Spacecraft and Rockets, vol. 42, no. 3, pp. 435–443, May 2005. DOI: 10.2514/1.15173.

[5] R. Tolson et al., “Atmospheric Modeling Using Accelerometer Data During Mars Reconnaissance Orbiter Aerobraking Operations,” Journal of Spacecraft and Rockets, vol. 45, no. 3, pp. 511–518, May 2008. DOI: 10.2514/1.34301.

Slide 10

[1] M. M. Munk and S. A. Moon, “Aerocapture Technology Development Overview,” in 2008 IEEE Aerospace Conference, Big Sky, MT, USA: IEEE, Mar. 2008, pp. 1– 7. DOI: 10.1109/AERO.2008.4526545.

[2] J. L. Prince, J. A. Dec, and R. H. Tolson, “Autonomous Aerobraking Using Thermal Response Surface Analysis,” *Journal of Spacecraft and Rockets*, vol. 46, no. 2, pp. 292–298, Mar. 2009, doi: 10.2514/1.32793.

[3] D. G. Murri, R. W. Powell, and J. L. Prince, “Development of Autonomous Aerobraking (Phase 1),” NASA, Langley Research Center, Technical Memorandum 20120001988, 2012.[5]

[4] D. G. Murri, “Development of Autonomous Aerobraking (Phase 2),” NASA, Langley Research Center, Technical Memorandum 20140001125, 2013.

[5] J. A. Dec and M. N. Thornblom, “Autonomous Aerobraking: Thermal Analysis and Response Surface Development,” in 2011 AAS/AIAA Astrodynamics Specialist Conference, Girdwood, Alaska: American Astronautical Society, American Inst. of Aeronautics and Astronautics, 2011.

Slide 11

[1] G. Falcone and Z. R. Putnam, “Design and Development of an Aerobraking Trajectory Simulation Tool,” in *AIAA Scitech 2021 Forum*, VIRTUAL EVENT: American Institute of Aeronautics and Astronautics, Jan. 2021. doi: 10.2514/6.2021-1065.

[2] H. L. Justh, “Mars Global Reference Atmospheric Model 2010 Version: Users Guide,” *Users Guide*, 2010.

Slide 14

[1] K. Weiss, T. M. Khoshgoftaar, and D. Wang, “A survey of transfer learning,” *J Big Data*, vol. 3, no. 1, p. 9, Dec. 2016, doi: 10.1186/s40537-016-0043-6.
[2] W. M. Kouw and M. Loog, “An introduction to domain adaptation and transfer learning,” Jan. 14, 2019, *arXiv*: arXiv:1812.11806. Accessed: Nov. 07, 2024. [Online]. Available: http://arxiv.org/abs/1812.11806

[3] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, “A theory of learning from different domains,” *Mach Learn*, vol. 79, no. 1–2, pp. 151–175, May 2010, doi: 10.1007/s10994-009-5152-4.

[4] B. Gong, Y. Shi, F. Sha, and K. Grauman, “Geodesic flow kernel for unsupervised domain adaptation,” in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, June 2012, pp. 2066–2073. doi: 10.1109/CVPR.2012.6247911.

[5] B. Fernando, A. Habrard, M. Sebban, and T. Tuytelaars, “Unsupervised Visual Domain Adaptation Using Subspace Alignment,” in *2013 IEEE International Conference on Computer Vision*, Sydney, Australia: IEEE, Dec. 2013, pp. 2960–2967. doi: 10.1109/ICCV.2013.368.

[6] J. Hoffman, T. Darrell, and K. Saenko, “Continuous Manifold Based Adaptation for Evolving Visual Domains,” in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, Columbus, OH, USA: IEEE, June 2014, pp. 867–874. doi: 10.1109/CVPR.2014.116.

[7] Y. Ganin *et al.*, “Domain-Adversarial Training of Neural Networks,” *Journal of Machine Learning Research*, vol. 17, pp. 1–35, Jan. 2016, doi: 10.1007/978-3-319-58347-1_10.

[8] K. K. Chen, J. H. Tu, and C. W. Rowley, “Variants of dynamic mode decomposition: boundary conditions, Koopman, and Fourier analyses”.

Slide 15

[1] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, “A theory of learning from different domains,” *Mach Learn*, vol. 79, no. 1–2, pp. 151–175, May 2010, doi: 10.1007/s10994-009-5152-4.
[2] W. L. Oberkampf, T. G. Trucano, and C. Hirsch, “Verification, validation, and predictive capability in computational engineering and physics,” Applied Mechanics Reviews, vol. 57, no. 5, pp. 345–384, Dec. 2004. DOI: 10.1115/1.1767847.

Slide 16

[1] J. Huang, A. Gretton, K. Borgwardt, B. Sch¨olkopf, and A. Smola, “Correcting Sample Selection Bias by Unlabeled Data,” in Advances in Neural Information Processing Systems, vol. 19, MIT Press, 2006.
[2] M. Sugiyama, S. Nakajima, H. Kashima, P. Buenau, and M. Kawanabe, “Direct Importance Estimation with Model Selection and Its Application to Covariate Shift Adaptation,” in Advances in Neural Information Processing Systems, vol. 20, Curran Associates, Inc., 2007. 104
[3] M. Loog, “Nearest neighbor-based importance weighting,” in 2012 IEEE International Workshop on Machine Learning for Signal Processing, Sep. 2012, pp. 1–6. DOI: 10.1109/MLSP.2012.6349714.

[4] B. Gong, Y. Shi, F. Sha, and K. Grauman, “Geodesic flow kernel for unsupervised domain adaptation,” in *2012 IEEE Conference on Computer Vision and Pattern Recognition*, June 2012, pp. 2066–2073. doi: 10.1109/CVPR.2012.6247911.

[5] B. Fernando, A. Habrard, M. Sebban, and T. Tuytelaars, “Unsupervised Visual Domain Adaptation Using Subspace Alignment,” in *2013 IEEE International Conference on Computer Vision*, Sydney, Australia: IEEE, Dec. 2013, pp. 2960–2967. doi: 10.1109/ICCV.2013.368.

[6] J. Hoffman, T. Darrell, and K. Saenko, “Continuous Manifold Based Adaptation for Evolving Visual Domains,” in *2014 IEEE Conference on Computer Vision and Pattern Recognition*, Columbus, OH, USA: IEEE, June 2014, pp. 867–874. doi: 10.1109/CVPR.2014.116.

[7] Y. Ganin *et al.*, “Domain-Adversarial Training of Neural Networks,” *Journal of Machine Learning Research*, vol. 17, pp. 1–35, Jan. 2016, doi: 10.1007/978-3-319-58347-1_10.
[8] M. Long, Z. Cao, J.Wang, and M. I. Jordan, “Conditional adversarial domain adaptation,” in Advances in Neural Information Processing Systems, vol. 31, 2018.

[9] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks,” in 2017 IEEE International Conference on Computer Vision (ICCV), Venice: IEEE, Oct. 2017, pp. 2242–2251, ISBN: 978-1-5386-1032-9. DOI: 10.1109/ICCV.2017.244.
[10] J. Hoffman et al., “CyCADA: Cycle-Consistent Adversarial Domain Adaptation,” in Proceedings of the 35th International Conference on Machine Learning, PMLR, Jul. 2018, pp. 1989–1998.

[11] E. Tzeng, J. Hoffman, K. Saenko, and T. Darrell, “Adversarial Discriminative Domain Adaptation,” in 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI: IEEE, Jul. 2017, pp. 2962–2971, ISBN: 978-1-5386- 0457-1. DOI: 10.1109/CVPR.2017.316. 105

Slide 17

[1] Y. Ganin *et al.*, “Domain-Adversarial Training of Neural Networks,” *Journal of Machine Learning Research*, vol. 17, pp. 1–35, Jan. 2016, doi: 10.1007/978-3-319-58347-1_10.

[2] S. Ben-David, J. Blitzer, K. Crammer, and F. Pereira, “Analysis of Representations for Domain Adaptation,” in *Advances in Neural Information Processing Systems*, MIT Press, 2006. Accessed: Feb. 17, 2025. [Online]. Available: https://papers.nips.cc/paper_files/paper/2006/hash/b1b0432ceafb0ce714426e9114852ac7-Abstract.html

[3] S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, “A theory of learning from different domains,” *Mach Learn*, vol. 79, no. 1–2, pp. 151–175, May 2010, doi: 10.1007/s10994-009-5152-4.

[4] S. Ben-David, T. Lu, T. Luu, and D. Pal, “Impossibility Theorems for Domain Adaptation,” in *Proceedings of the Thirteenth International Conference on Artificial Intelligence and Statistics*, JMLR Workshop and Conference Proceedings, Mar. 2010, pp. 129–136. Accessed: Feb. 10, 2025. [Online]. Available: https://proceedings.mlr.press/v9/david10a.html

[5] C. Cortes and M. Mohri, “Domain Adaptation in Regression,” in *Algorithmic Learning Theory*, vol. 6925, J. Kivinen, C. Szepesvári, E. Ukkonen, and T. Zeugmann, Eds., in Lecture Notes in Computer Science, vol. 6925. , Berlin, Heidelberg: Springer Berlin Heidelberg, 2011, pp. 308–323. doi: 10.1007/978-3-642-24412-4_25.

[6] J.-Y. Zhu, T. Park, P. Isola, and A. A. Efros, “Unpaired Image-to-Image Translation Using Cycle-Consistent Adversarial Networks,” in 2017 IEEE International Conference on Computer Vision (ICCV), Venice: IEEE, Oct. 2017, pp. 2242–2251, ISBN: 978-1-5386-1032-9. DOI: 10.1109/ICCV.2017.244.

Slide 18

[1] Takens, F. (1981). Detecting strange attractors in turbulence. In: Rand, D., Young, LS. (eds) Dynamical Systems and Turbulence, Warwick 1980. Lecture Notes in Mathematics, vol 898. Springer, Berlin, Heidelberg. https://doi.org/10.1007/BFb0091924

[2] J.-N. Juang and R. S. Pappa, "An eigensystem realization algorithm for modal parameter identification and model reduction," *Journal of Guidance, Control, and Dynamics*, vol. 8, no. 5, pp. 620–627, Sept. 1985, doi: 10.2514/3.20031.

[3] D. S. Broomhead and R. Jones, "Time-Series Analysis," *Proceedings of the Royal Society of London. Series A, Mathematical and Physical Sciences*, vol. 423, no. 1864, pp. 103–121, 1989.

[4] C. Wang and S. Mahadevan, "Manifold alignment using Procrustes analysis," in *Proceedings of the 25th international conference on Machine learning - ICML '08*, Helsinki, Finland: ACM Press, 2008, pp. 1120–1127. doi: 10.1145/1390156.1390297.

Slide 19

[1] J.-N. Juang and R. S. Pappa, "An eigensystem realization algorithm for modal parameter identification and model reduction," *Journal of Guidance, Control, and Dynamics*, vol. 8, no. 5, pp. 620–627, Sept. 1985, doi: 10.2514/3.20031.

[2] S. L. Brunton, B.W. Brunton, J. L. Proctor, E. Kaiser, and J. N. Kutz, "Chaos as an intermittently forced linear system," Nature Communications, vol. 8, no. 1, p. 19, May 2017. DOI: 10.1038/s41467-017-00030-8.

[3] D. S. Broomhead and G. P. King, "Extracting qualitative dynamics from experimental data," *Physica D: Nonlinear Phenomena*, vol. 20, no. 2–3, pp. 217–236, June 1986, doi: 10.1016/0167-2789(86)90031-X.

[4] E. R. Deyle and G. Sugihara, "Generalized Theorems for Nonlinear State Space Reconstruction," PLoS ONE, vol. 6, no. 3, M. Oresic, Ed., e18295, Mar. 2011. DOI: 10.1371/journal.pone.0018295.

Slide 22

[1] C. Wang and S. Mahadevan, "Manifold alignment using Procrustes analysis," in *Proceedings of the 25th international conference on Machine learning - ICML '08*, Helsinki, Finland: ACM Press, 2008, pp. 1120–1127. doi: 10.1145/1390156.1390297.

Slide 24

[1] L. Balzano, Y. Chi, and Y. M. Lu, "Streaming PCA and Subspace Tracking: The Missing Data Case," Proceedings of the IEEE, vol. 106, no. 8, pp. 1293–1310, Aug. 2018. DOI: 10.1109/JPROC.2018.2847041.

Slide 34

[1] G. Falcone and Z. R. Putnam, “Design and Development of an Aerobraking Trajectory Simulation Tool,” in AIAA Scitech 2021 Forum, VIRTUAL EVENT: American Institute of Aeronautics and Astronautics, Jan. 2021, ISBN: 978-1-62410-609-5. DOI: 10.2514/6.2021-1065.

[2] H. L. Justh, “Mars Global Reference Atmospheric Model 2010 Version: Users Guide,” Users Guide, 2010.

[3] D. Savage, F. O’Donnell, and M. Hardin, “2001 Mars Odyssey Arrival,” NASA, Press Kit, 2001.

Slide 35

[1] R. Turner et al., “Bayesian optimization is superior to random search for machine learning hyperparameter tuning: Analysis of the black-box optimization challenge 2020,” CoRR, vol. abs/2104.10201, 2021. arXiv: 2104.10201.

[2] R. Garnett, Bayesian Optimization. Cambridge University Press, 2023.

[3] P. I. Frazier, A tutorial on bayesian optimization, 2018. arXiv: 1807.02811 [stat.ML].

[4] H. Li, Z. Xu, G. Taylor, C. Studer, and T. Goldstein, Visualizing the loss landscape of neural nets, 2018. arXiv: 1712.09913 [cs.LG].

[5] Z. Yao, A. Gholami, K. Keutzer, and M. Mahoney, PyHessian: Neural Networks Through the Lens of the Hessian, Mar. 2020. DOI: 10.48550/arXiv.1912.07145. arXiv: 1912.07145 [cs].